

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

数据库

聚合

内存管理

树索引

排序

查询规划

索引构建

分组

并发控制

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**

称为 **AI 内容创作者**？回复 [添砖加瓦]

09 | Index Concurrency



Intro to Database Systems
15-445/15-645
Fall 2019

AP

Andy Pavlo
Computer Science
Carnegie Mellon University

ADMINISTRIVIA

Project #1 is due Fri Sept 27th @ 11:59pm

Homework #2 is due Mon Sept 30th @ 11:59pm

Project #2 will be released Mon Sept 30th

OBSERVATION

We assumed that all the data structures that we have discussed so far are single-threaded.

But we need to allow multiple threads to safely access our data structures to take advantage of additional CPU cores and hide disk I/O stalls.



They Don't Do This!

CONCURRENCY CONTROL

A **concurrency control** protocol is the method that the DBMS uses to ensure "correct" results for concurrent operations on a shared object.

A protocol's correctness criteria can vary:

- **Logical Correctness:** Can I see the data that I am supposed to see?
- **Physical Correctness:** Is the internal representation of the object sound?

TODAY'S AGENDA

Latches Overview

Hash Table Latching

B+Tree Latching

Leaf Node Scans

Delayed Parent Updates



LOCKS VS. LATCHES

Locks

- Protects the database's logical contents from other txns.
- Held for txn duration.
- Need to be able to rollback changes.

Latches

- Protects the critical sections of the DBMS's internal data structure from other threads.
- Held for operation duration.
- Do not need to be able to rollback changes.

LOCKS VS. LATCHES

Lecture 17

Locks

Separate...	User transactions
Protect...	Database Contents
During...	Entire Transactions
Modes...	Shared, Exclusive, Update, Intention
Deadlock	Detection & Resolution
...by...	Waits-for, Timeout, Aborts
Kept in...	Lock Manager

Latches

Threads
In-Memory Data Structures
Critical Sections
Read, Write
Avoidance
Coding Discipline
Protected Data Structure

Source: [Goetz Graefe](#)

LATCH MODES

Read Mode

- Multiple threads can read the same object at the same time.
- A thread can acquire the read latch if another thread has it in read mode.

Write Mode

- Only one thread can access the object.
- A thread cannot acquire a write latch if another thread holds the latch in any mode.

Compatibility Matrix

	Read	Write
Read	✓	X
Write	X	X

LATCH IMPLEMENTATIONS

Approach #1: Blocking OS Mutex

- Simple to use
- Non-scalable (about 25ns per lock/unlock invocation)
- Example: **std::mutex**

```
std::mutex m;  
:  
m.lock();  
// Do something special...  
m.unlock();
```

LATCH IMPLEMENTATIONS

Approach #2: Test-and-Set Spin Latch (TAS)

- Very efficient (single instruction to latch/unlatch)
- Non-scalable, not cache friendly
- Example: `std::atomic<T>`

std::atomic<bool>

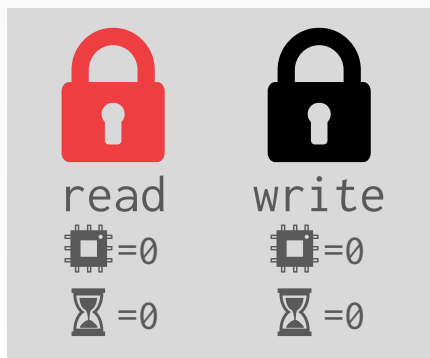
```
std::atomic_flag latch;  
:  
while (latch.test_and_set(...)) {  
    // Retry? Yield? Abort?  
}
```

LATCH IMPLEMENTATIONS

Approach #3: Reader-Writer Latch

- Allows for concurrent readers
- Must manage read/write queues to avoid starvation
- Can be implemented on top of spinlocks

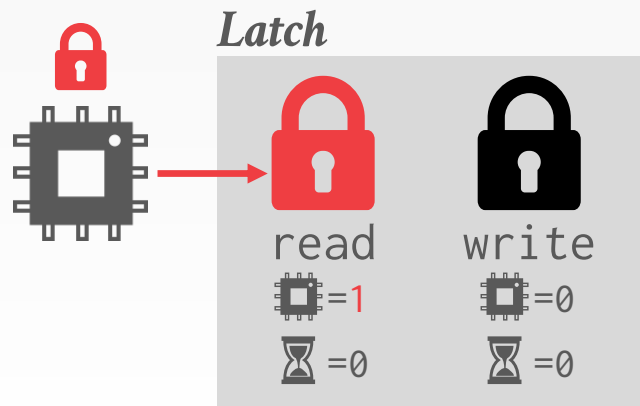
Latch



LATCH IMPLEMENTATIONS

Approach #3: Reader-Writer Latch

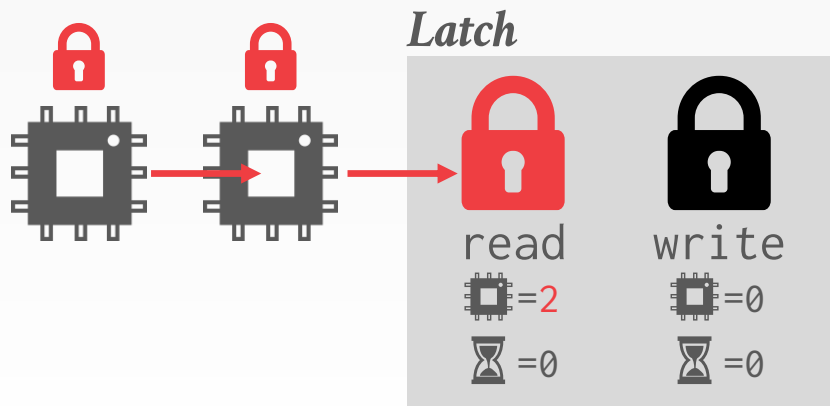
- Allows for concurrent readers
- Must manage read/write queues to avoid starvation
- Can be implemented on top of spinlocks



LATCH IMPLEMENTATIONS

Approach #3: Reader-Writer Latch

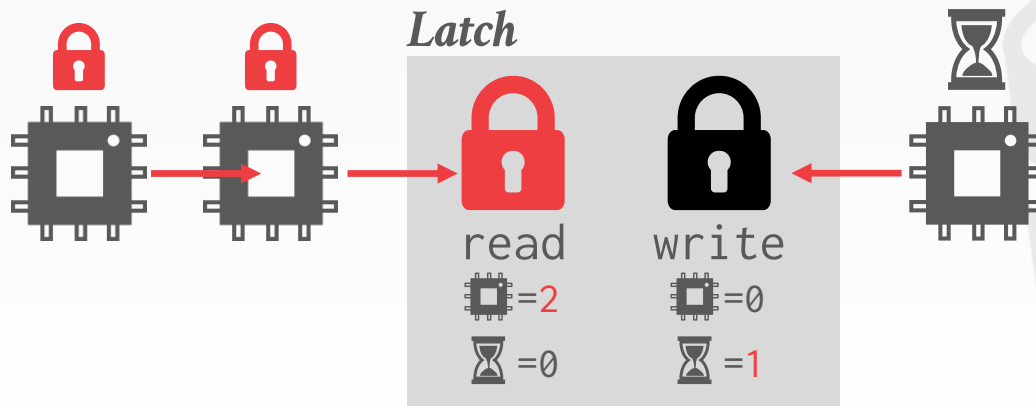
- Allows for concurrent readers
- Must manage read/write queues to avoid starvation
- Can be implemented on top of spinlocks



LATCH IMPLEMENTATIONS

Approach #3: Reader-Writer Latch

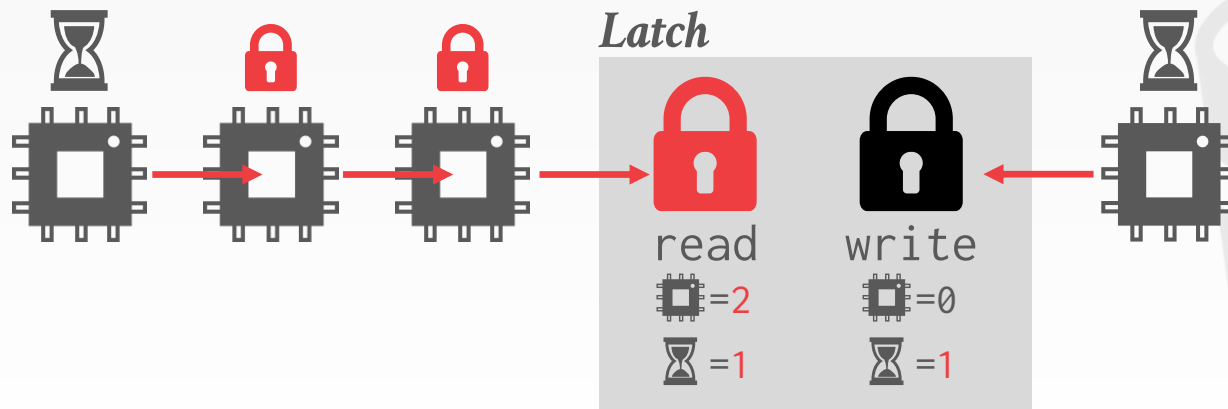
- Allows for concurrent readers
- Must manage read/write queues to avoid starvation
- Can be implemented on top of spinlocks



LATCH IMPLEMENTATIONS

Approach #3: Reader-Writer Latch

- Allows for concurrent readers
- Must manage read/write queues to avoid starvation
- Can be implemented on top of spinlocks



HASH TABLE LATCHING

Easy to support concurrent access due to the limited ways threads access the data structure.

- All threads move in the same direction and only access a single page/slot at a time.
- Deadlocks are not possible.

To resize the table, take a global latch on the entire table (i.e., in the header page).

HASH TABLE LATCHING

Approach #1: Page Latches

- Each page has its own reader-write latch that protects its entire contents.
- Threads acquire either a read or write latch before they access a page.

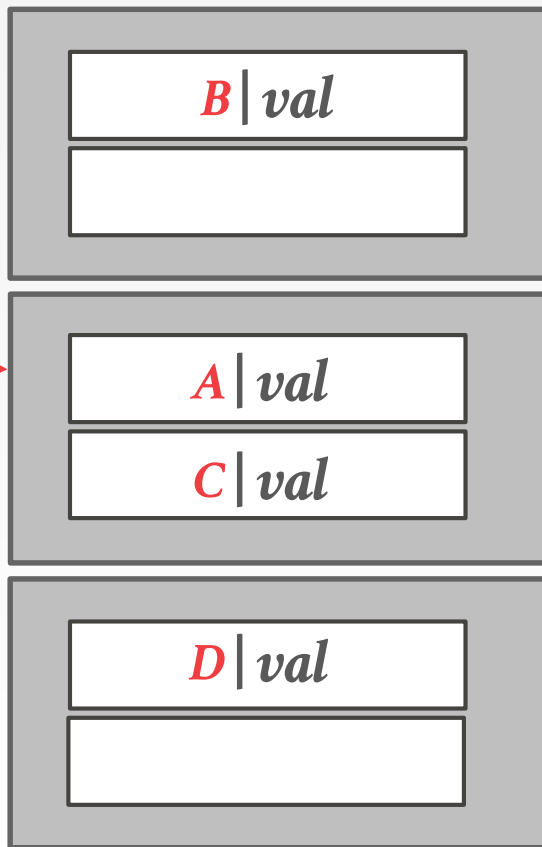
Approach #2: Slot Latches

- Each slot has its own latch.
- Can use a single mode latch to reduce meta-data and computational overhead.

HASH TABLE – PAGE LATCHES

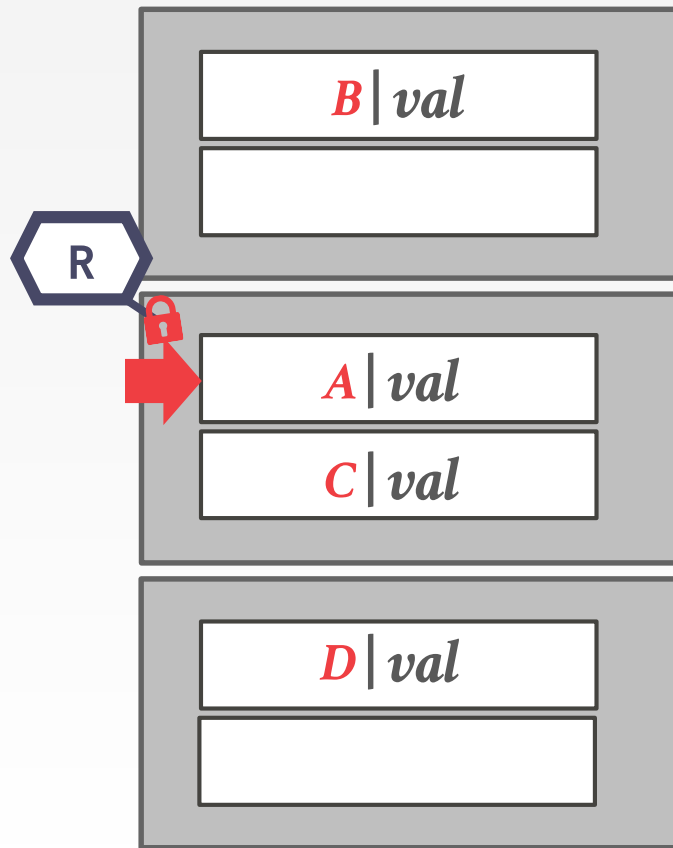
T_1 : Find D

$hash(D)$ ●



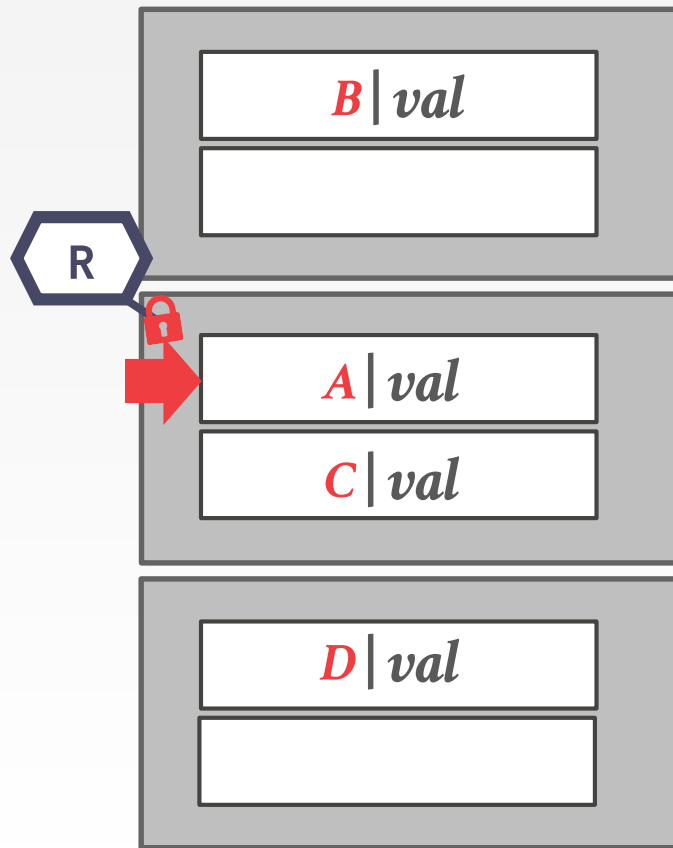
HASH TABLE – PAGE LATCHES

T_1 : Find D
hash(D)

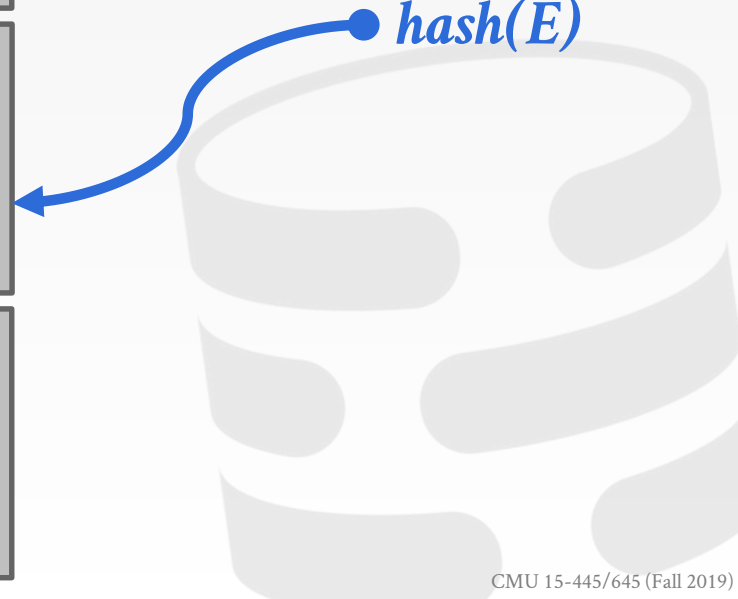


HASH TABLE – PAGE LATCHES

T_1 : Find D
hash(D)

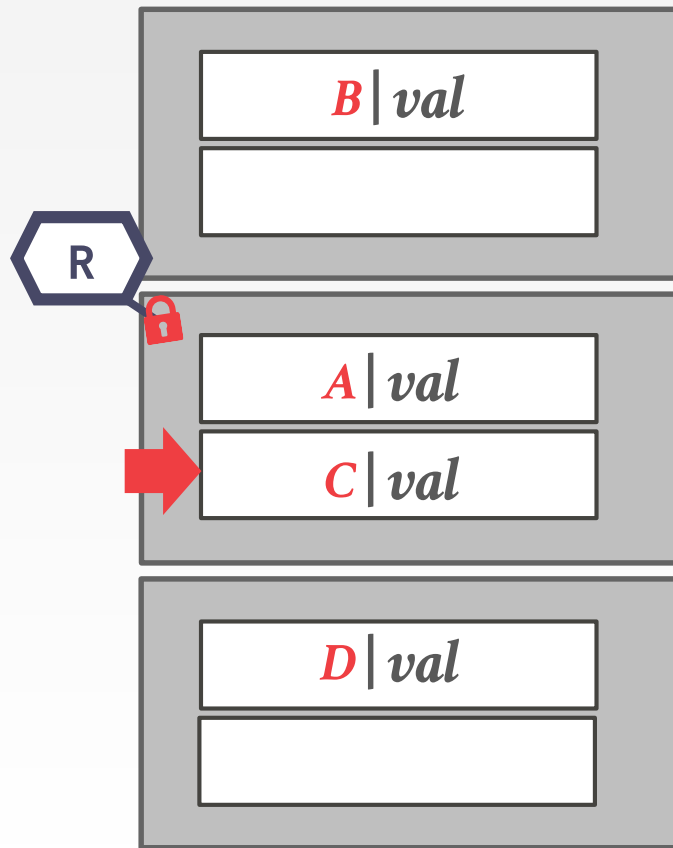


T_2 : Insert E
hash(E)



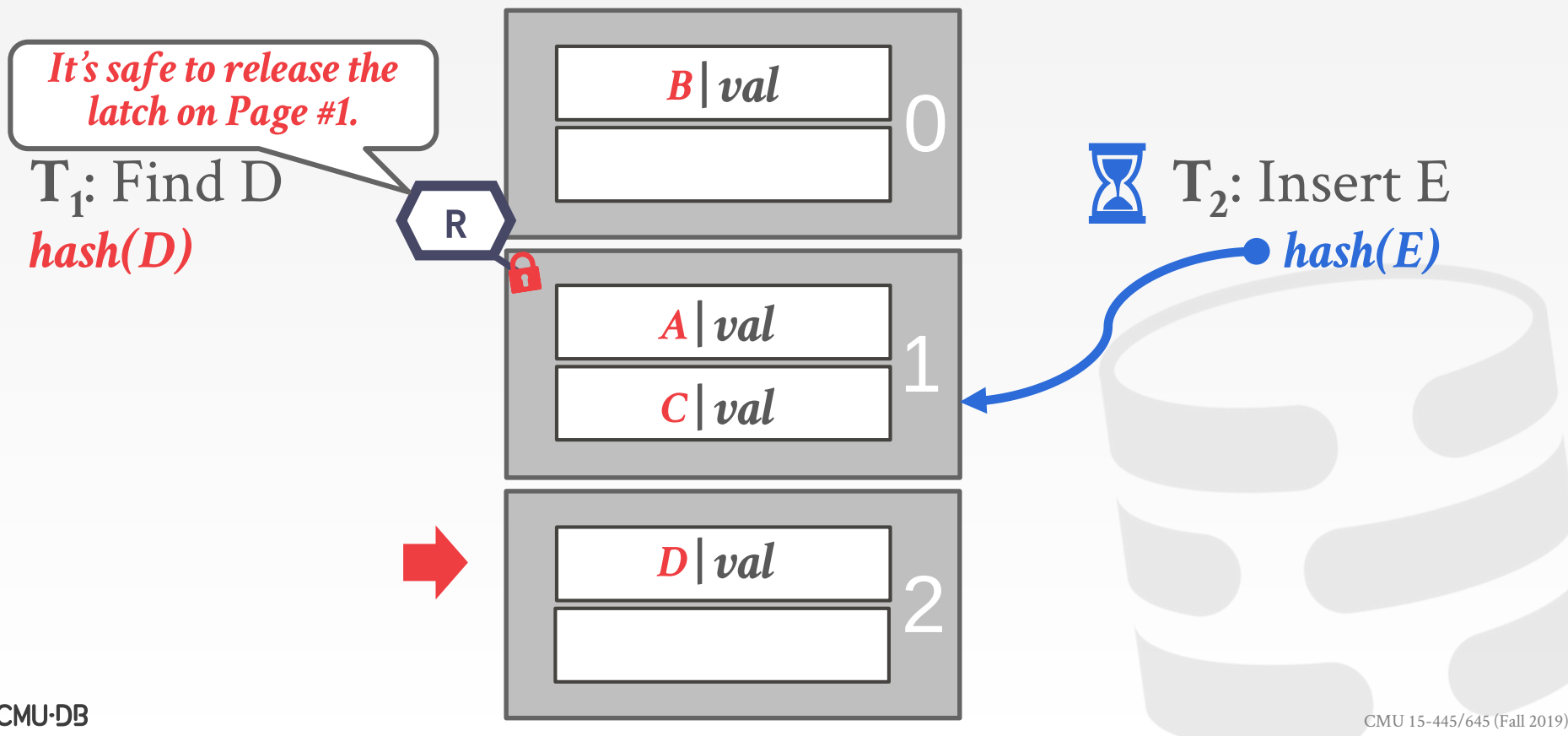
HASH TABLE – PAGE LATCHES

T_1 : Find D
hash(D)



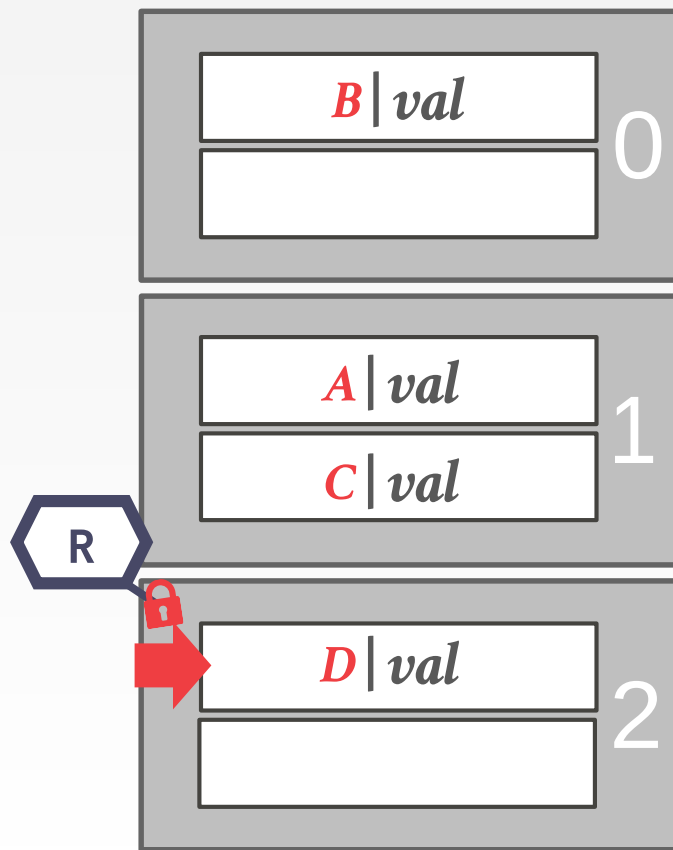
T_2 : Insert E
hash(E)

HASH TABLE – PAGE LATCHES



HASH TABLE – PAGE LATCHES

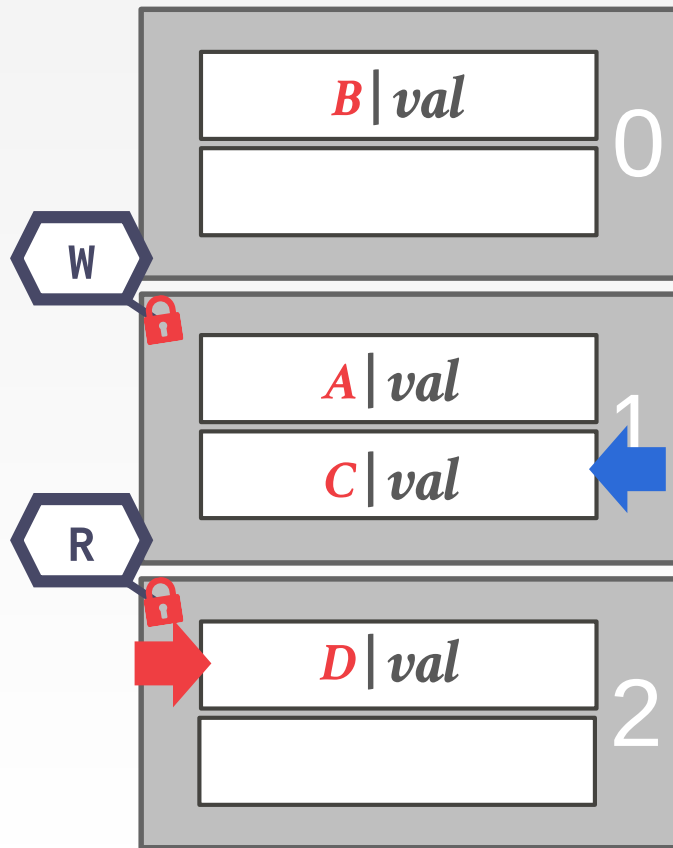
T_1 : Find D
hash(D)



T_2 : Insert E
hash(E)

HASH TABLE – PAGE LATCHES

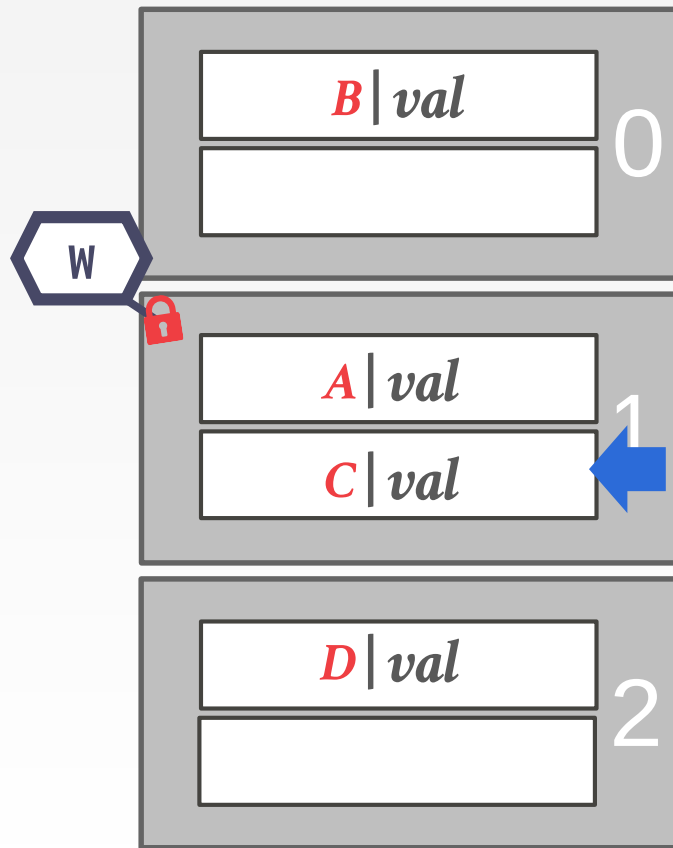
T_1 : Find D
hash(D)



T_2 : Insert E
hash(E)

HASH TABLE – PAGE LATCHES

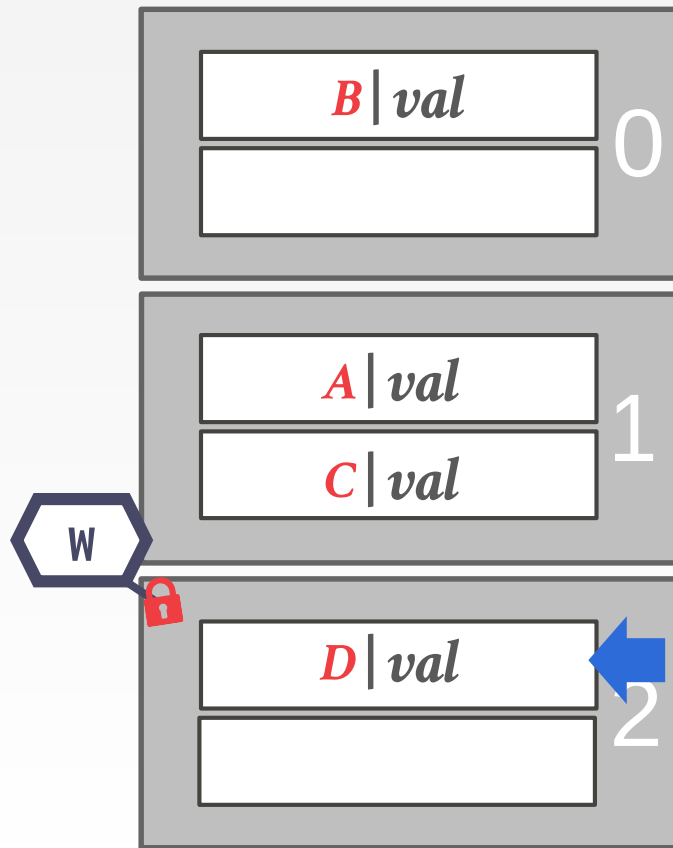
T_1 : Find D
hash(D)



T_2 : Insert E
hash(E)

HASH TABLE – PAGE LATCHES

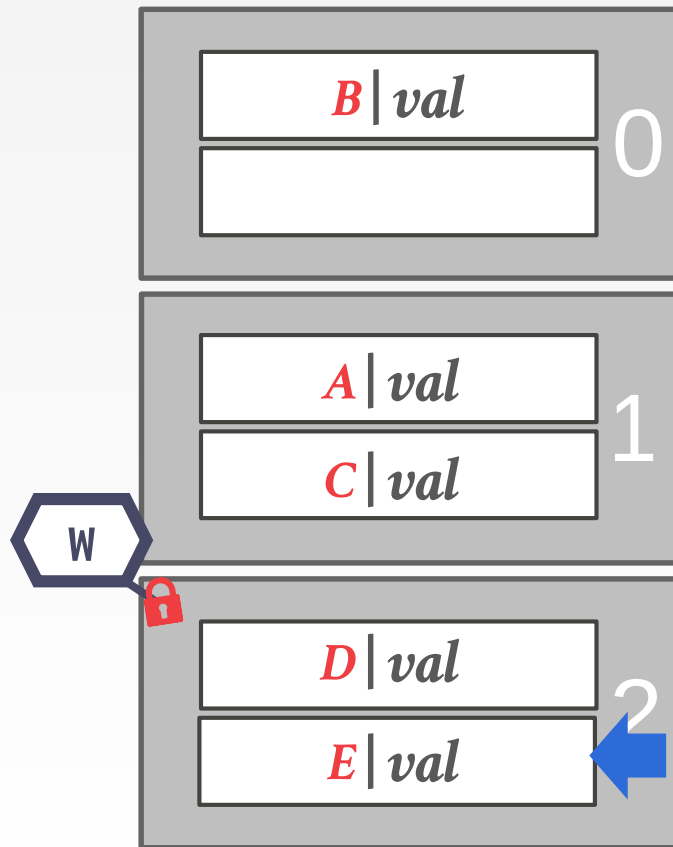
T_1 : Find D
hash(D)



T_2 : Insert E
hash(E)

HASH TABLE – PAGE LATCHES

T_1 : Find D
hash(D)

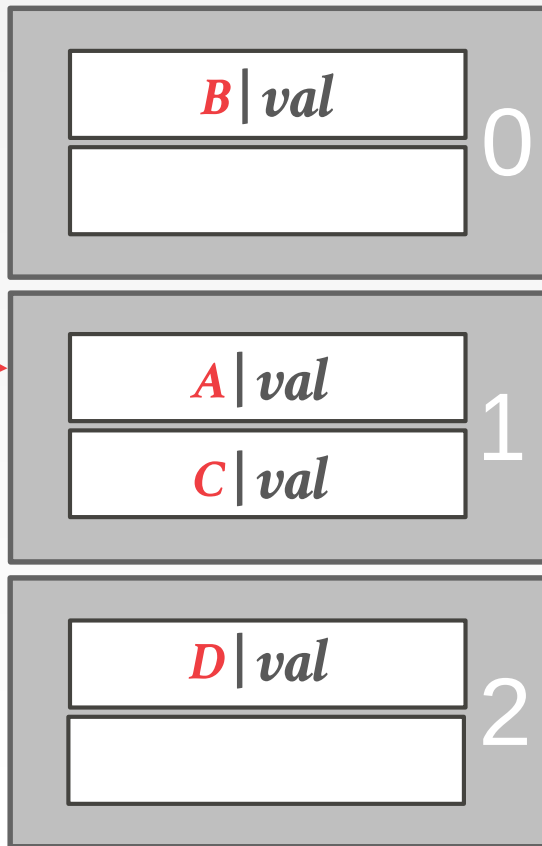


T_2 : Insert E
hash(E)

HASH TABLE – SLOT LATCHES

T_1 : Find D

$hash(D)$ ●

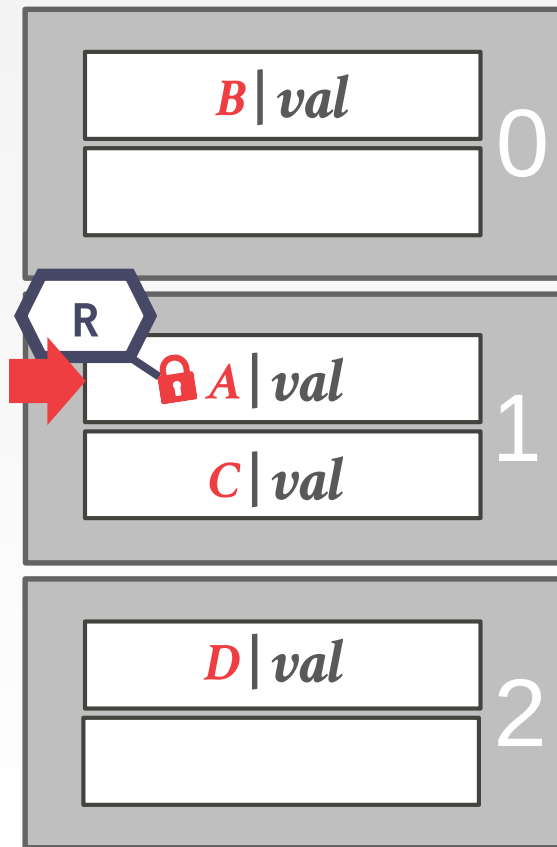


T_2 : Insert E

$hash(E)$

HASH TABLE – SLOT LATCHES

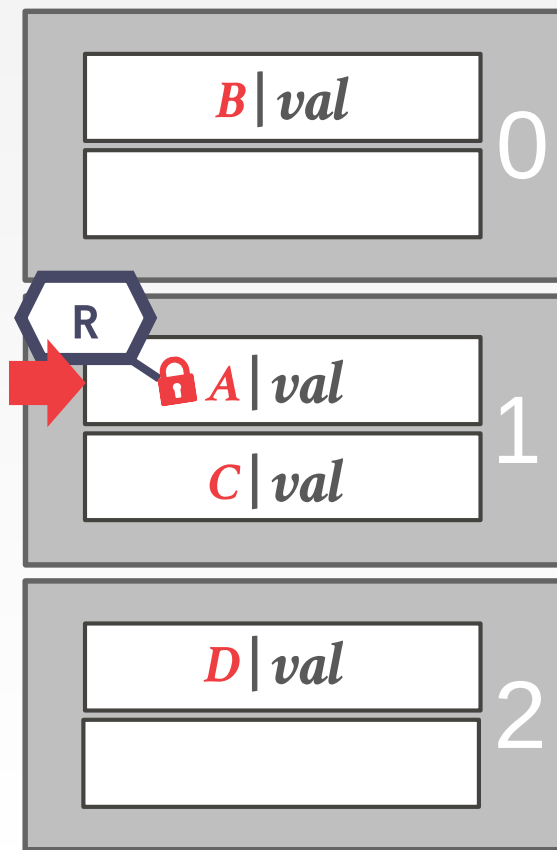
T_1 : Find D
hash(D)



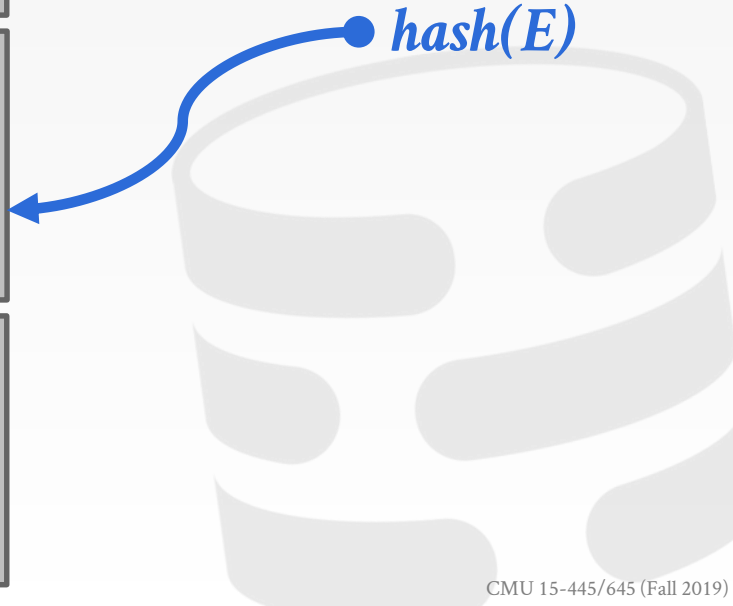
T_2 : Insert E
hash(E)

HASH TABLE – SLOT LATCHES

T_1 : Find D
hash(D)

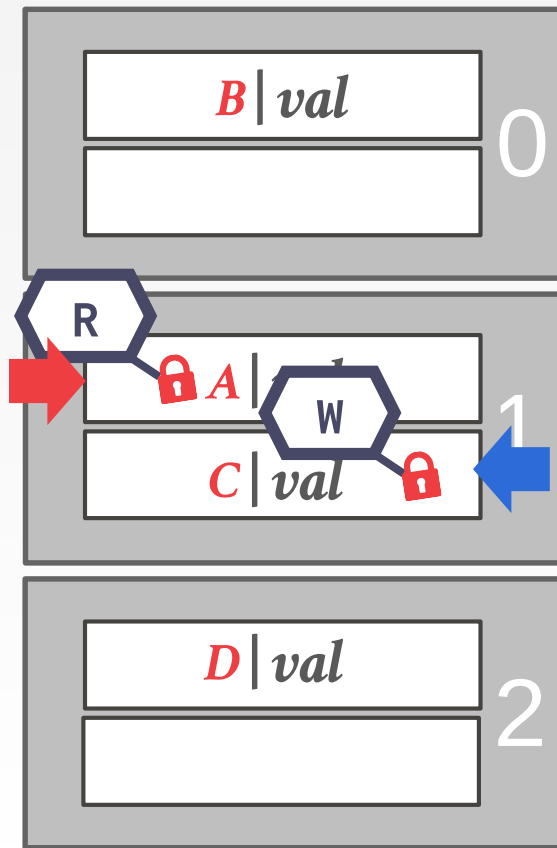


T_2 : Insert E
hash(E)



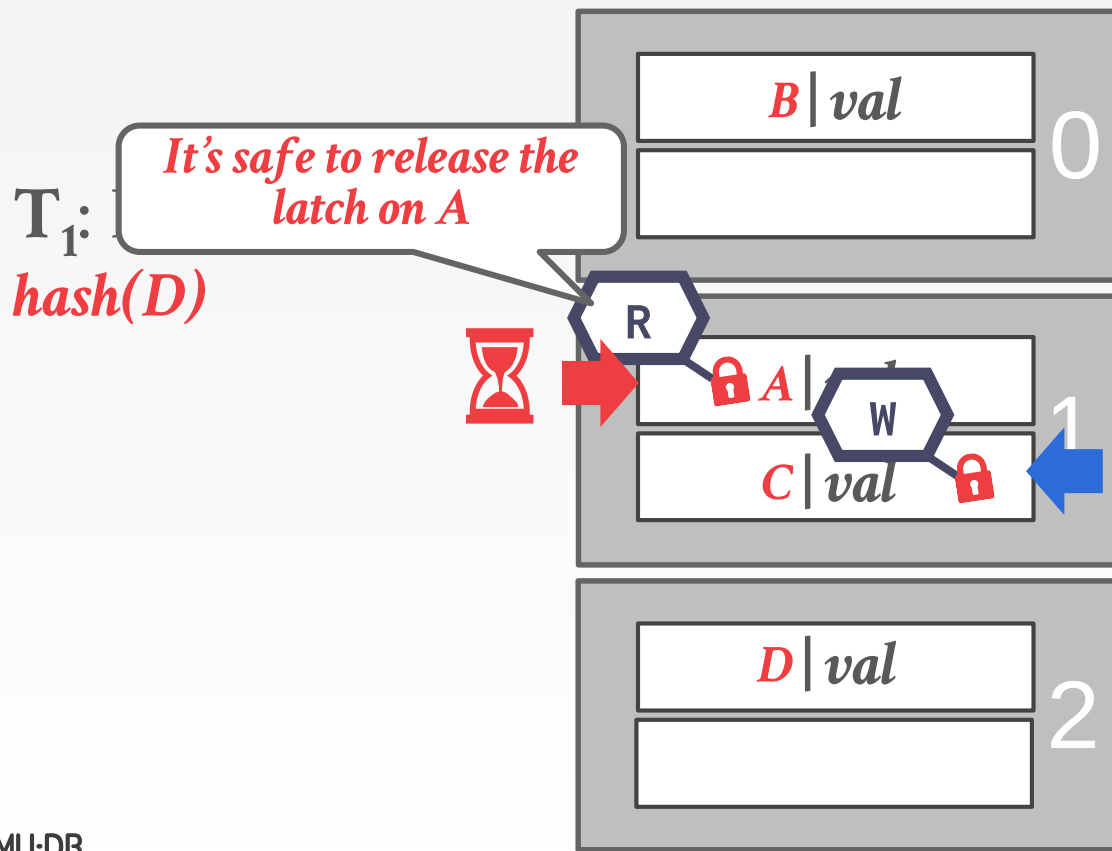
HASH TABLE – SLOT LATCHES

T_1 : Find D
hash(D)



T_2 : Insert E
hash(E)

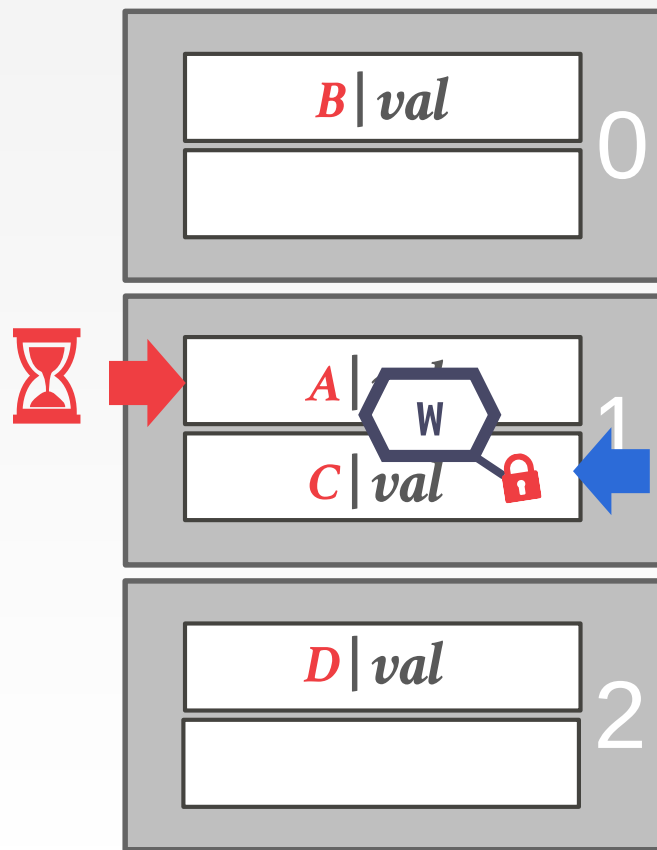
HASH TABLE – SLOT LATCHES



T_2 : Insert E
 $hash(E)$

HASH TABLE – SLOT LATCHES

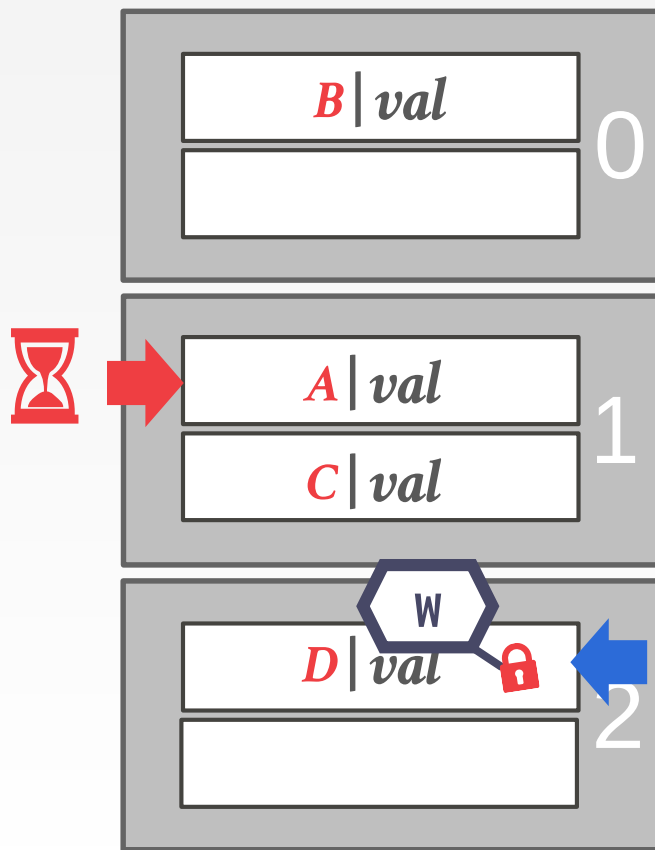
T_1 : Find D
hash(D)



T_2 : Insert E
hash(E)

HASH TABLE – SLOT LATCHES

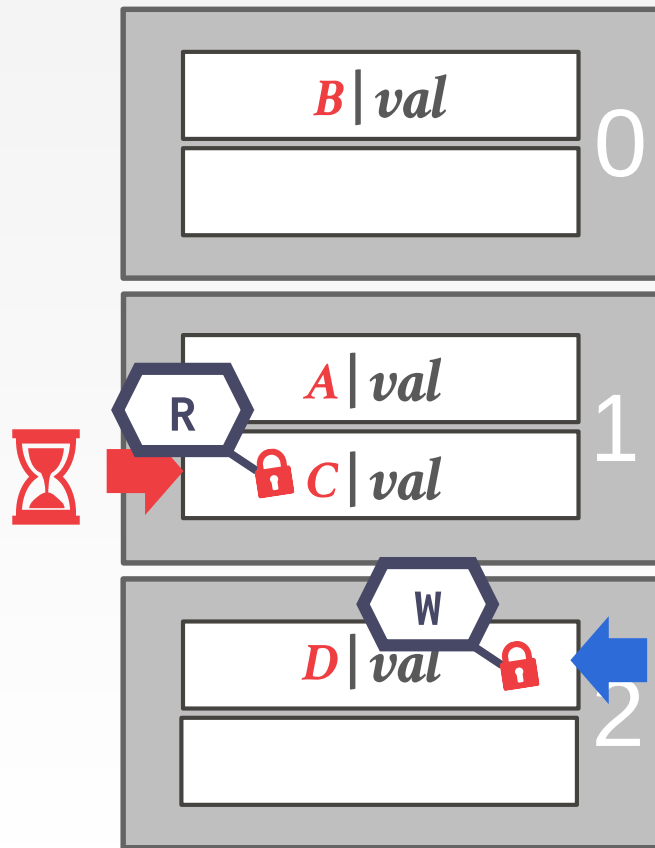
T_1 : Find D
hash(D)



T_2 : Insert E
hash(E)

HASH TABLE – SLOT LATCHES

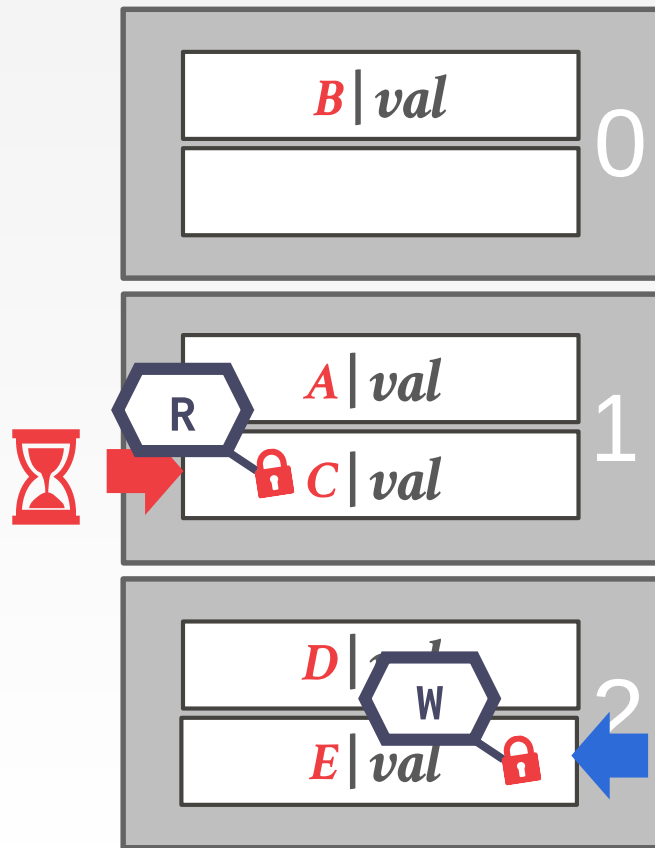
T_1 : Find D
hash(D)



T_2 : Insert E
hash(E)

HASH TABLE – SLOT LATCHES

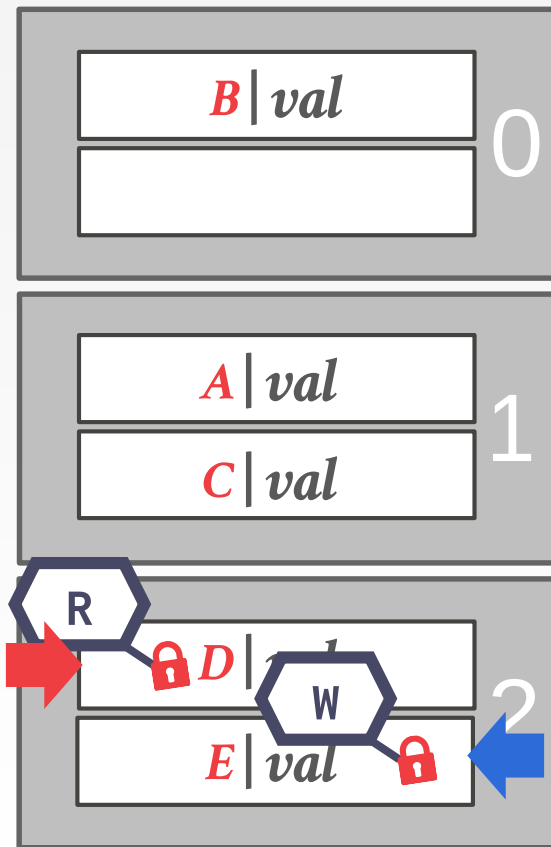
T_1 : Find D
hash(D)



T_2 : Insert E
hash(E)

HASH TABLE – SLOT LATCHES

T_1 : Find D
hash(D)



T_2 : Insert E
hash(E)

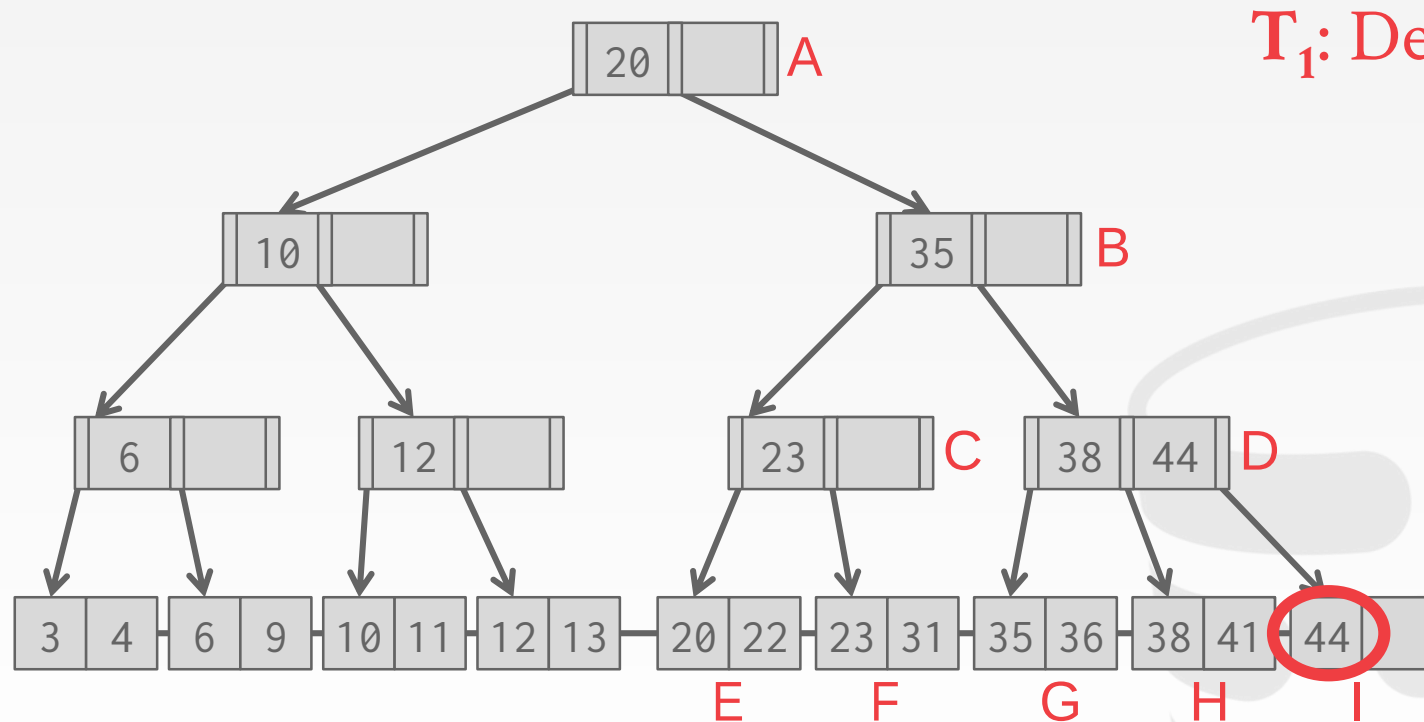
B+TREE CONCURRENCY CONTROL

We want to allow multiple threads to read and update a B+Tree at the same time.

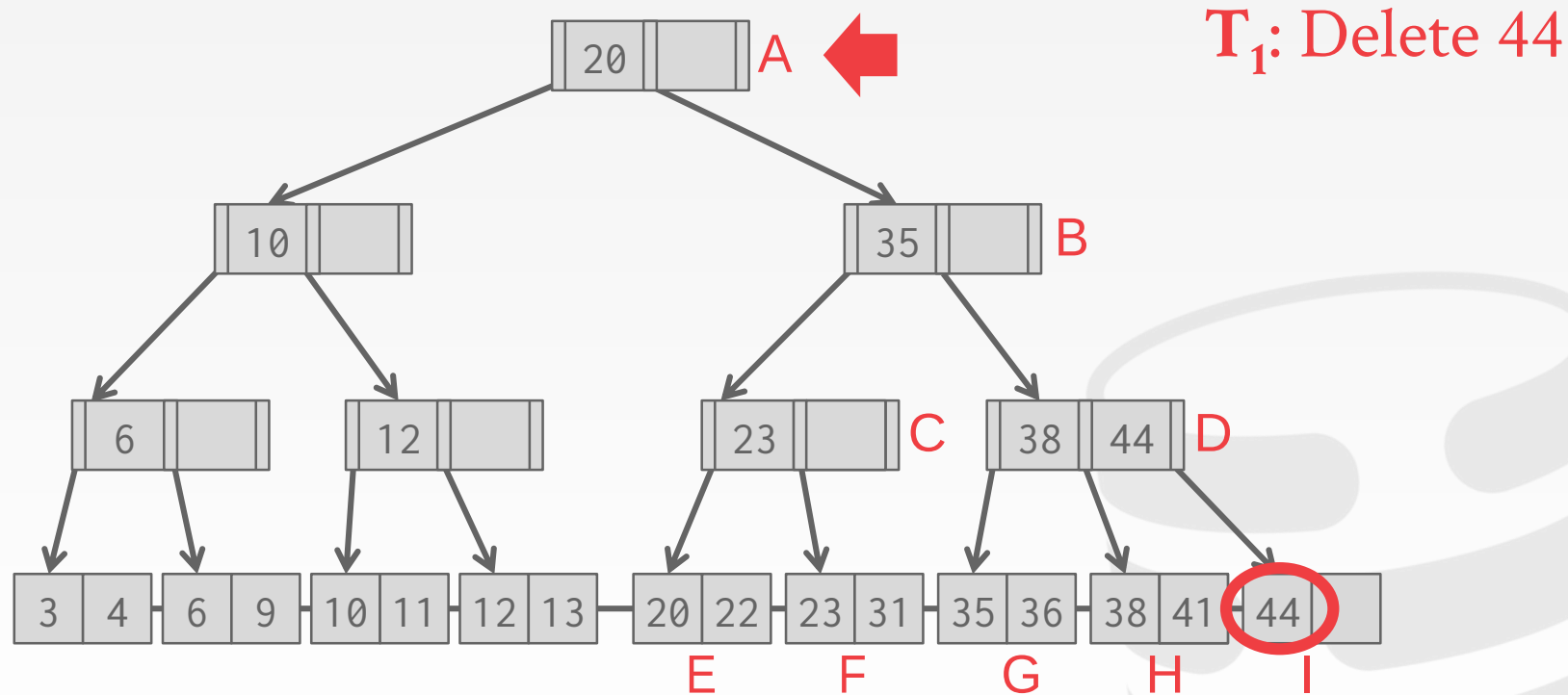
We need to protect from two types of problems:

- Threads trying to modify the contents of a node at the same time.
- One thread traversing the tree while another thread splits/merges nodes.

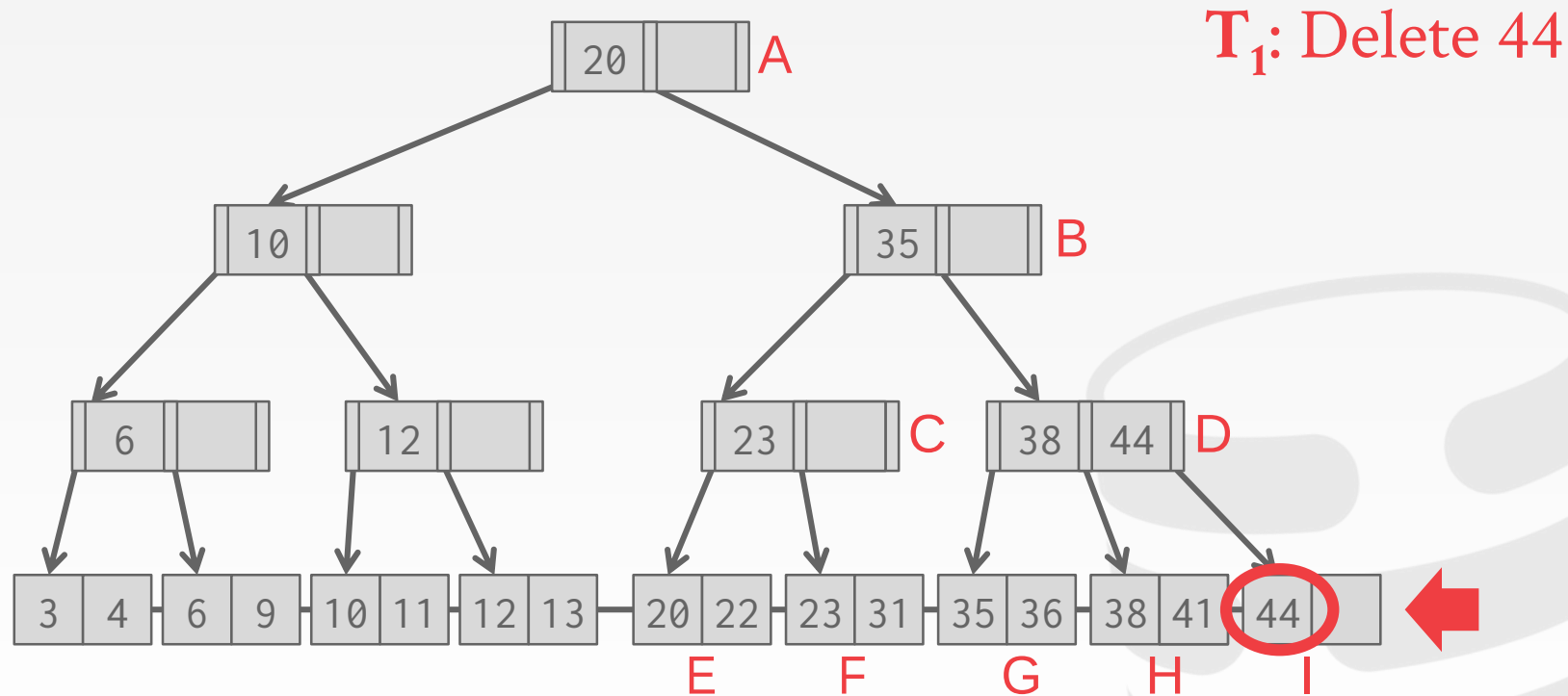
B+TREE MULTI-THREADED EXAMPLE



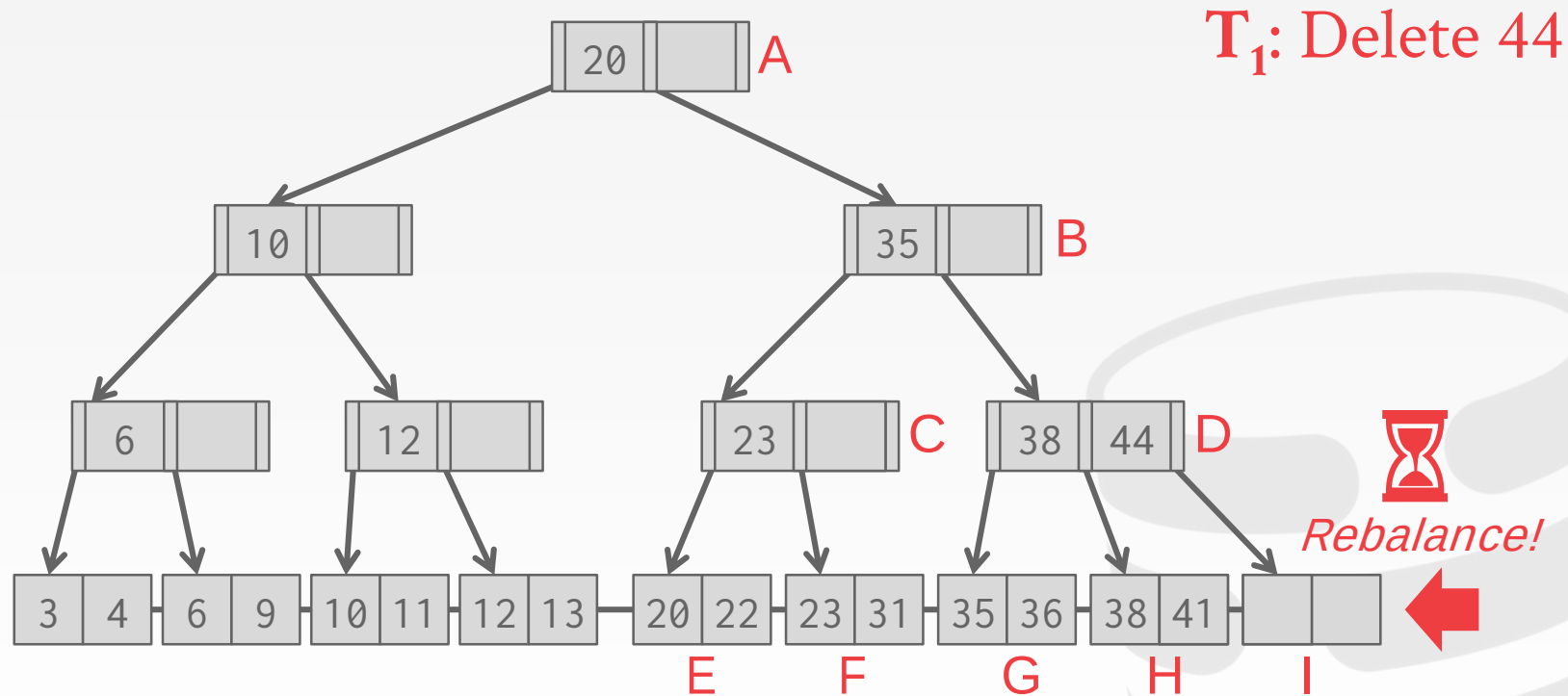
B+TREE MULTI-THREADED EXAMPLE



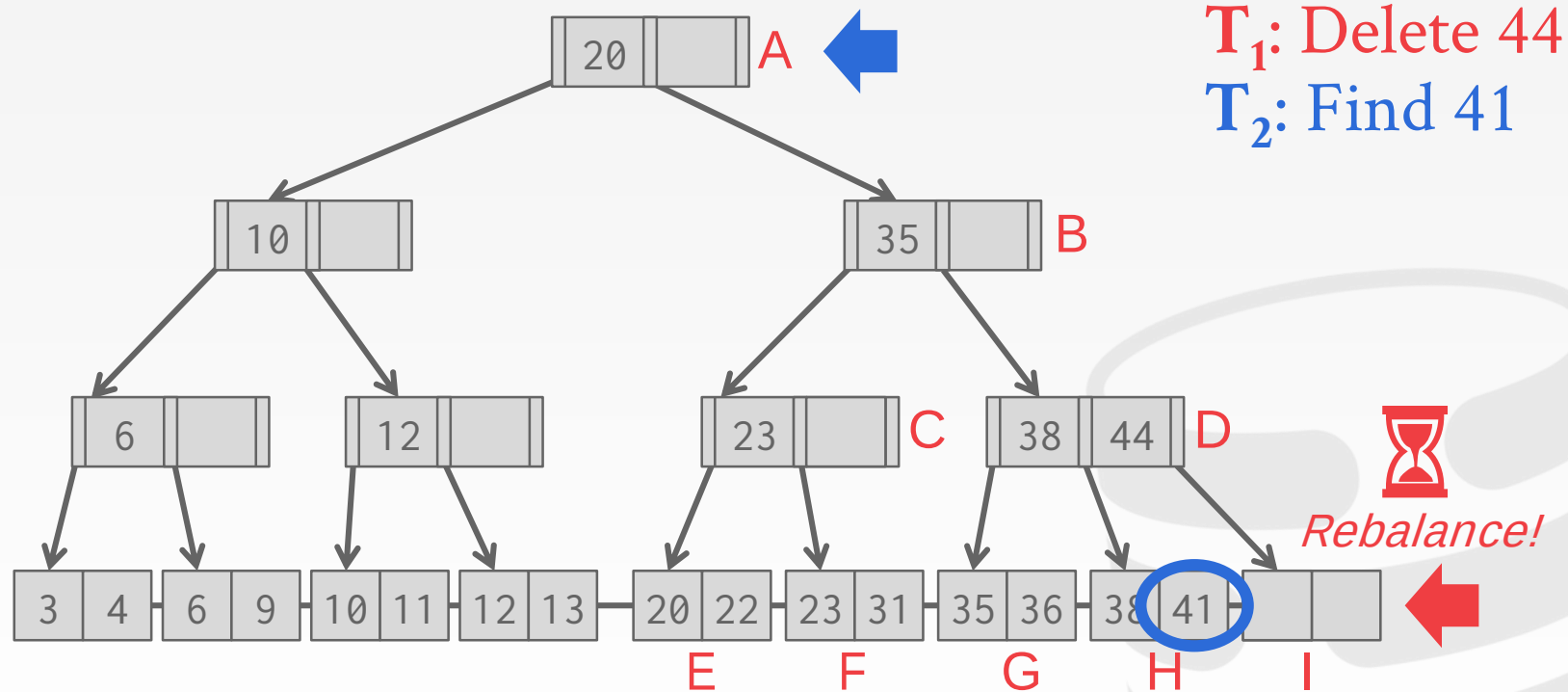
B+TREE MULTI-THREADED EXAMPLE



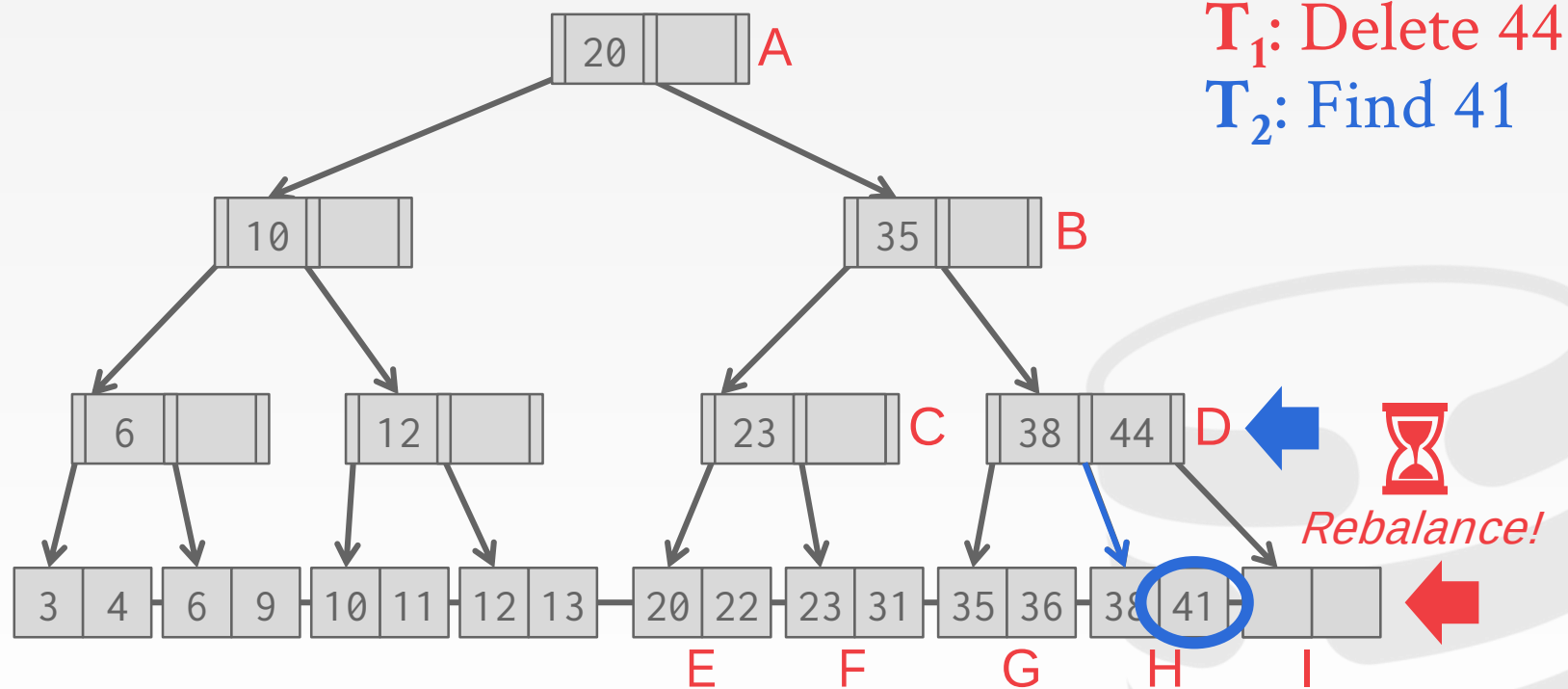
B+TREE MULTI-THREADED EXAMPLE



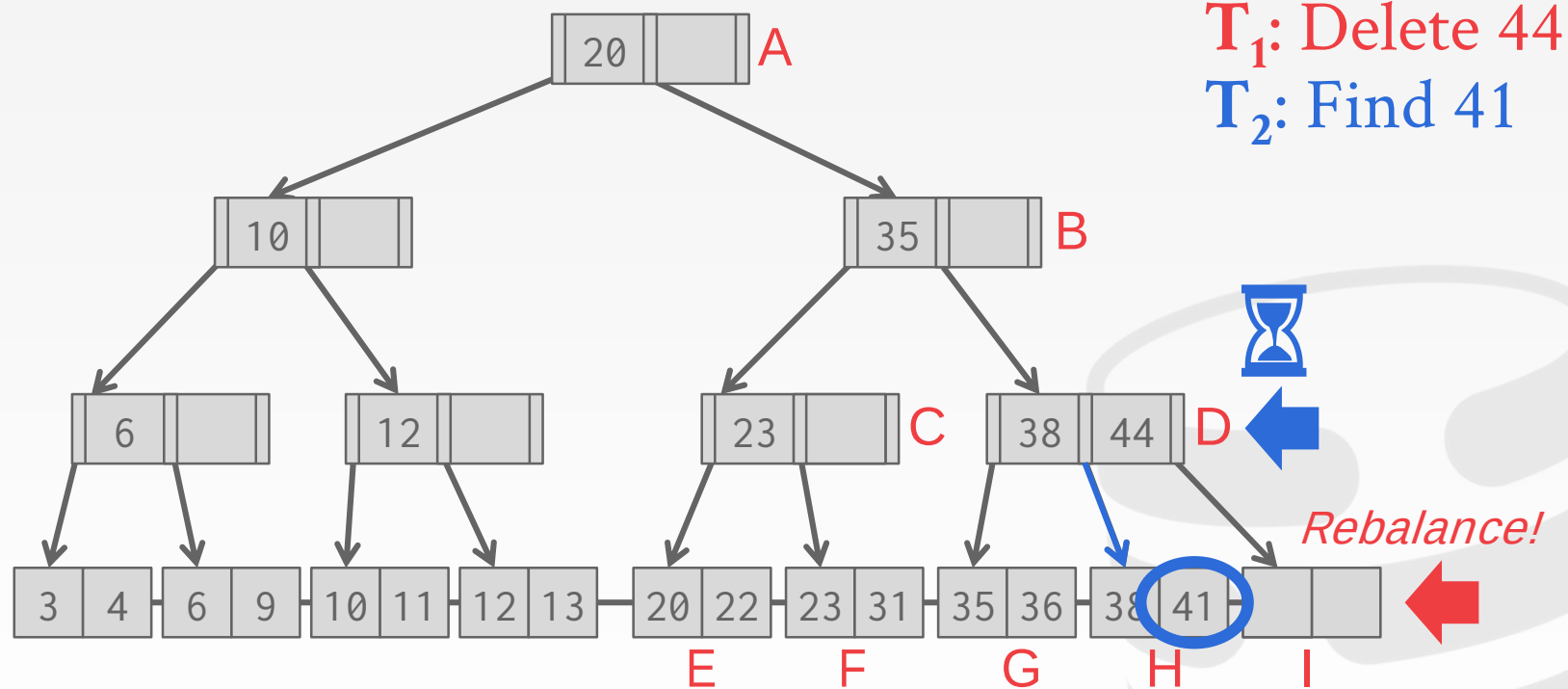
B+TREE MULTI-THREADED EXAMPLE



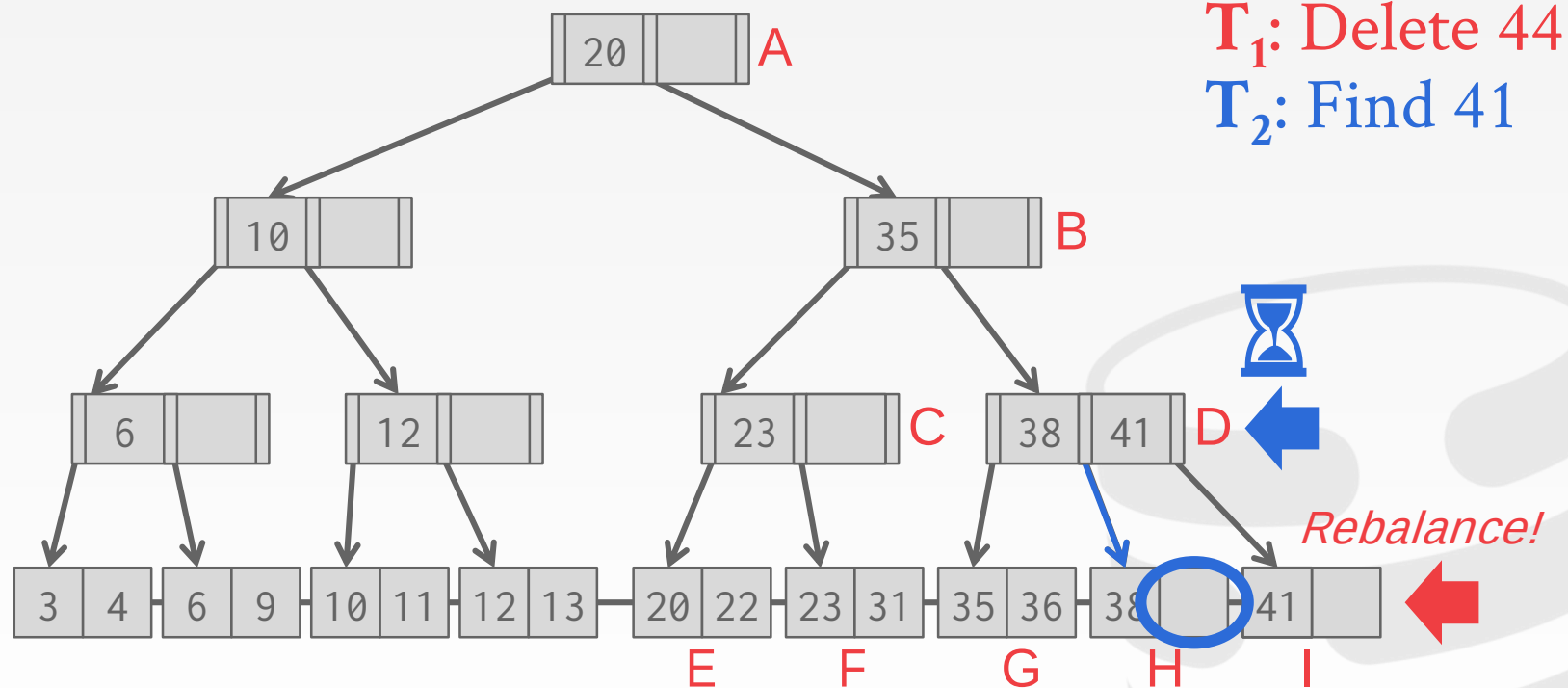
B+TREE MULTI-THREADED EXAMPLE



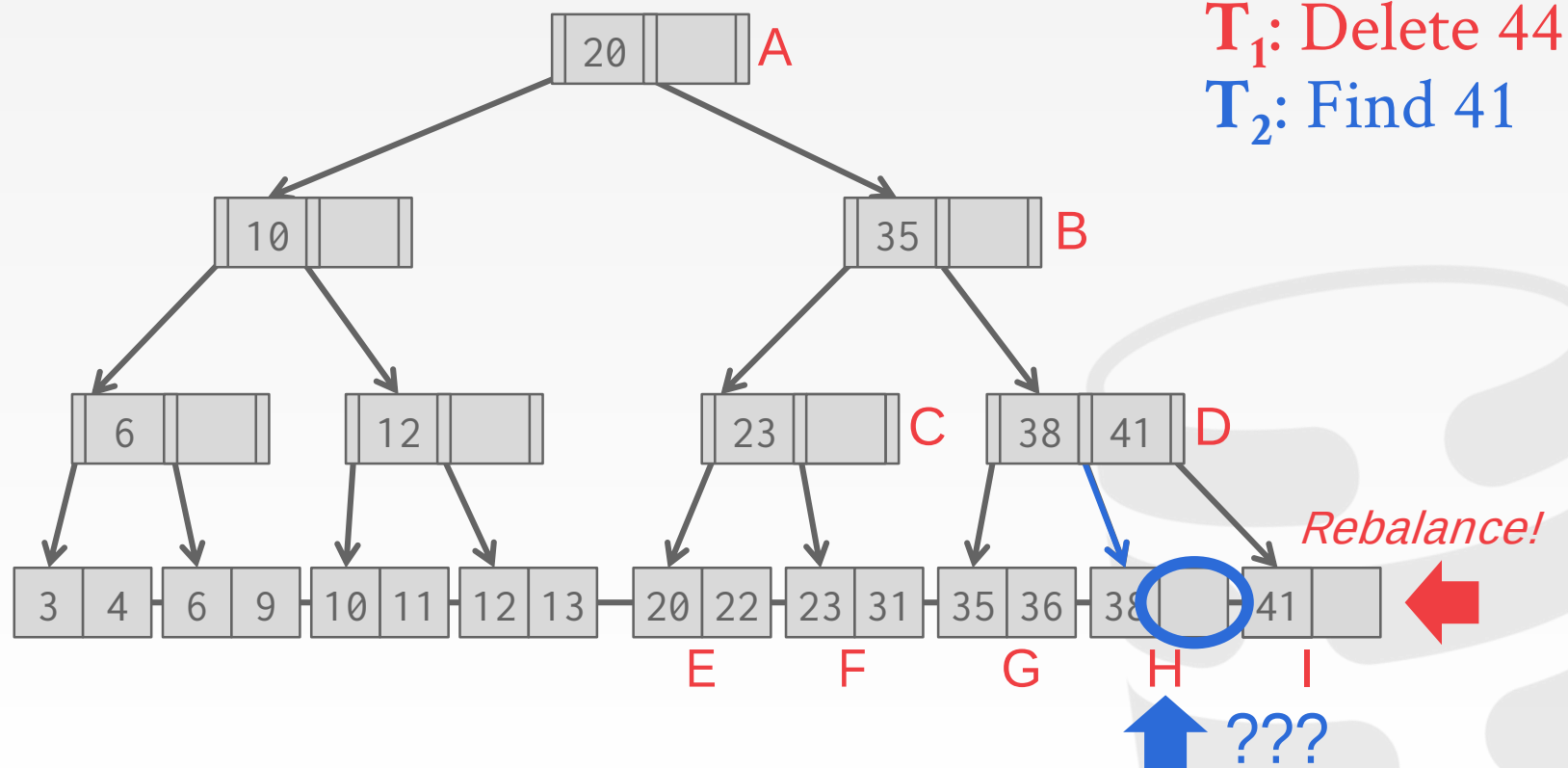
B+TREE MULTI-THREADED EXAMPLE



B+TREE MULTI-THREADED EXAMPLE



B+TREE MULTI-THREADED EXAMPLE



LATCH CRABBING/COUPLING

Protocol to allow multiple threads to access/modify B+Tree at the same time.

Basic Idea:

- Get latch for parent.
- Get latch for child
- Release latch for parent if “safe”.

A **safe node** is one that will not split or merge when updated.

- Not full (on insertion)
- More than half-full (on deletion)

LATCH CRABBING/COUPLING

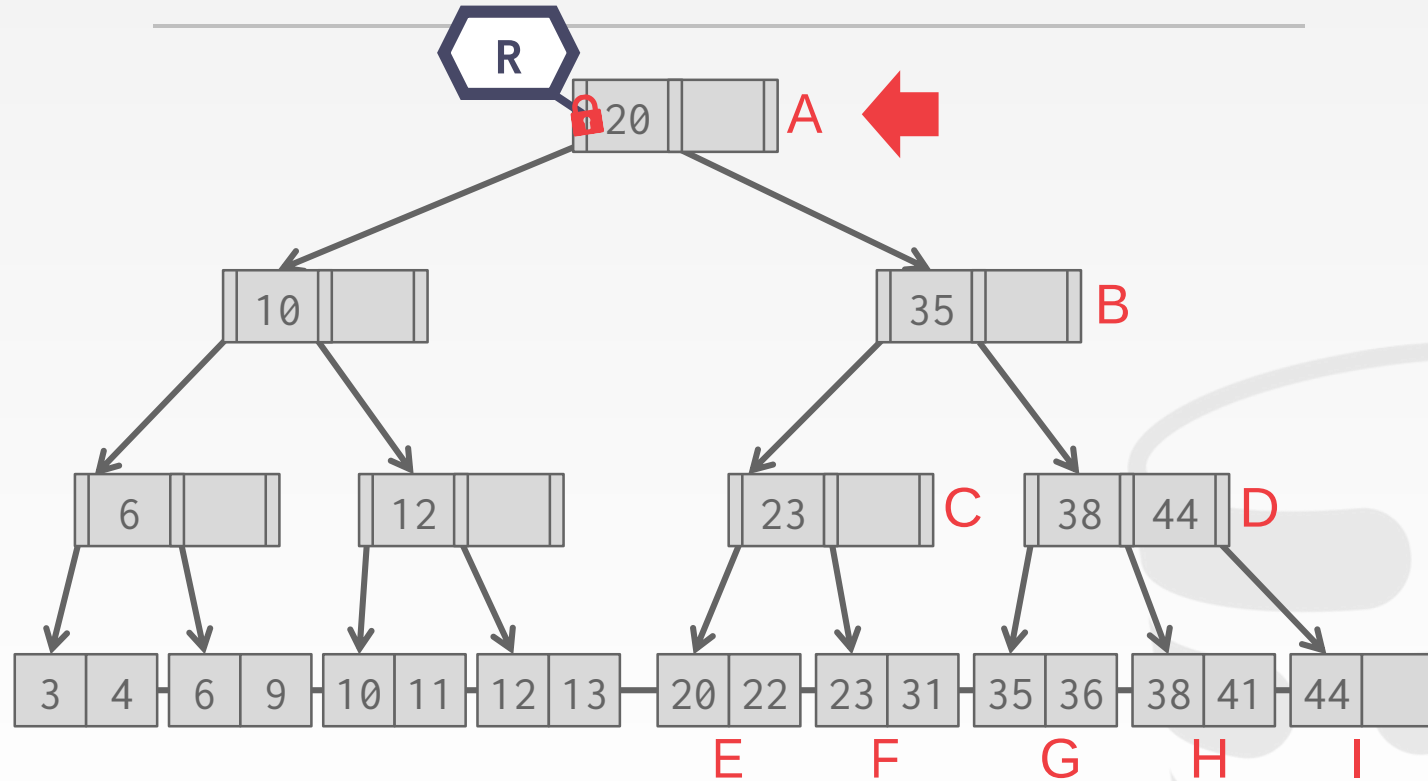
Find: Start at root and go down; repeatedly,

- Acquire **R** latch on child
- Then unlatch parent

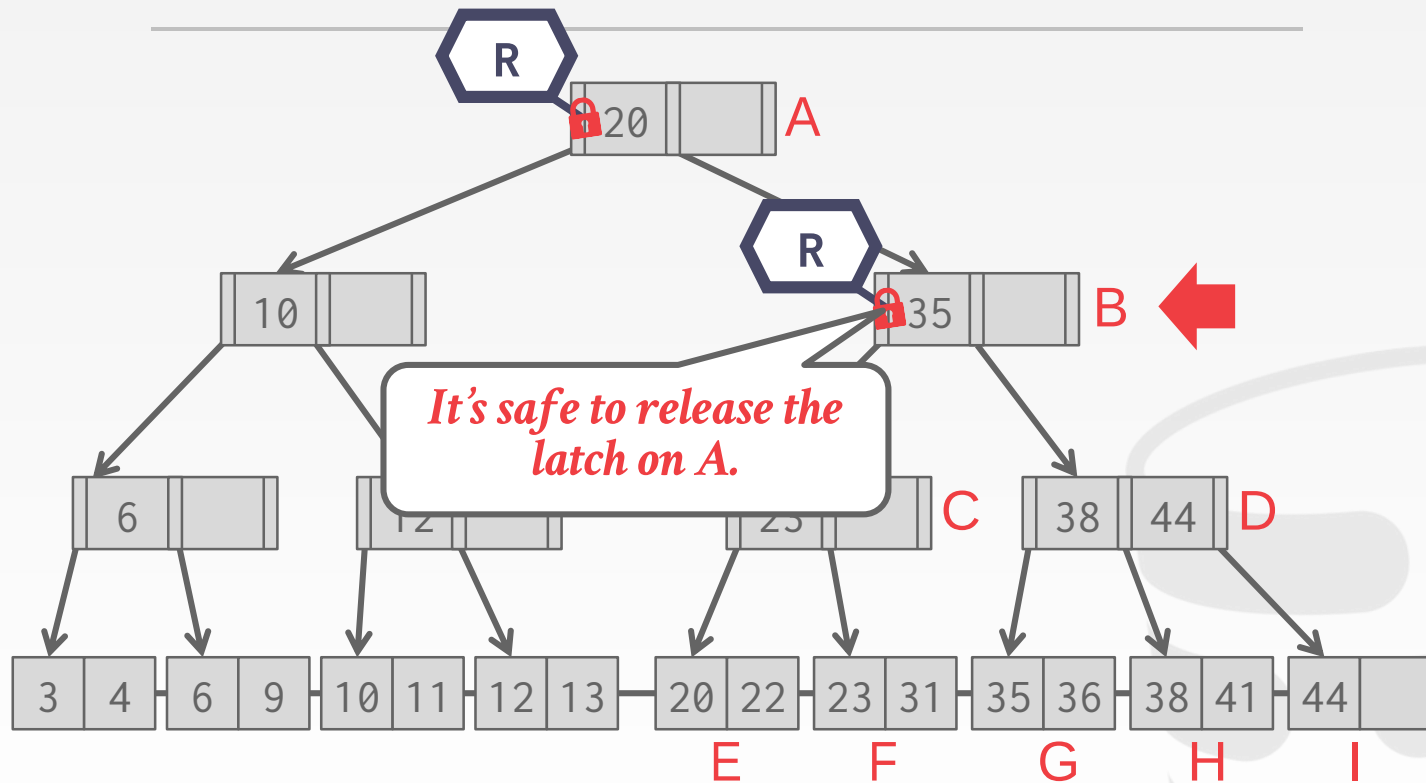
Insert/Delete: Start at root and go down, obtaining **W** latches as needed. Once child is latched, check if it is safe:

- If child is safe, release all latches on ancestors.

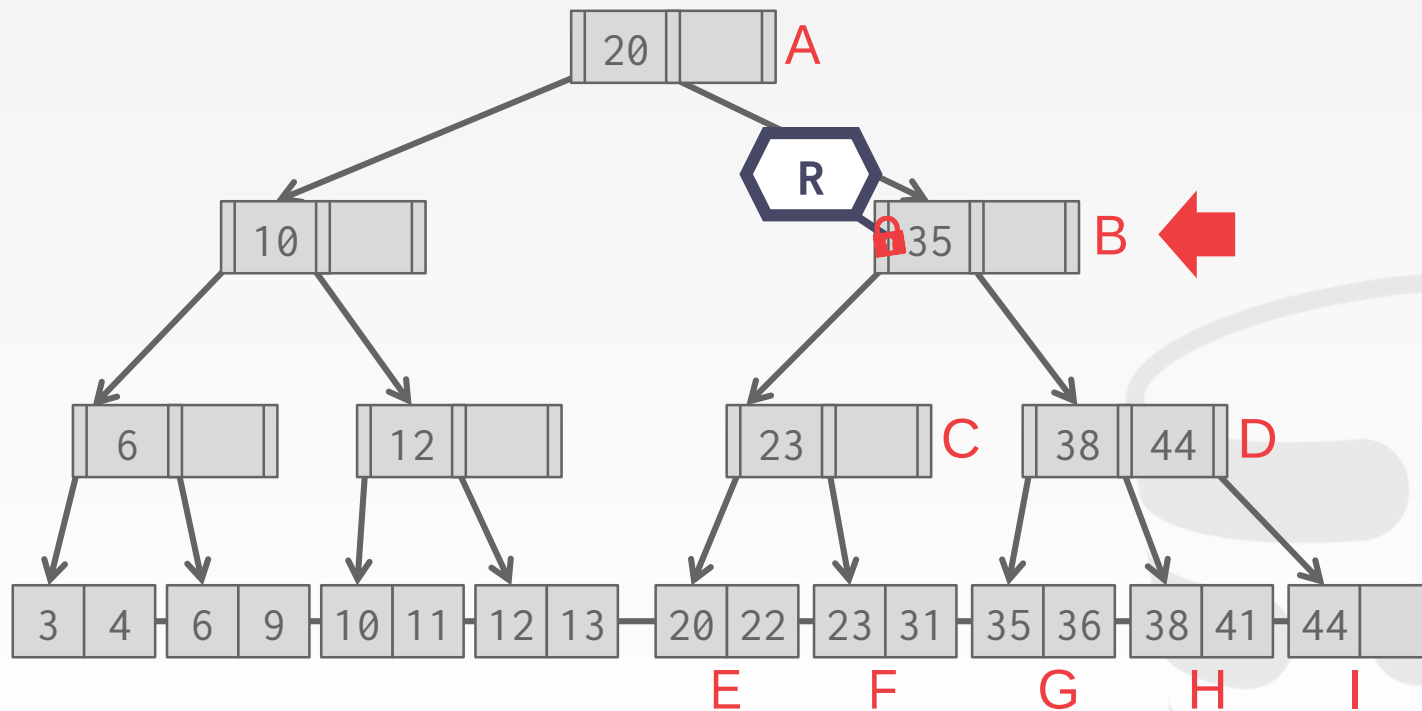
EXAMPLE #1 – FIND 38



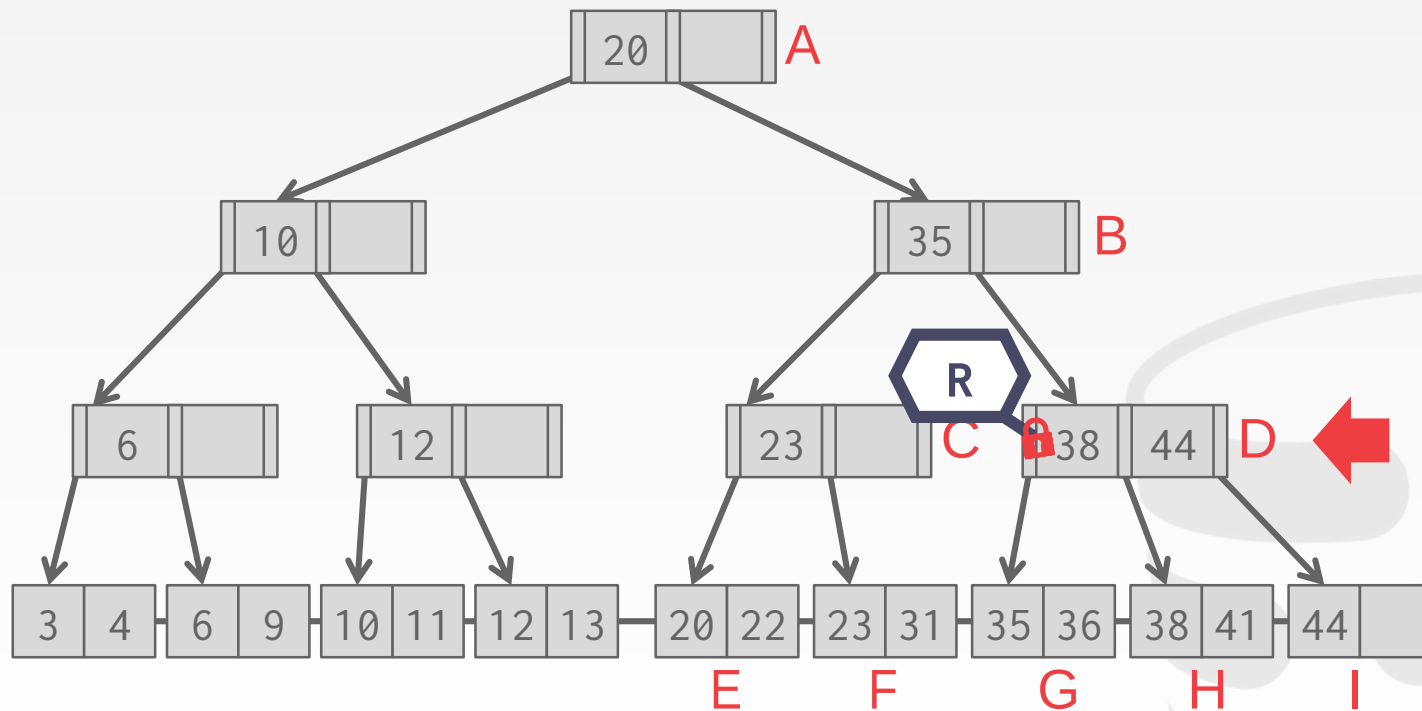
EXAMPLE #1 – FIND 38



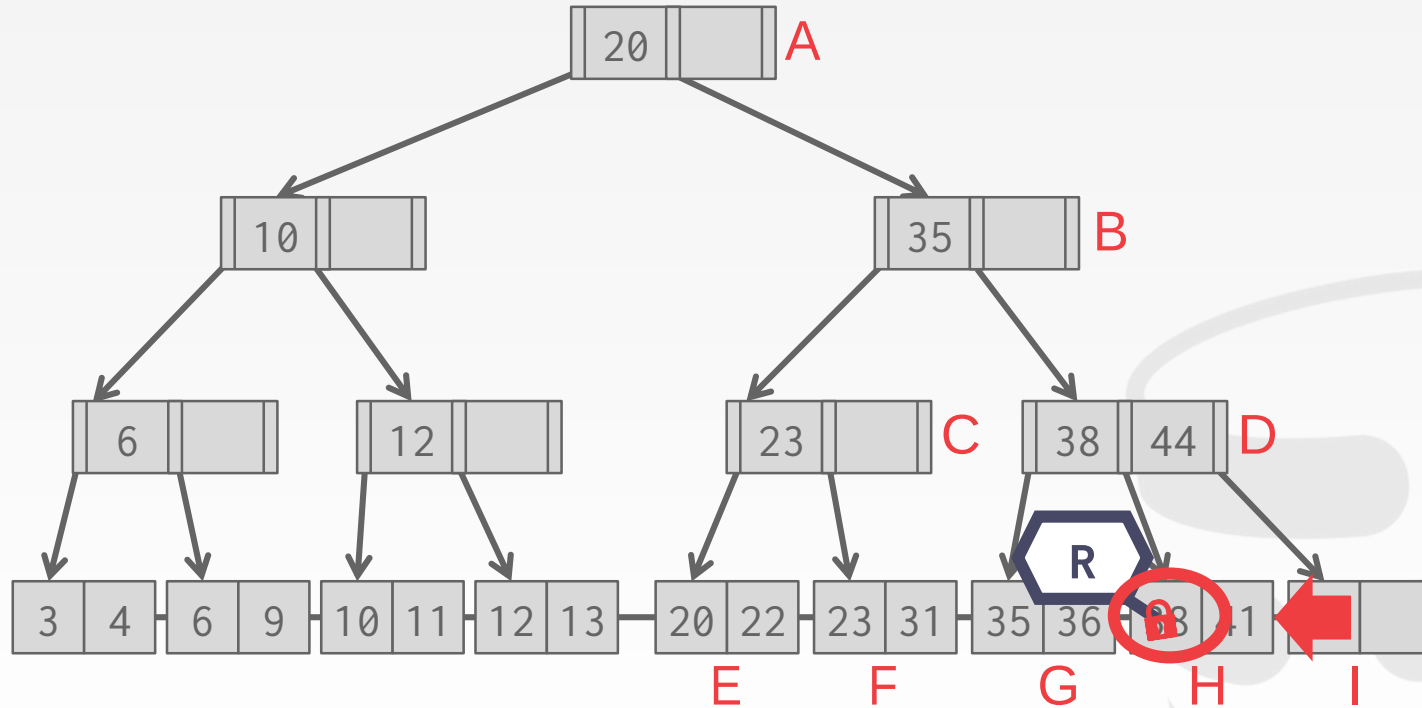
EXAMPLE #1 – FIND 38



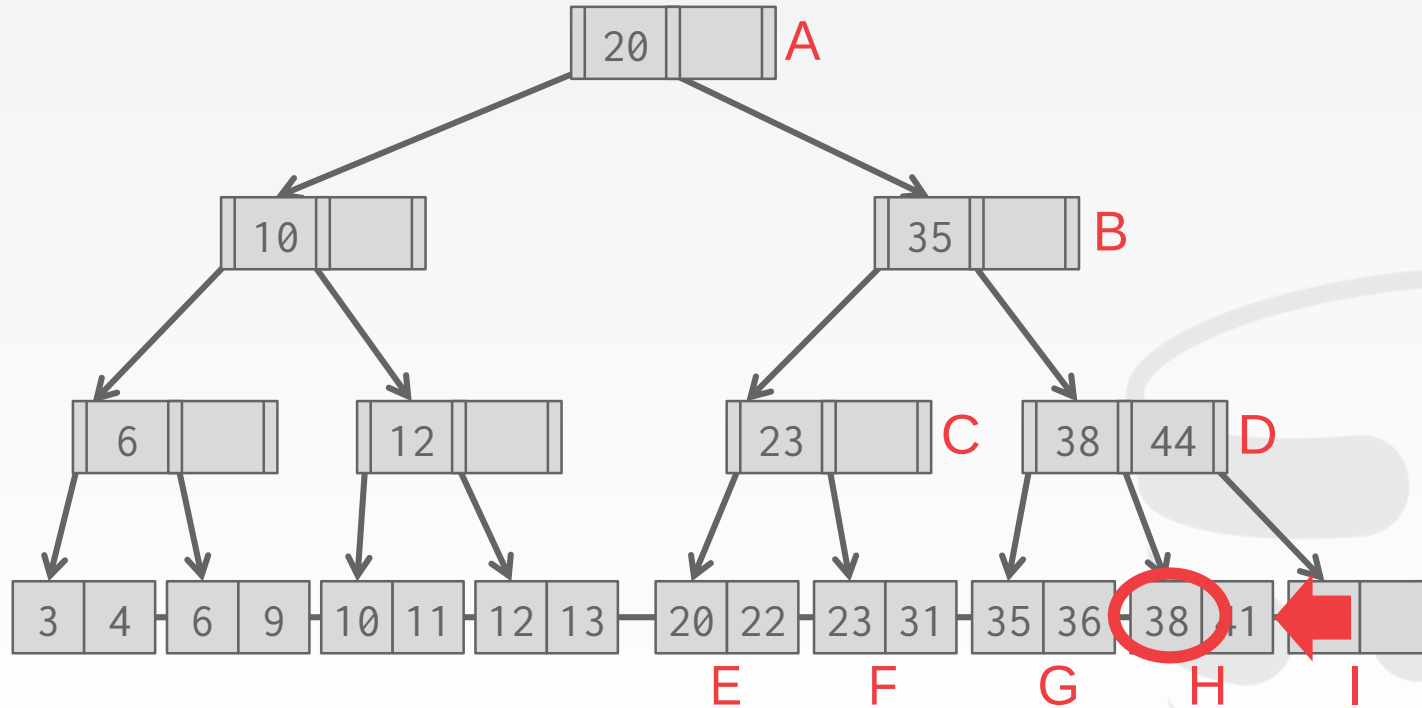
EXAMPLE #1 – FIND 38



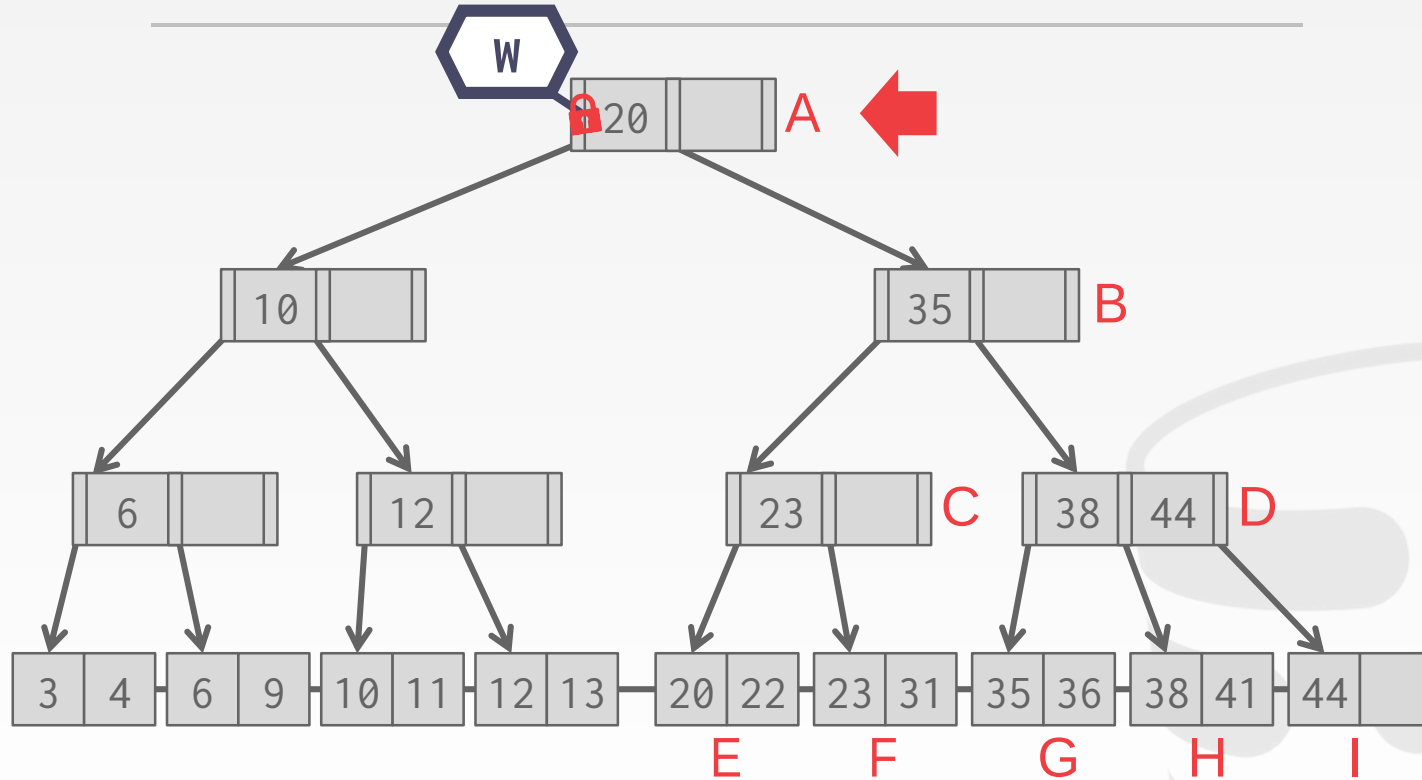
EXAMPLE #1 – FIND 38



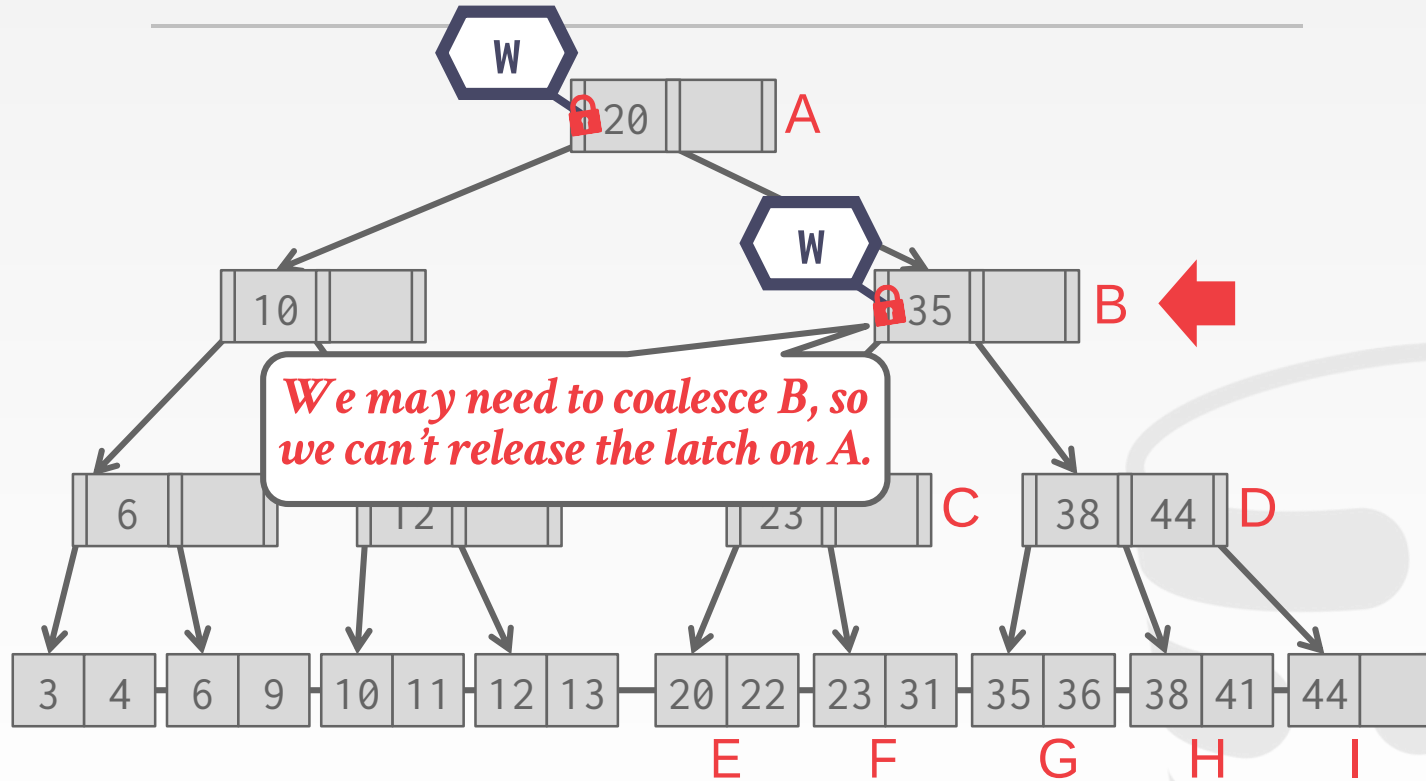
EXAMPLE #1 – FIND 38



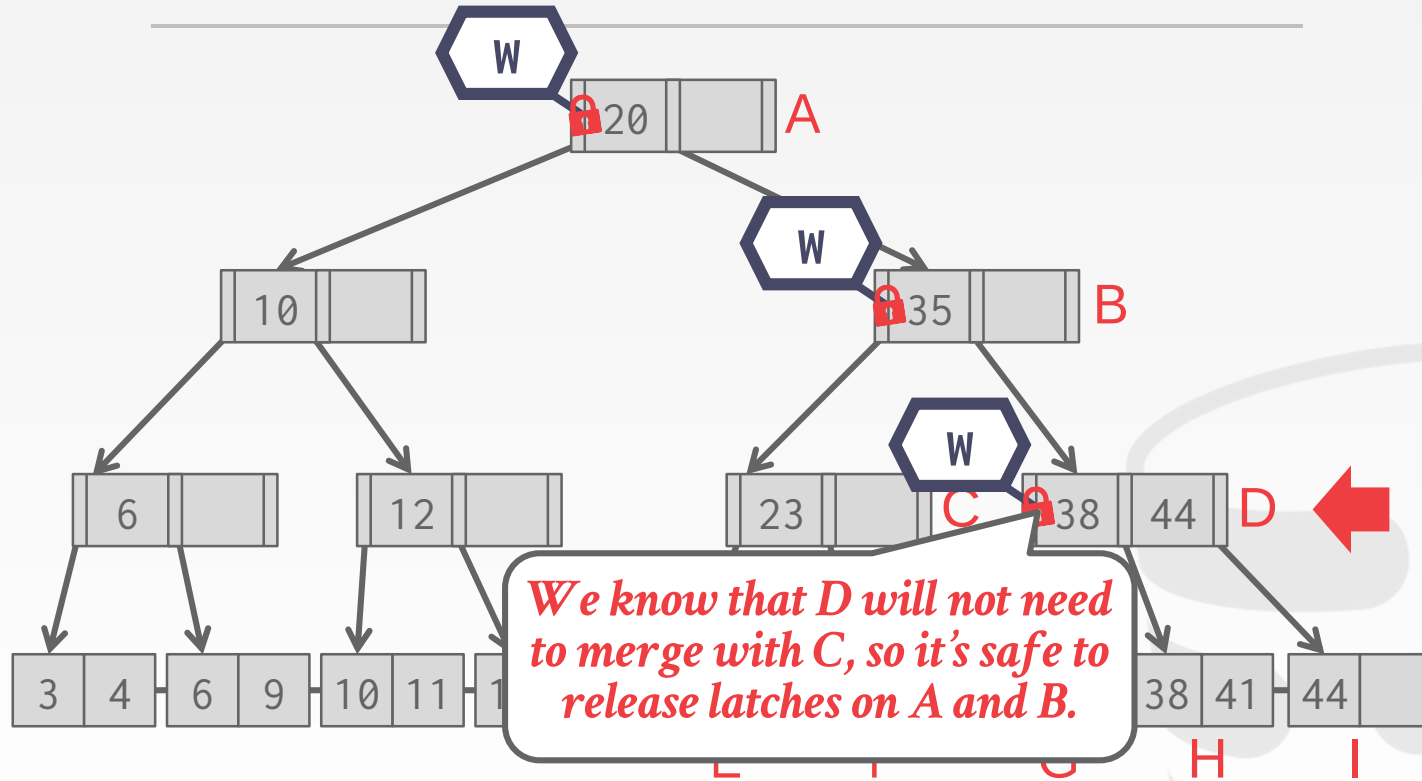
EXAMPLE #2 – DELETE 38



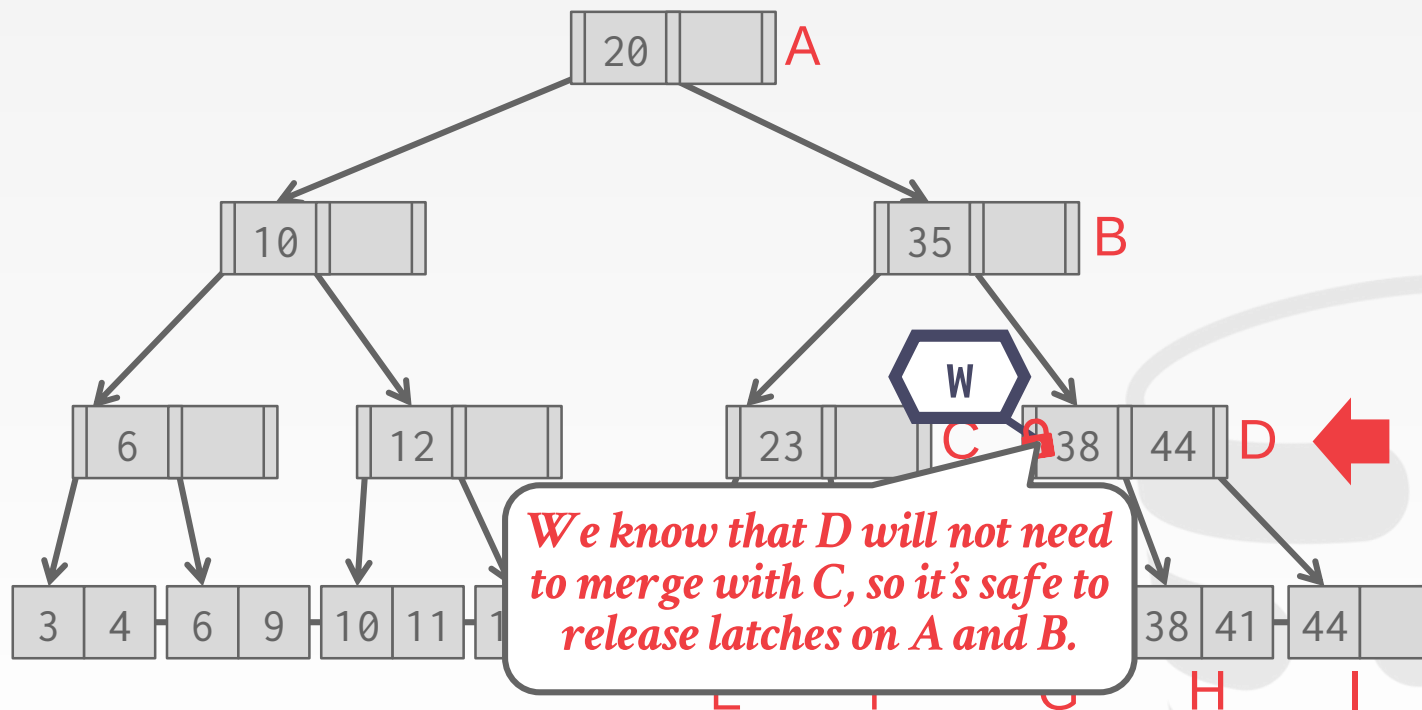
EXAMPLE #2 – DELETE 38



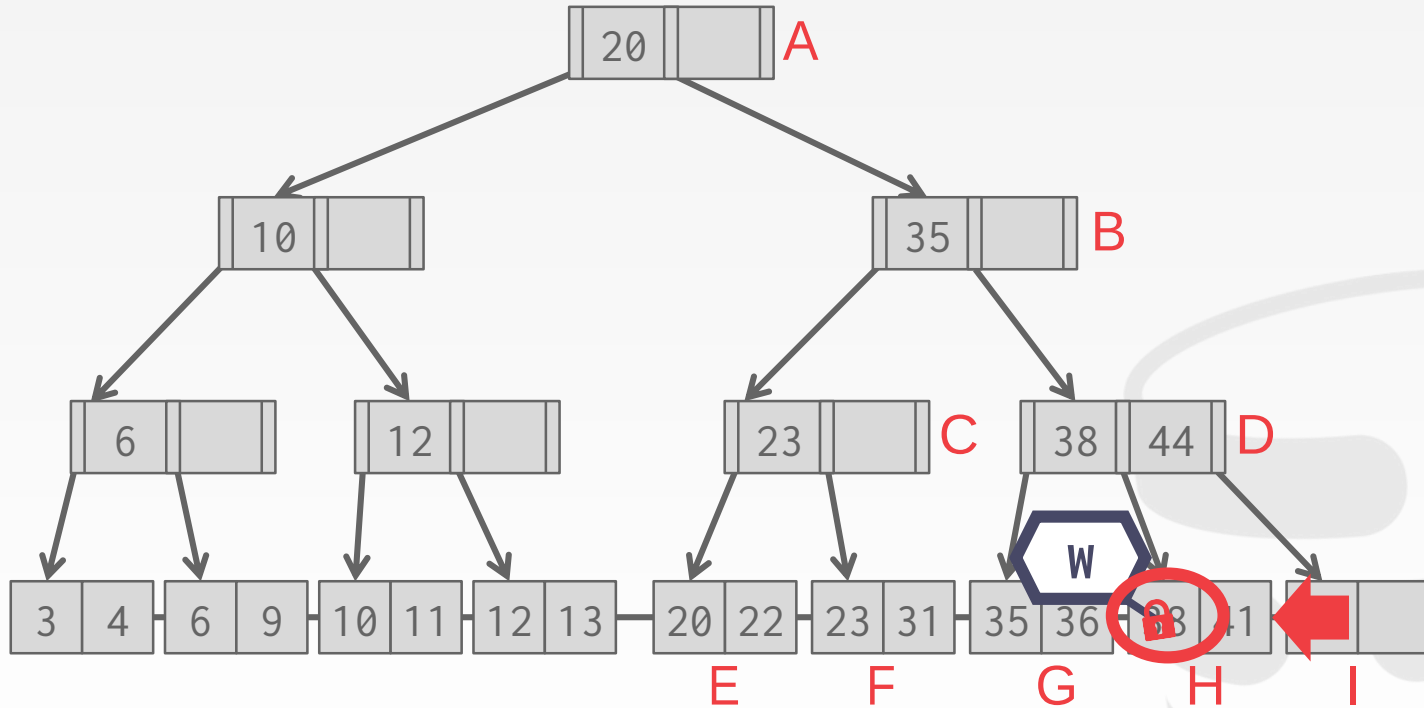
EXAMPLE #2 – DELETE 38



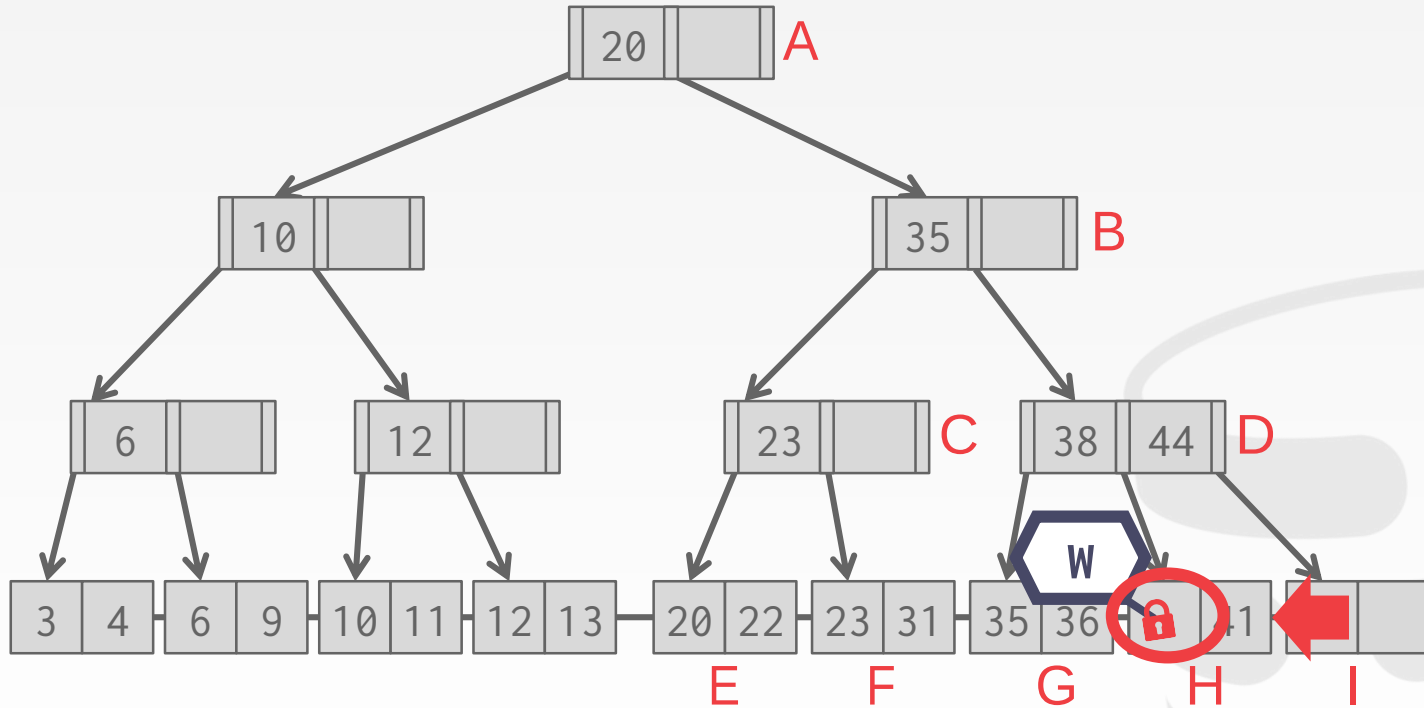
EXAMPLE #2 – DELETE 38



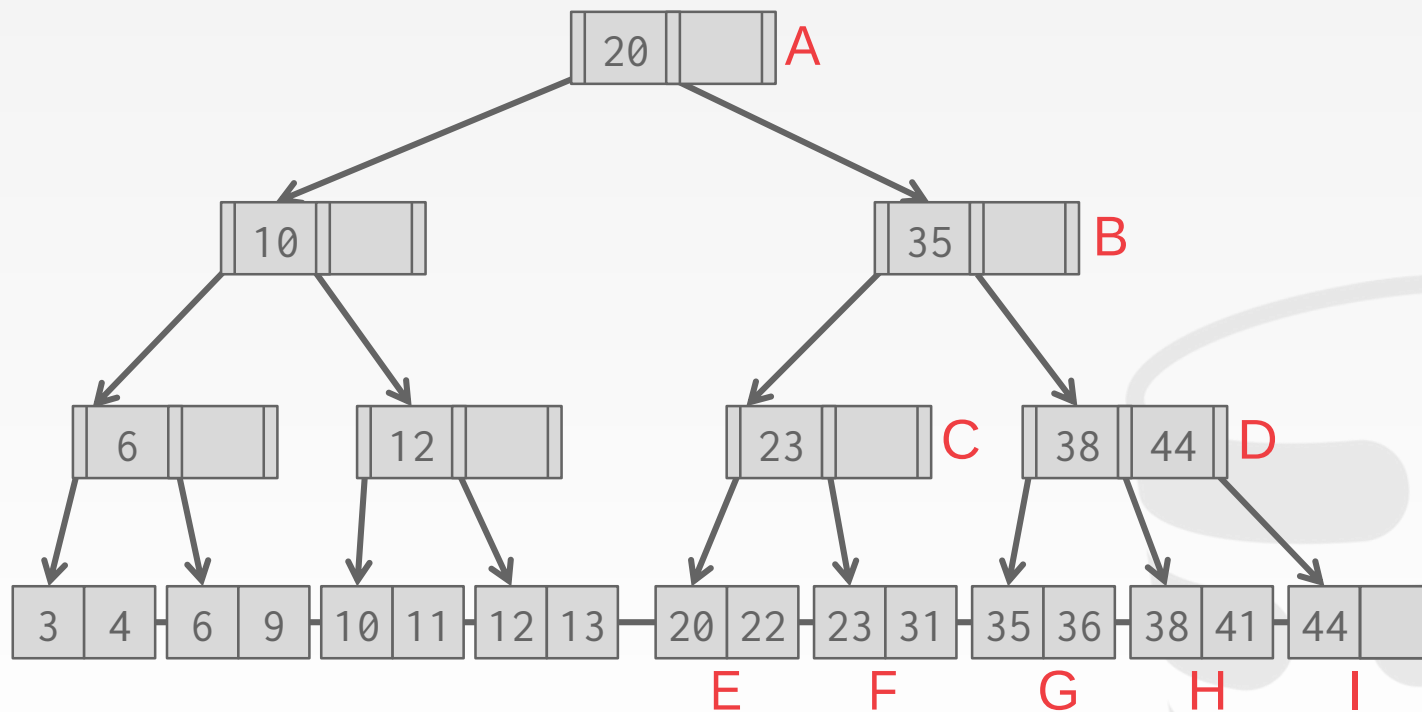
EXAMPLE #2 – DELETE 38



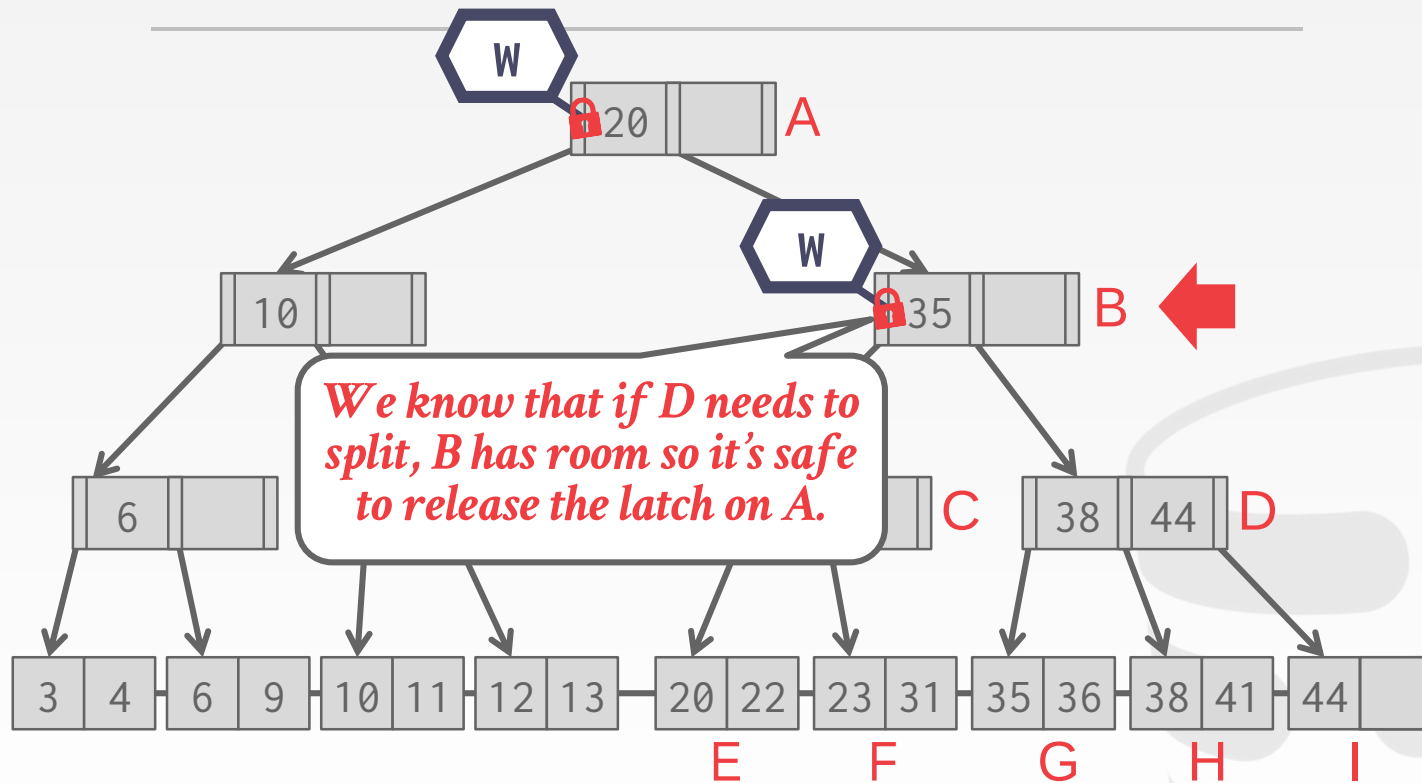
EXAMPLE #2 – DELETE 38



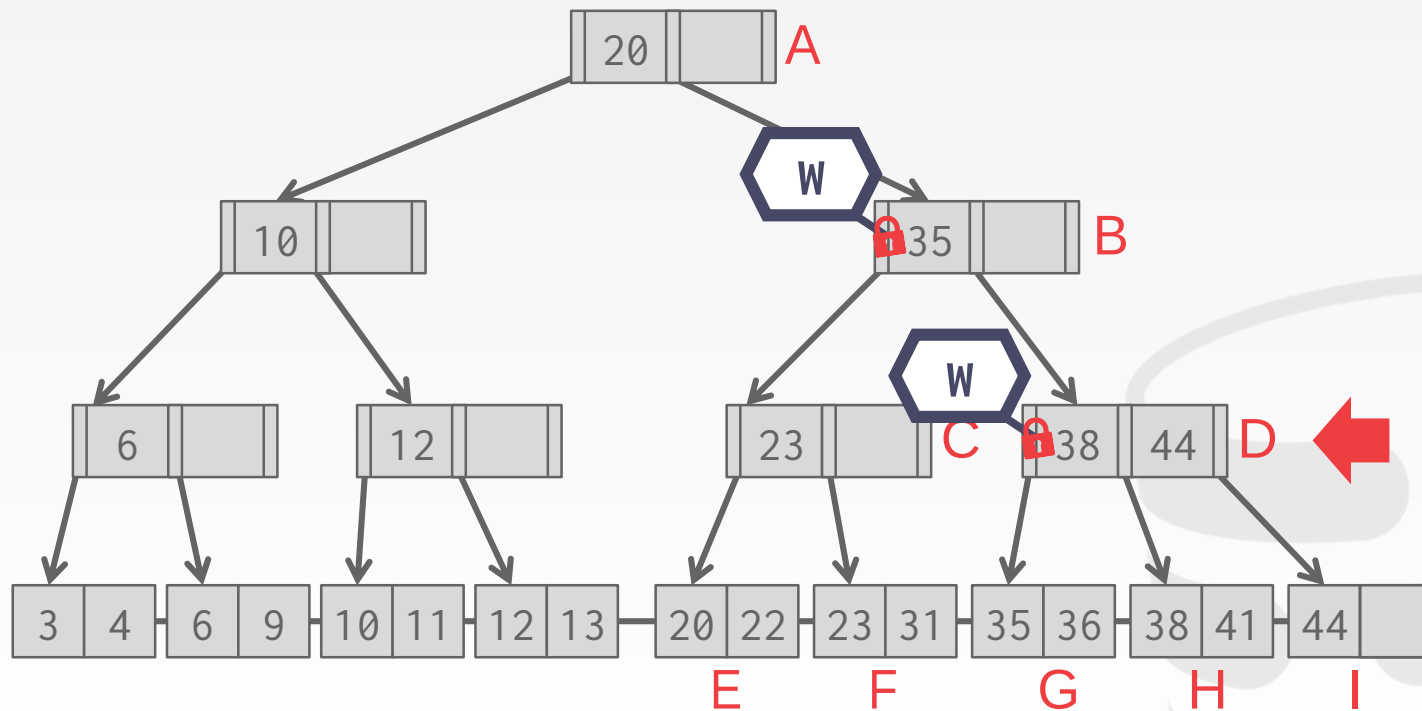
EXAMPLE #3 – INSERT 45



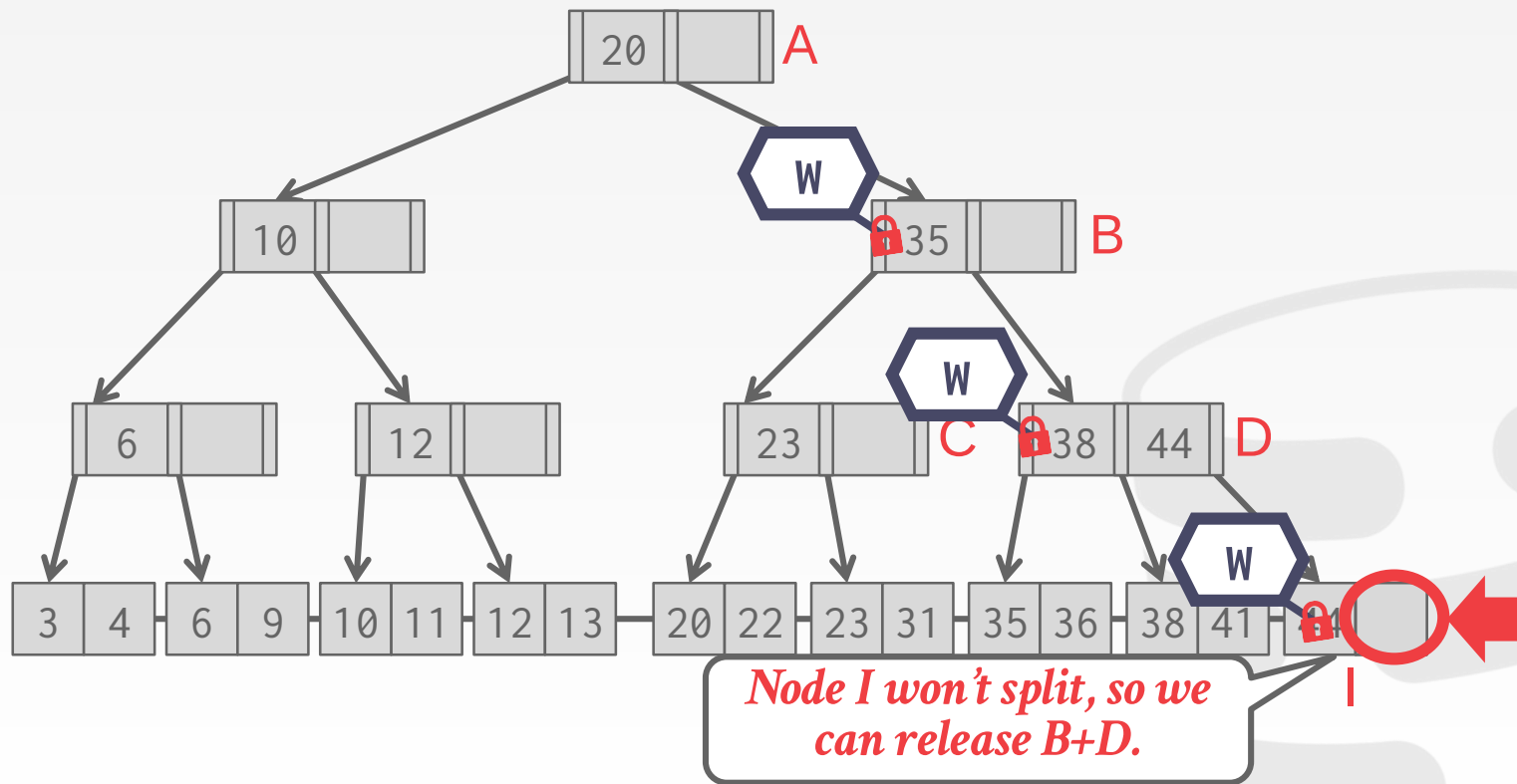
EXAMPLE #3 – INSERT 45



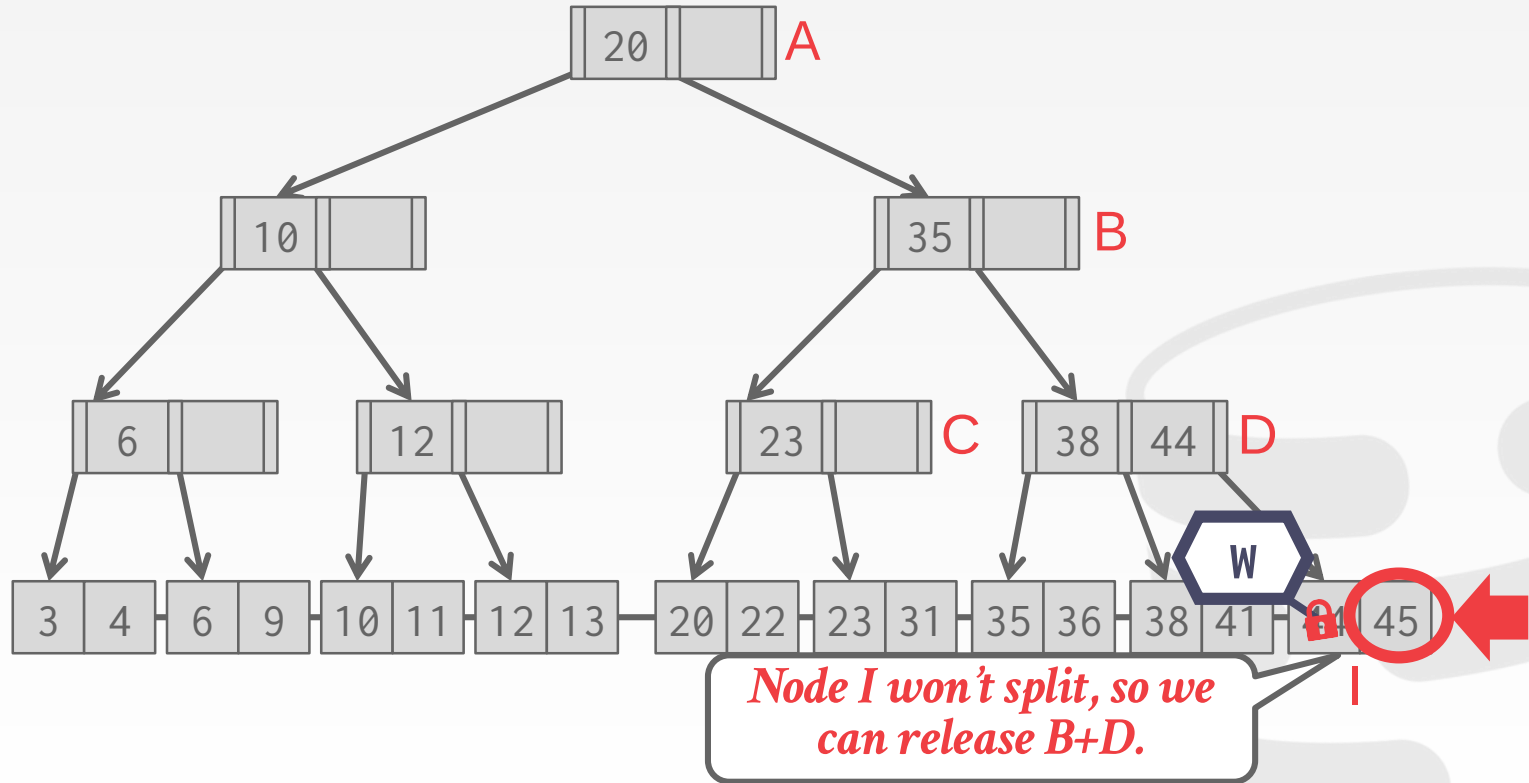
EXAMPLE #3 – INSERT 45



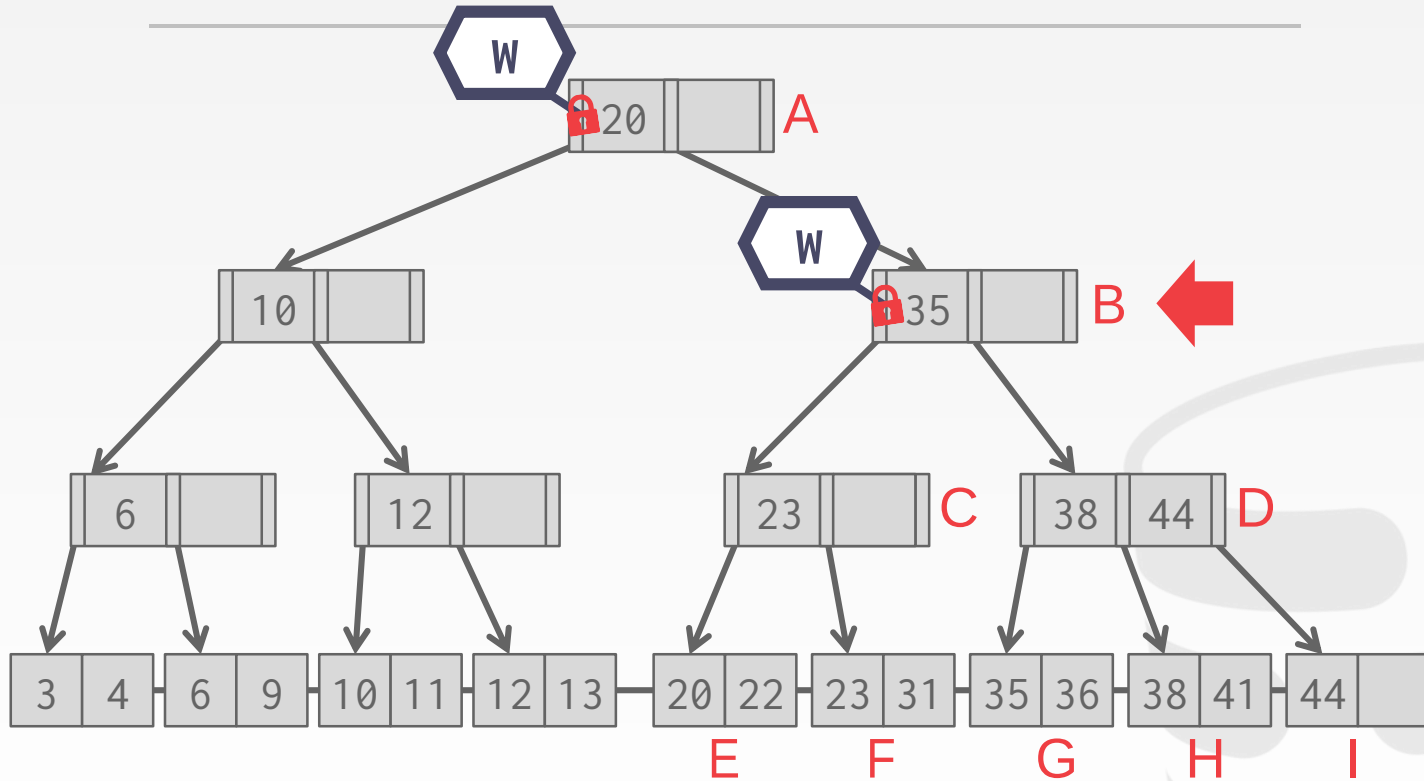
EXAMPLE #3 – INSERT 45



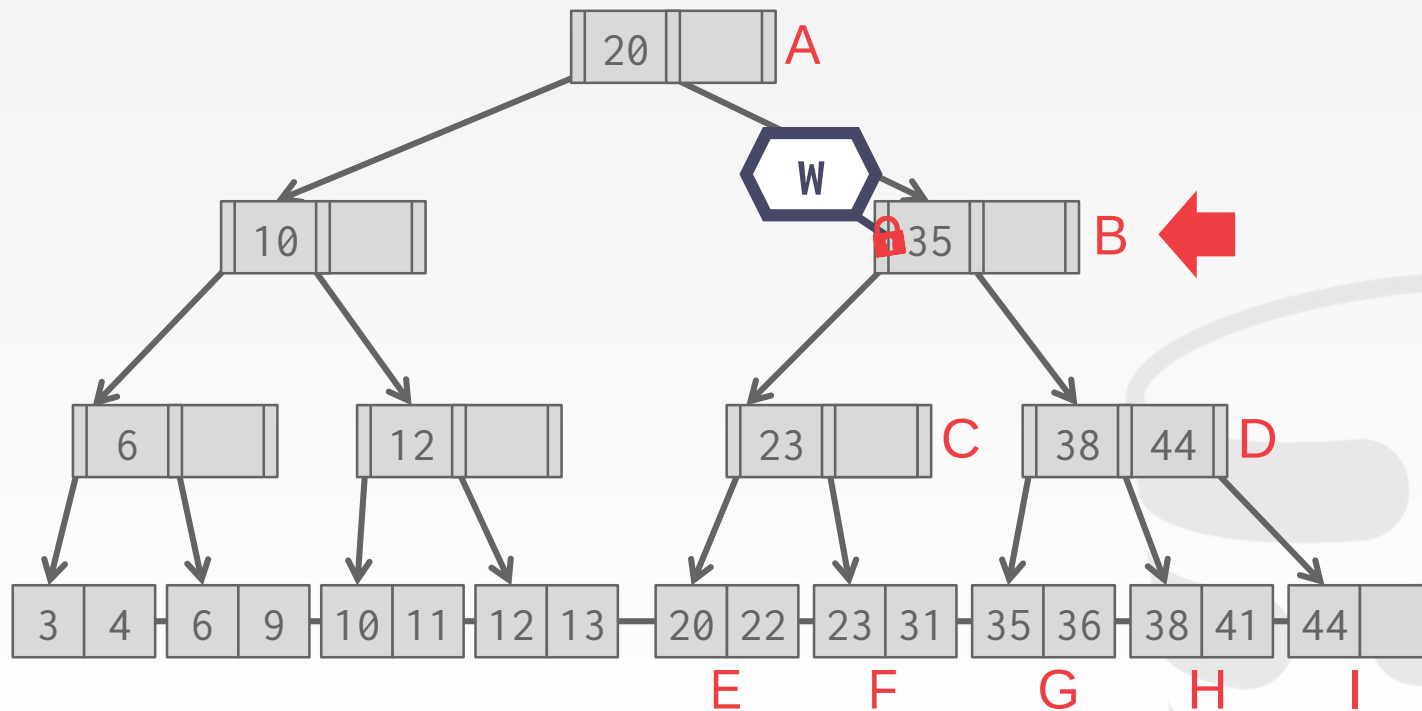
EXAMPLE #3 – INSERT 45



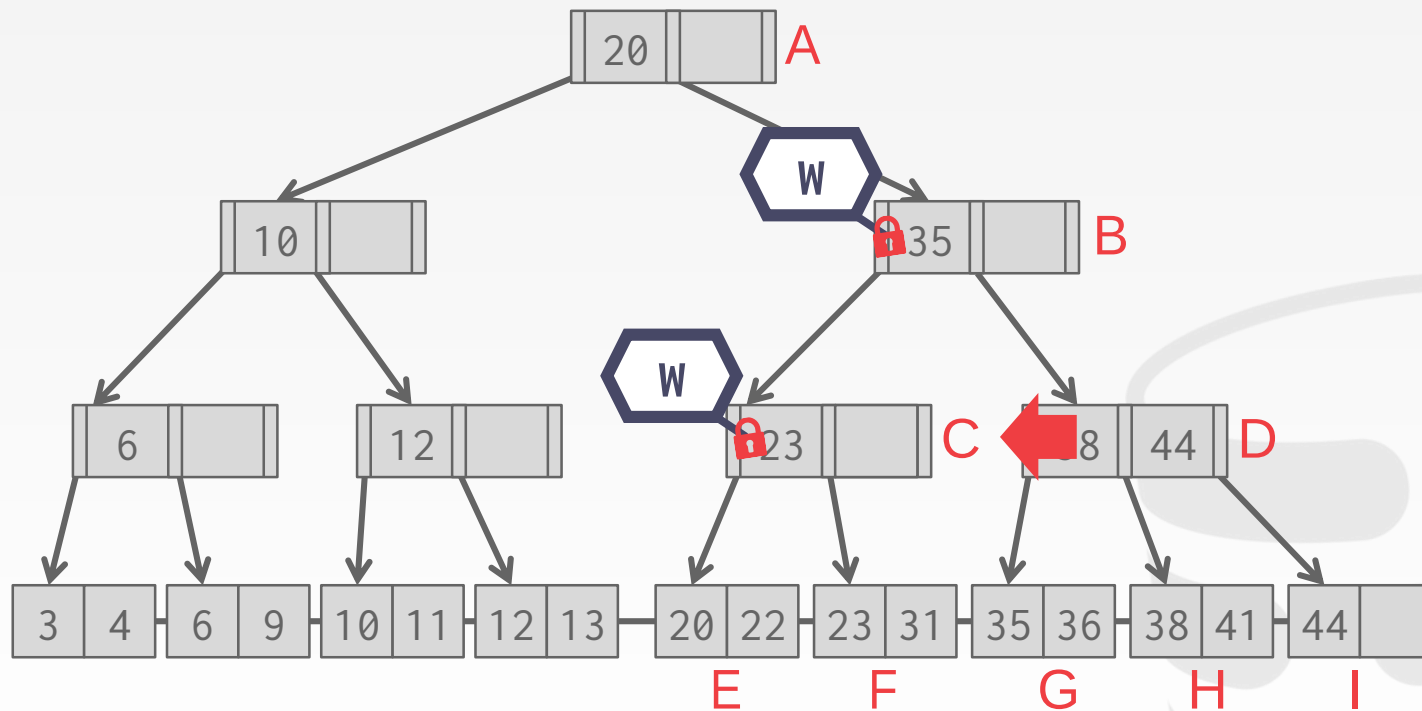
EXAMPLE #4 – INSERT 25



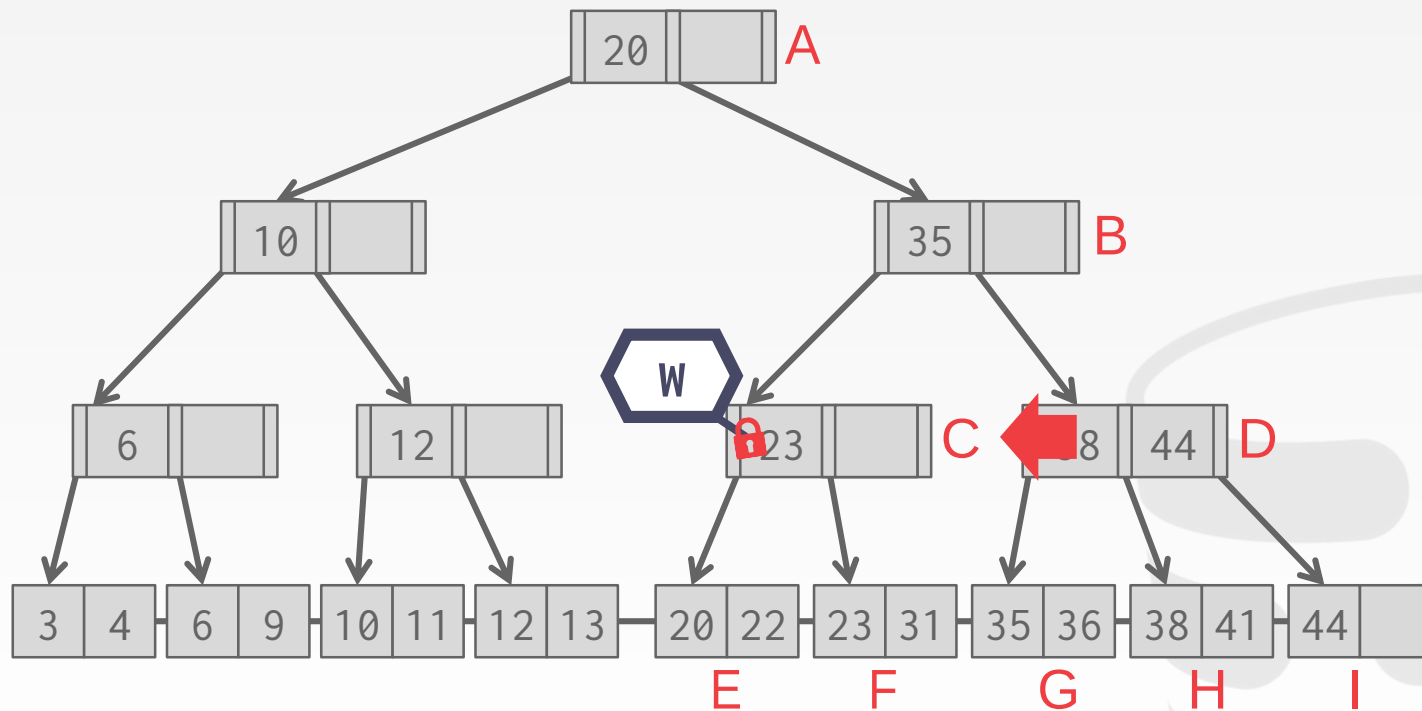
EXAMPLE #4 – INSERT 25



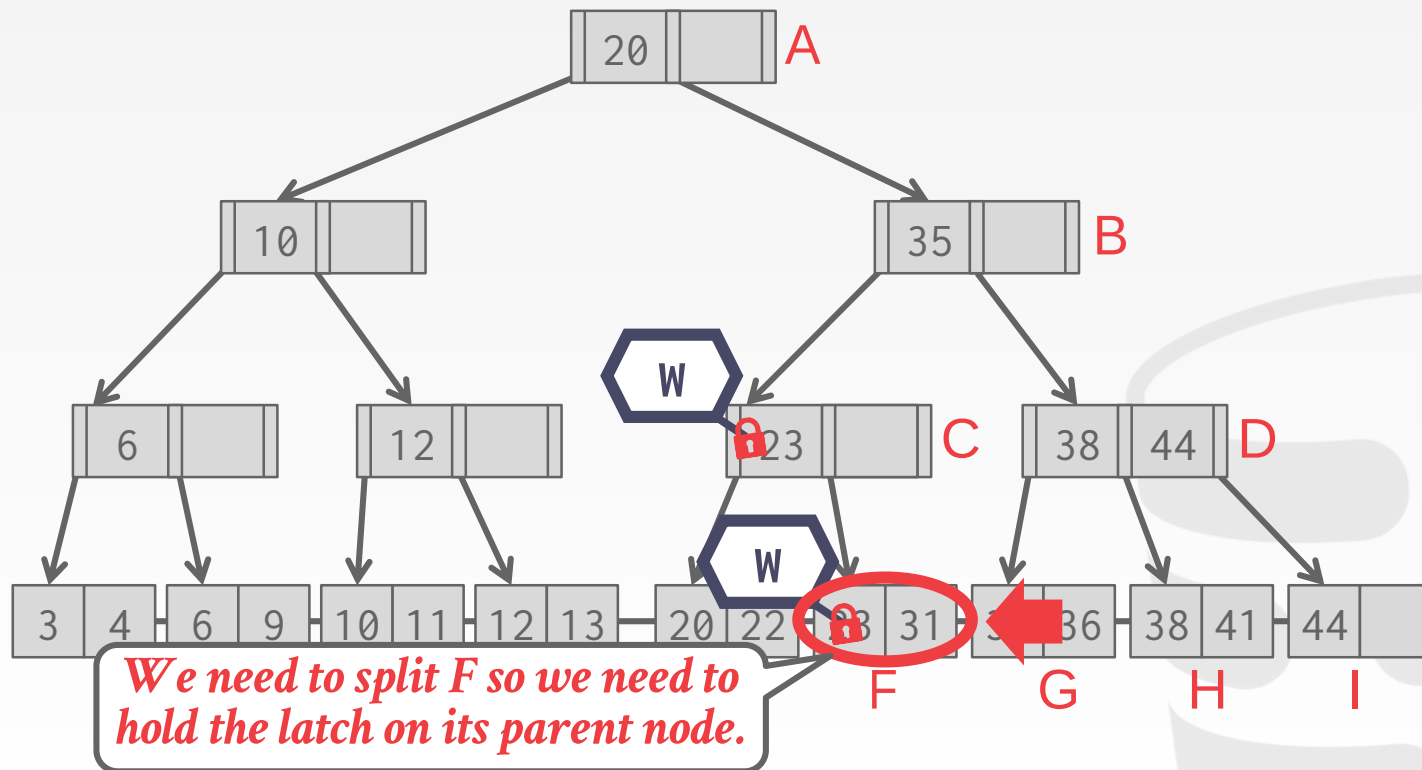
EXAMPLE #4 – INSERT 25



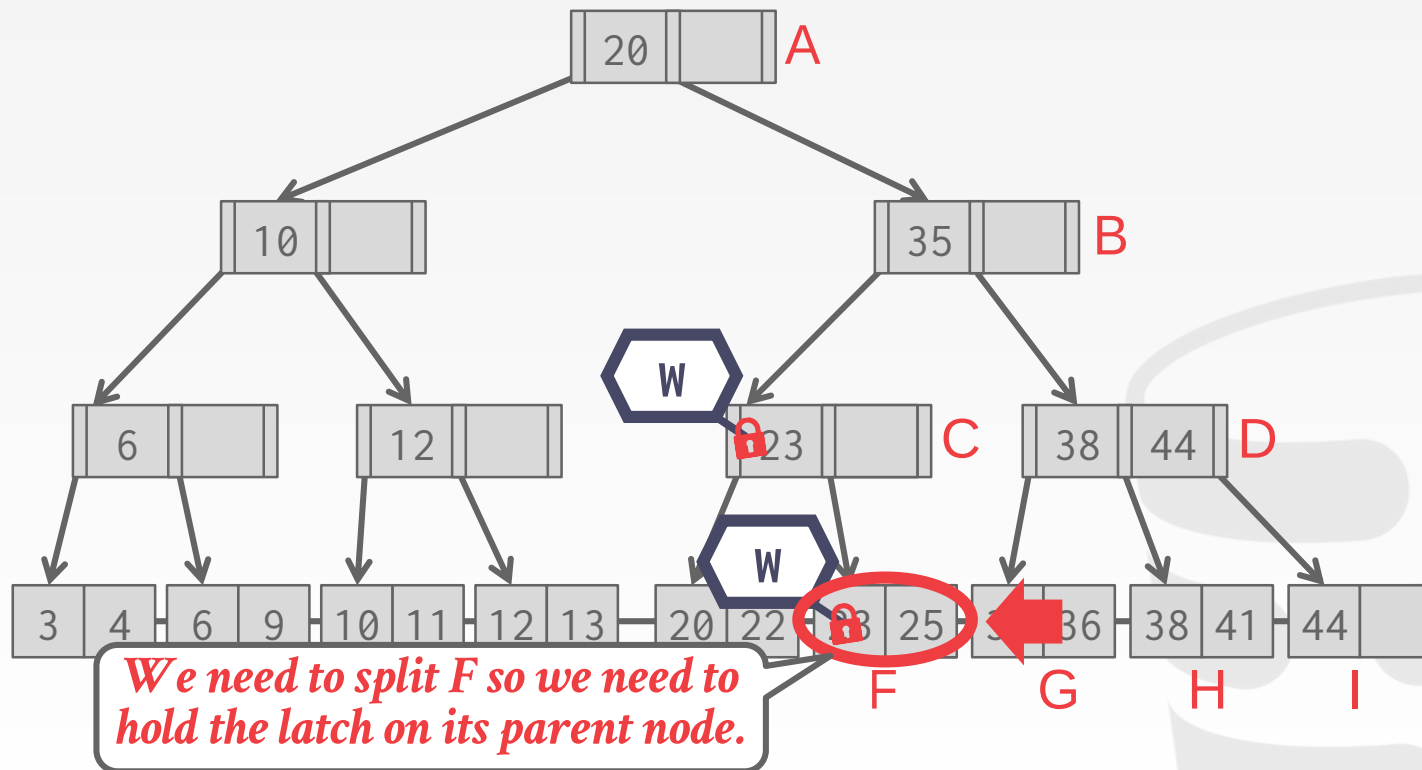
EXAMPLE #4 – INSERT 25



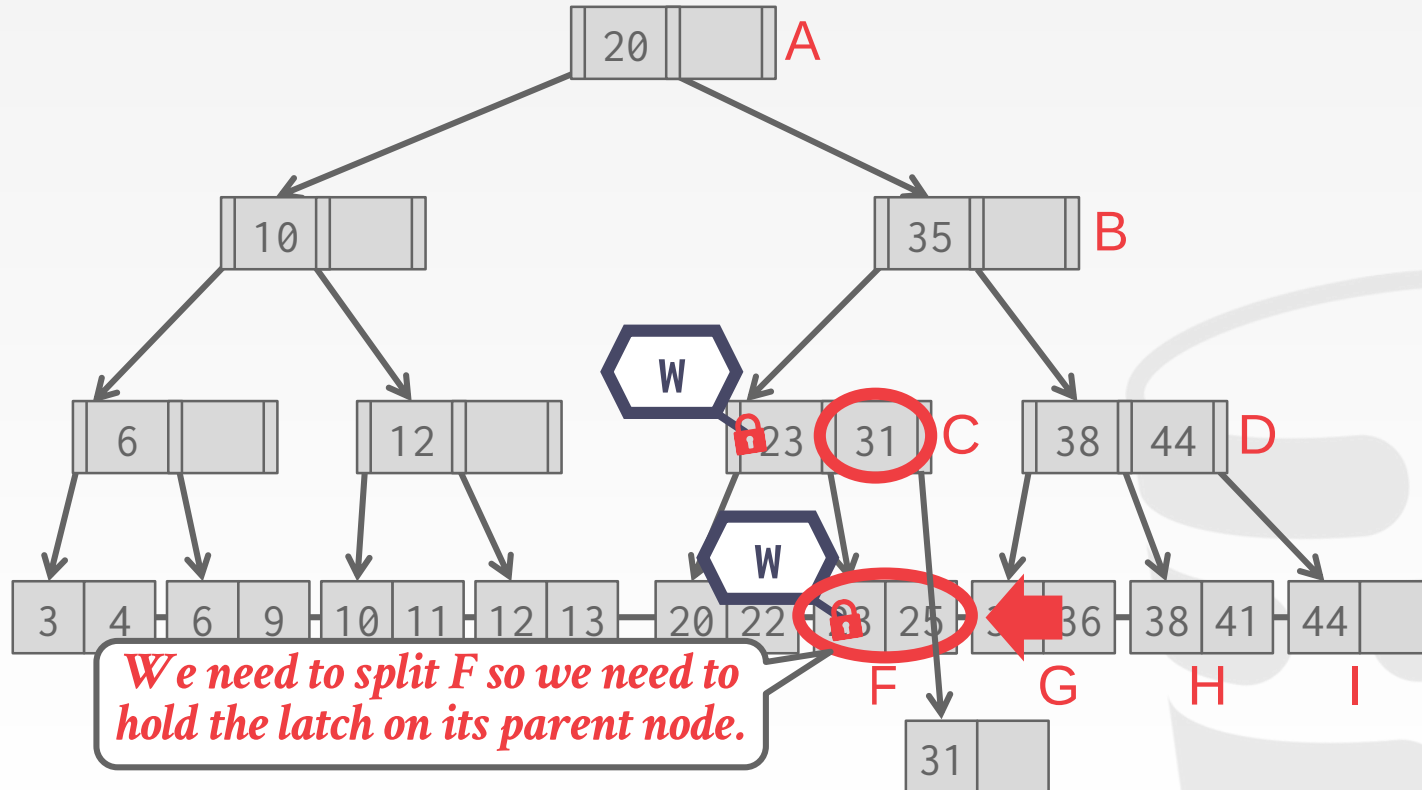
EXAMPLE #4 – INSERT 25



EXAMPLE #4 – INSERT 25



EXAMPLE #4 – INSERT 25



OBSERVATION

What was the first step that all the update examples did on the B+Tree?

Delete 38



Insert 45



Insert 25



OBSERVATION

What was the first step that all the update examples did on the B+Tree?

Taking a write latch on the root every time becomes a bottleneck with higher concurrency.

Can we do better?

BETTER LATCHING ALGORITHM

Assume that the leaf node is safe.

Use read latches and crabbing to reach it, and then verify that it is safe.

If leaf is not safe, then do previous algorithm using write latches.

Acta Informatica 9, 1–21 (1977)



Concurrency of Operations on *B*-Trees

R. Bayer* and M. Schkolnick
IBM Research Laboratory, San José, CA 95193, USA

Summary. Concurrent operations on *B*-trees pose the problem of insuring that each operation can be carried out without interfering with other operations being performed simultaneously by other users. This problem can become critical if these structures are being used to support access paths, like indexes, to data base systems. In this case, serializing access to one of these indexes can create an unacceptable bottleneck for the entire system. Thus, there is a need for locking protocols that can assure integrity for each access while at the same time providing a maximum possible degree of concurrency. Another feature required from these protocols is that they be deadlock free, since the cost to resolve a deadlock may be high. Recently, there has been some questioning on whether *B*-tree structures can support concurrent operations. In this paper, we examine the problem of concurrent access to *B*-trees. We present a deadlock free solution which can be tuned to specific requirements. An analysis is presented which allows the selection of parameters so as to satisfy these requirements.

The solution presented here uses simple locking protocols. Thus, we conclude that *B*-trees can be used advantageously in a multi-user environment.

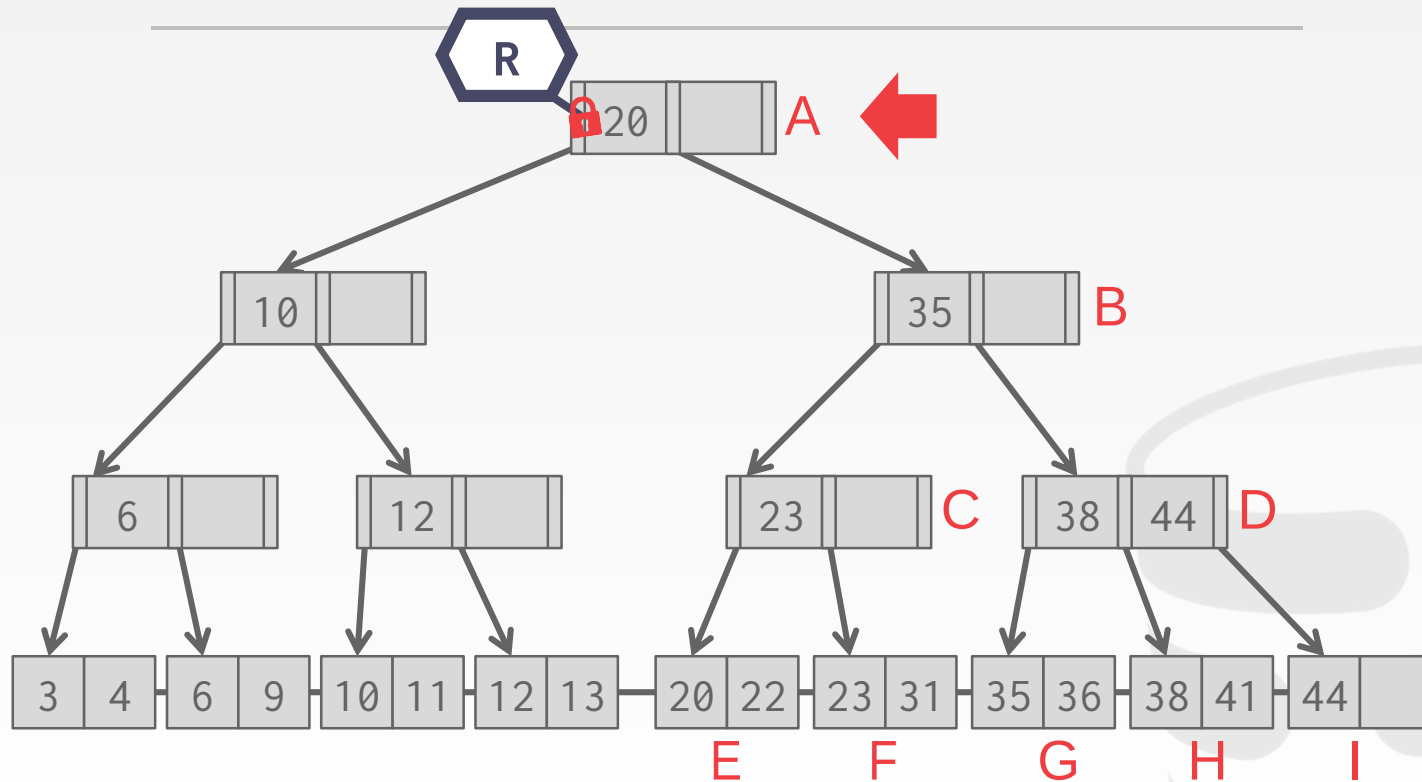
1. Introduction

In this paper, we examine the problem of concurrent access to indexes which are maintained as *B*-trees. This type of organization was introduced by Bayer and McCreight [2] and some variants of it appear in Knuth [10] and Wedekind [13]. Performance studies of it were restricted to the single user environment. Recently, these structures have been examined for possible use in a multi-user (concurrent) environment. Some initial studies have been made about the feasibility of their use in this type of situation [1, 6], and [11].

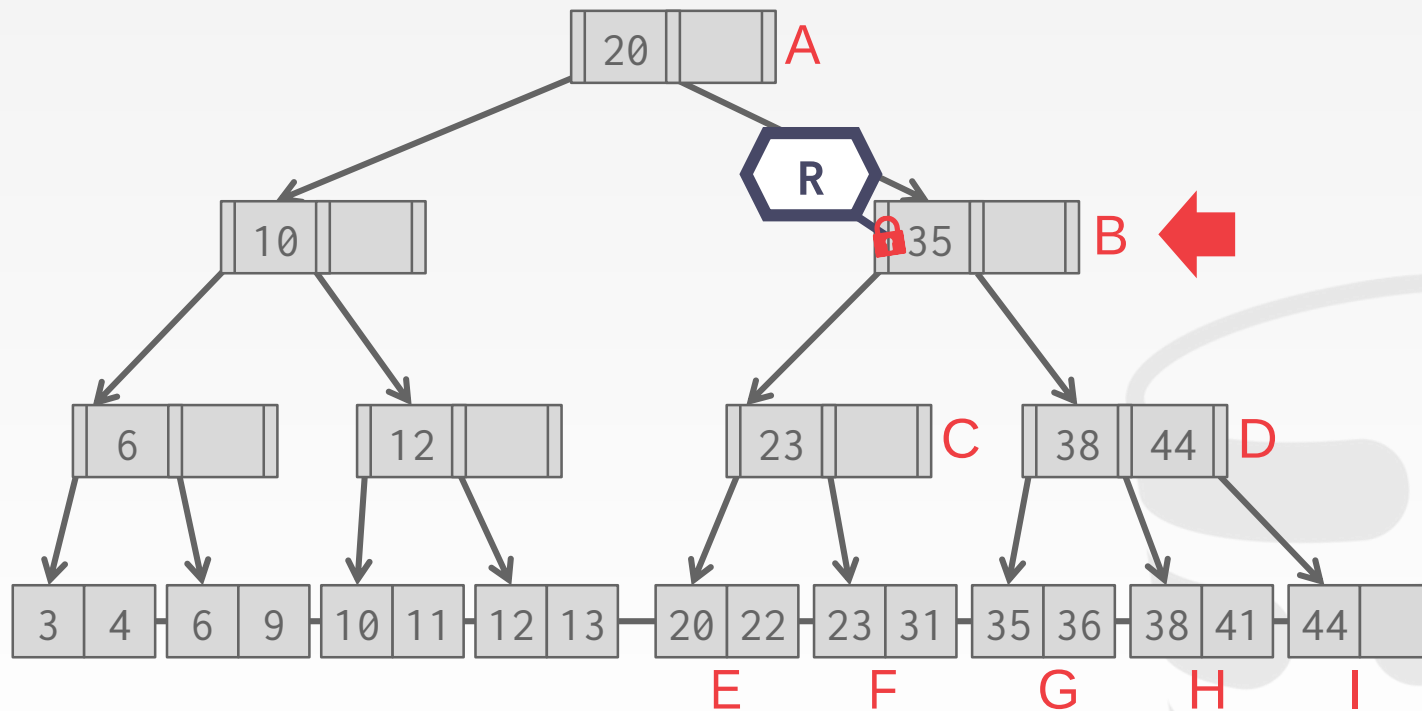
An accessing schema which achieves a high degree of concurrency in using the index will be presented. The schema allows dynamic tuning to adapt its performance to the profile of the current set of users. Another property of the

* Permanent address: Institut für Informatik der Technischen Universität München, Arcisstr. 21, D-8000 München 2, Germany (Fed. Rep.)

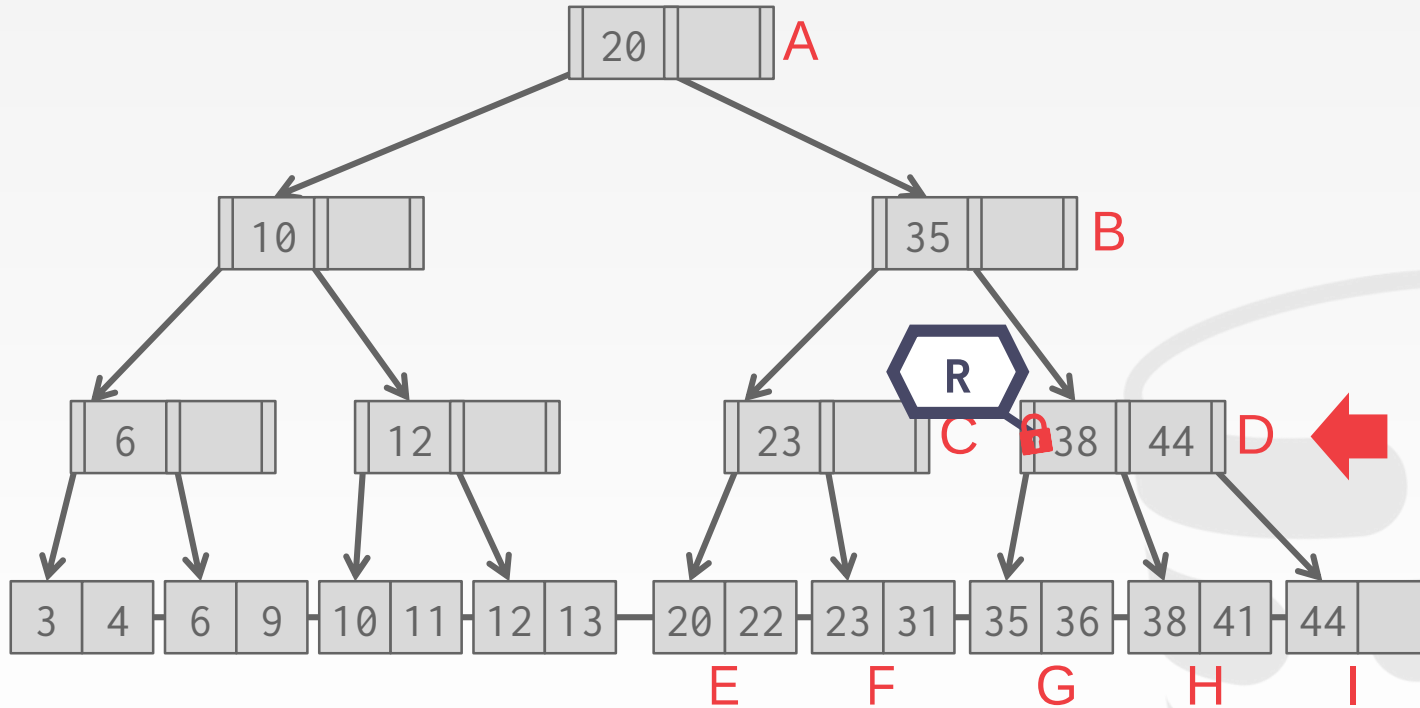
EXAMPLE #2 – DELETE 38



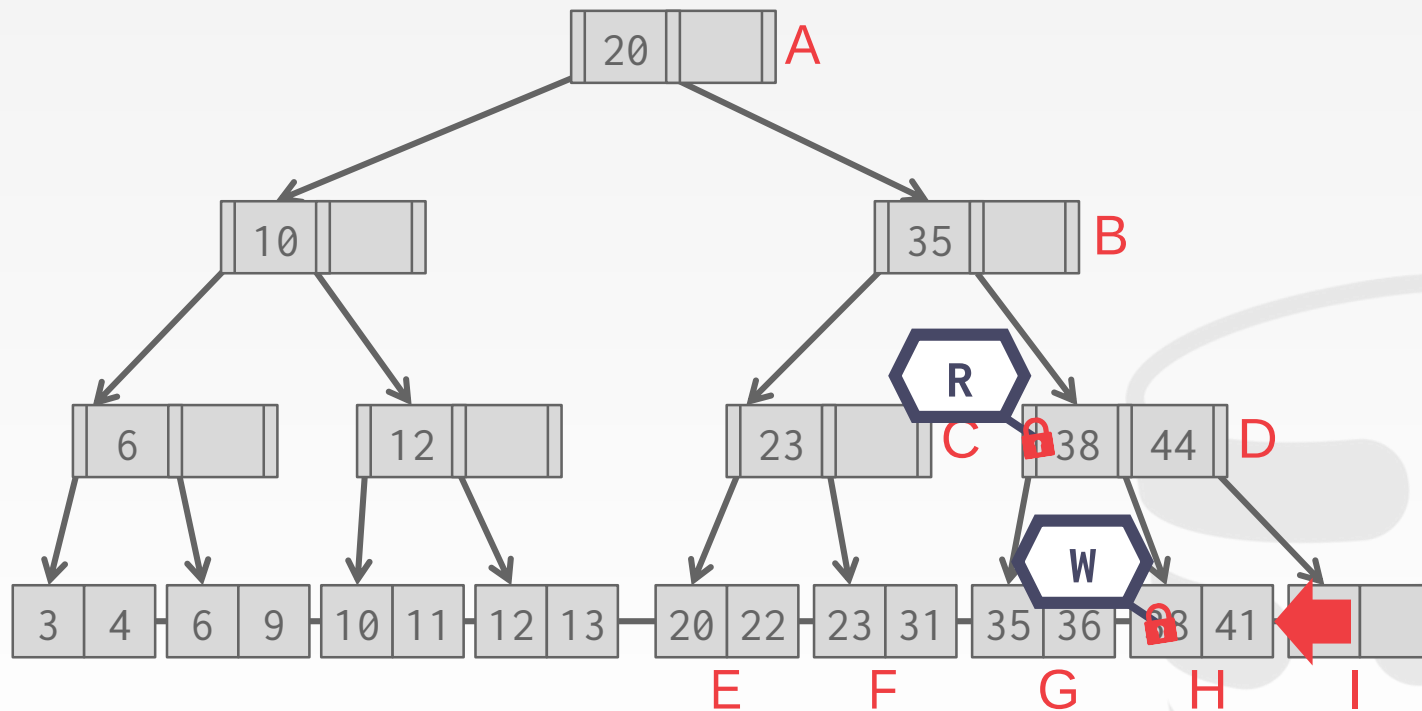
EXAMPLE #2 – DELETE 38



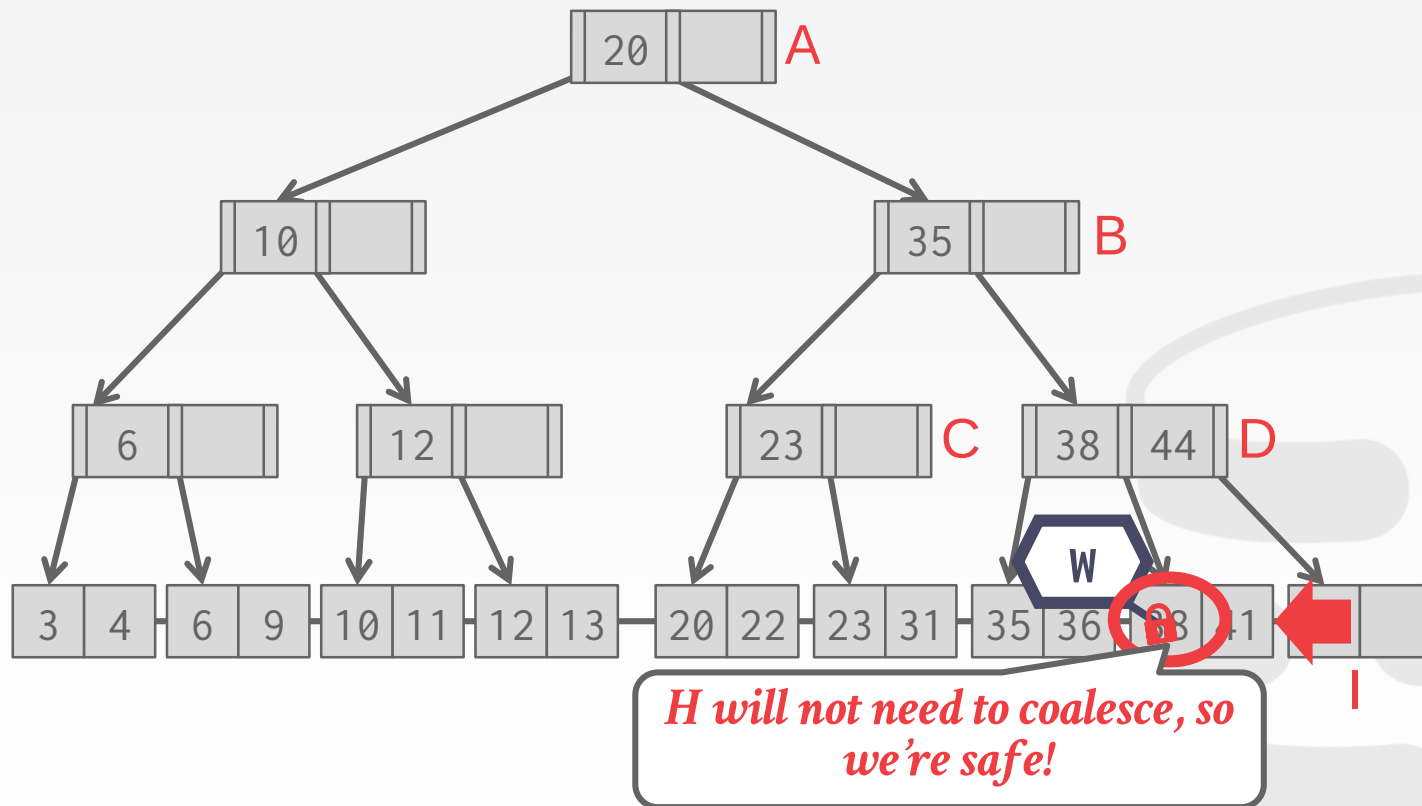
EXAMPLE #2 – DELETE 38



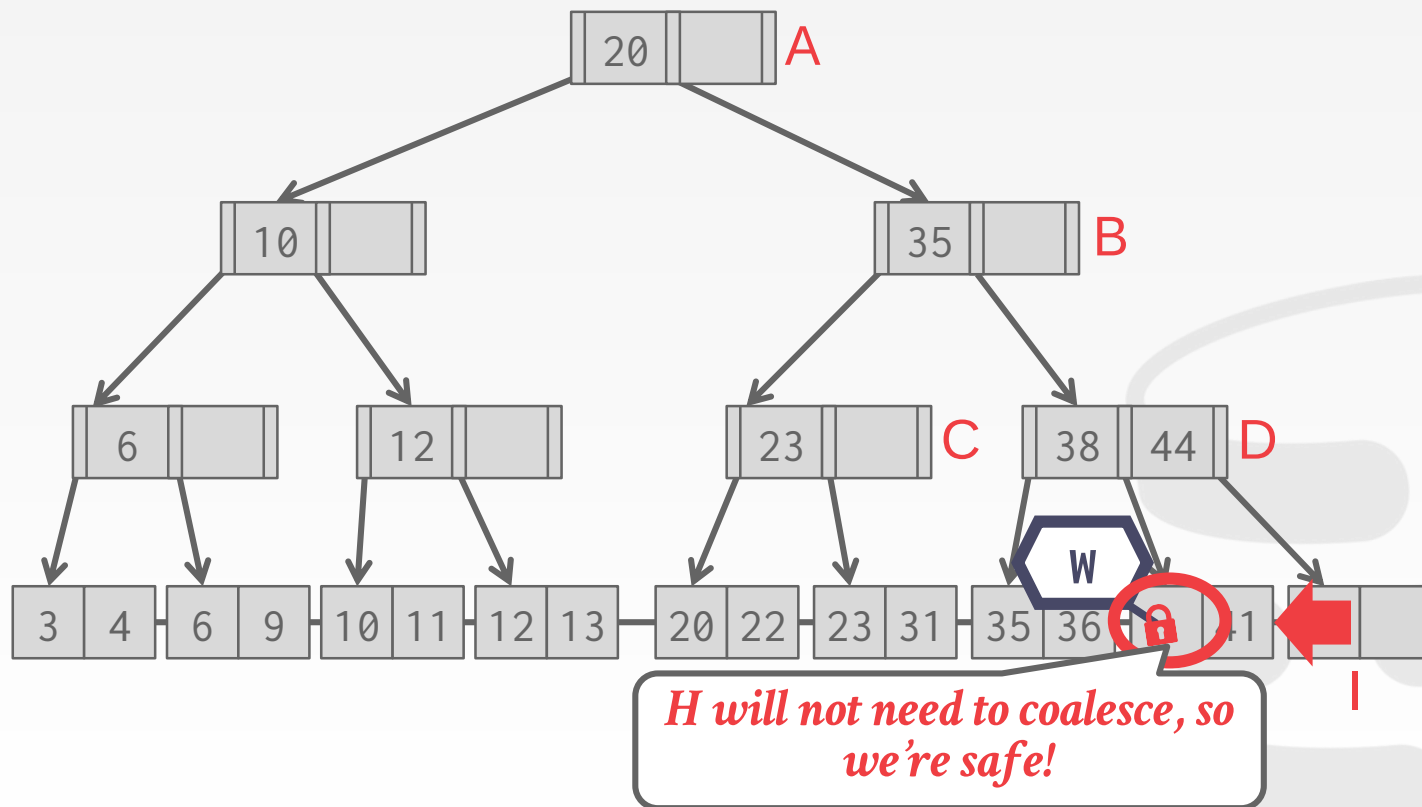
EXAMPLE #2 – DELETE 38



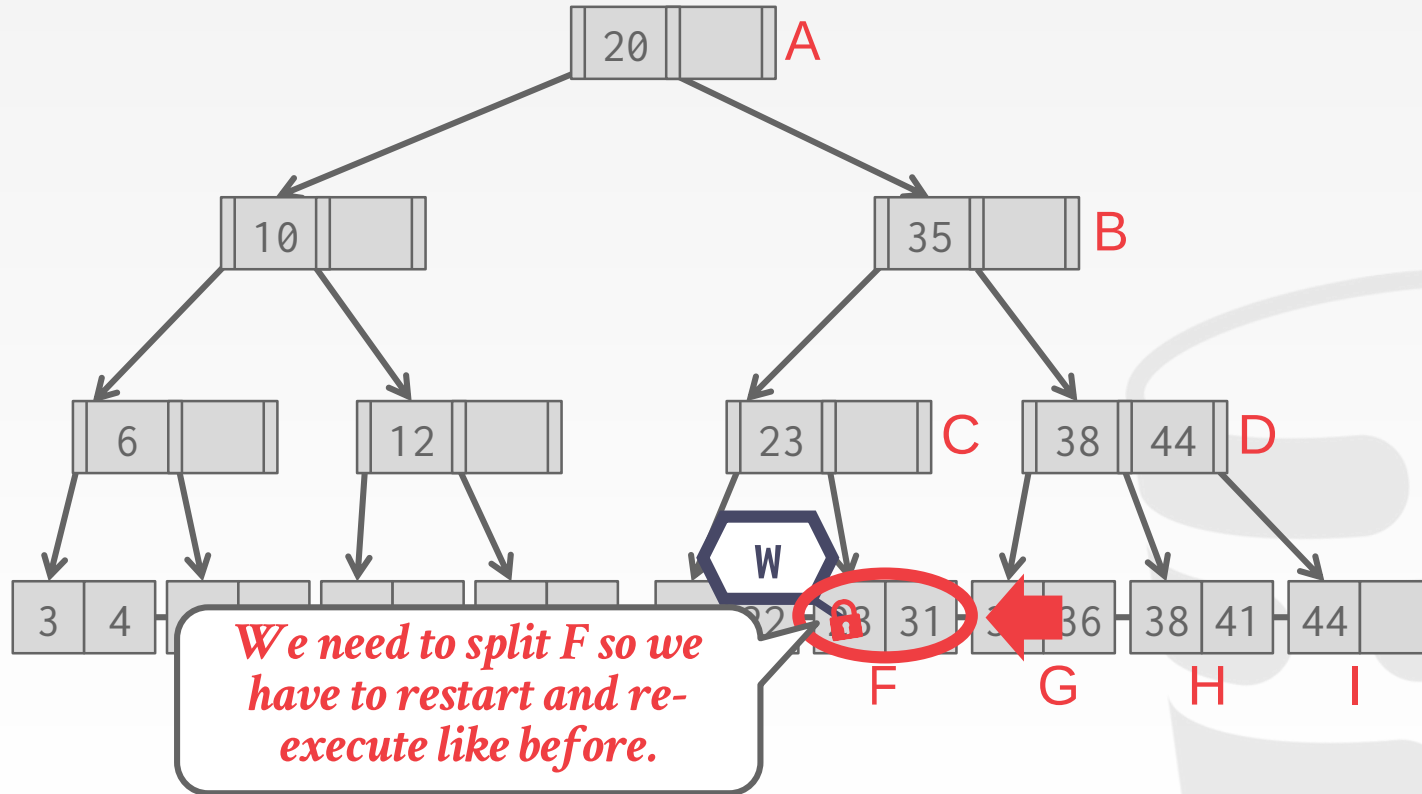
EXAMPLE #2 – DELETE 38



EXAMPLE #2 – DELETE 38



EXAMPLE #4 – INSERT 25



BETTER LATCHING ALGORITHM

Search: Same as before.

Insert/Delete:

- Set latches as if for search, get to leaf, and set **W** latch on leaf.
- If leaf is not safe, release all latches, and restart thread using previous insert/delete protocol with write latches.

This approach optimistically assumes that only leaf node will be modified; if not, **R** latches set on the first pass to leaf are wasteful.

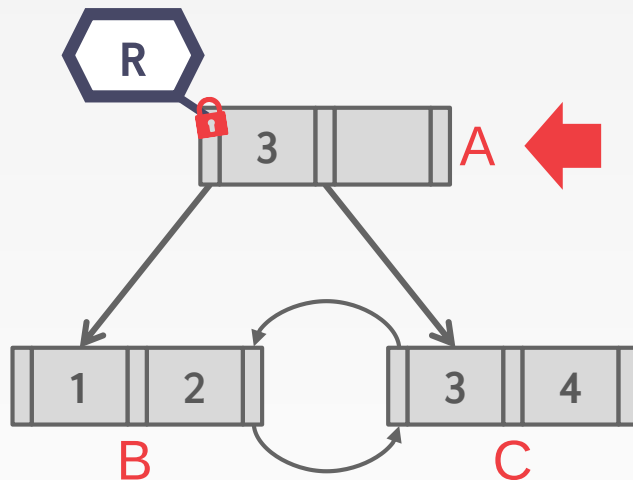
OBSERVATION

The threads in all the examples so far have acquired latches in a "top-down" manner.

- A thread can only acquire a latch from a node that is below its current node.
- If the desired latch is unavailable, the thread must wait until it becomes available.

But what if we want to move from one leaf node to another leaf node?

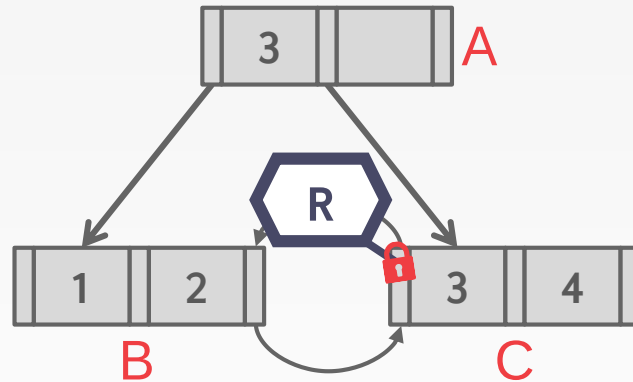
LEAF NODE SCAN EXAMPLE #1



T_1 : Find Keys < 4

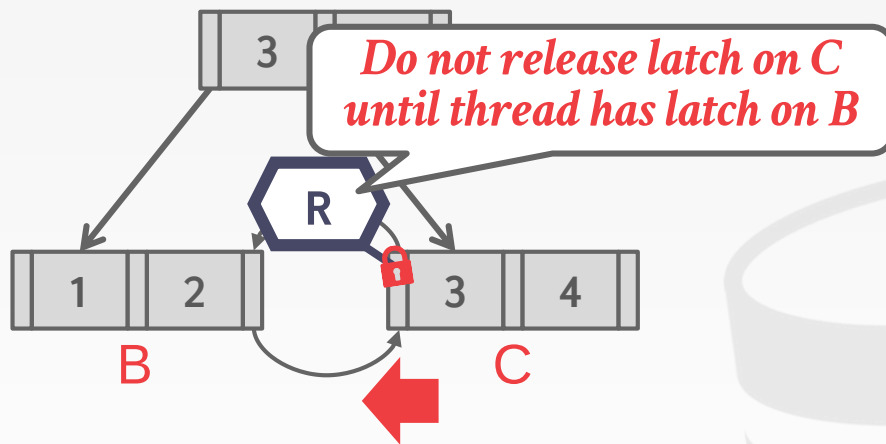
LEAF NODE SCAN EXAMPLE #1

T_1 : Find Keys < 4



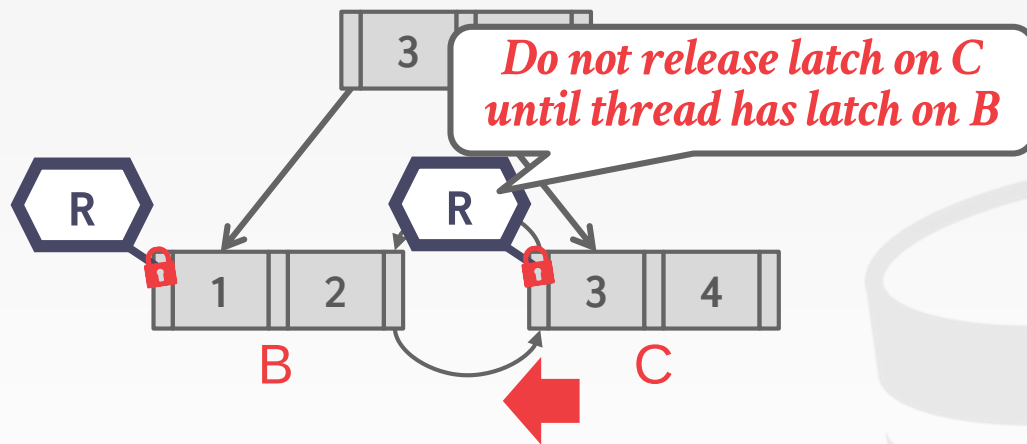
LEAF NODE SCAN EXAMPLE #1

T_1 : Find Keys < 4



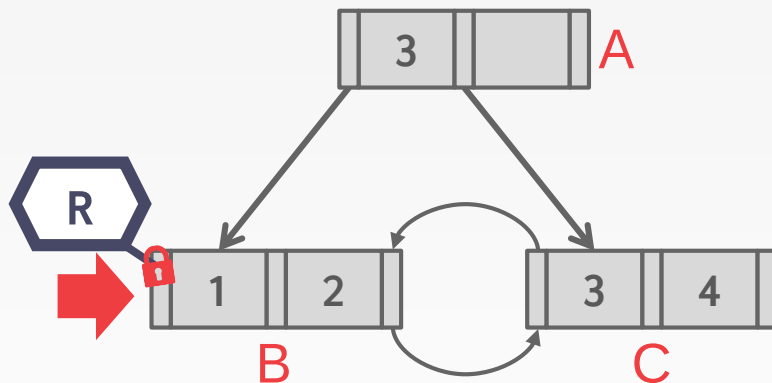
LEAF NODE SCAN EXAMPLE #1

T_1 : Find Keys < 4



LEAF NODE SCAN EXAMPLE #1

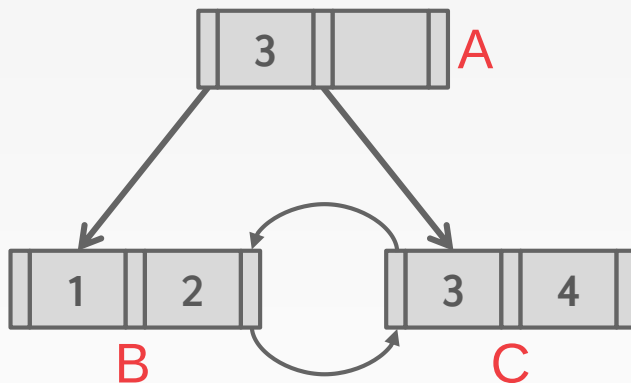
T_1 : Find Keys < 4



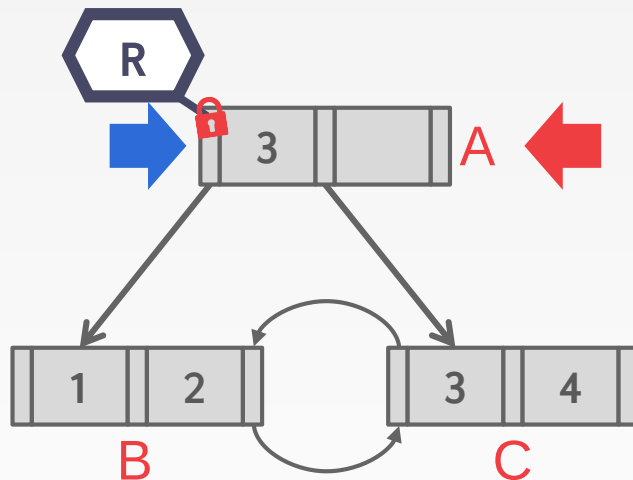
LEAF NODE SCAN EXAMPLE #2

T_1 : Find Keys < 4

T_2 : Find Keys > 1



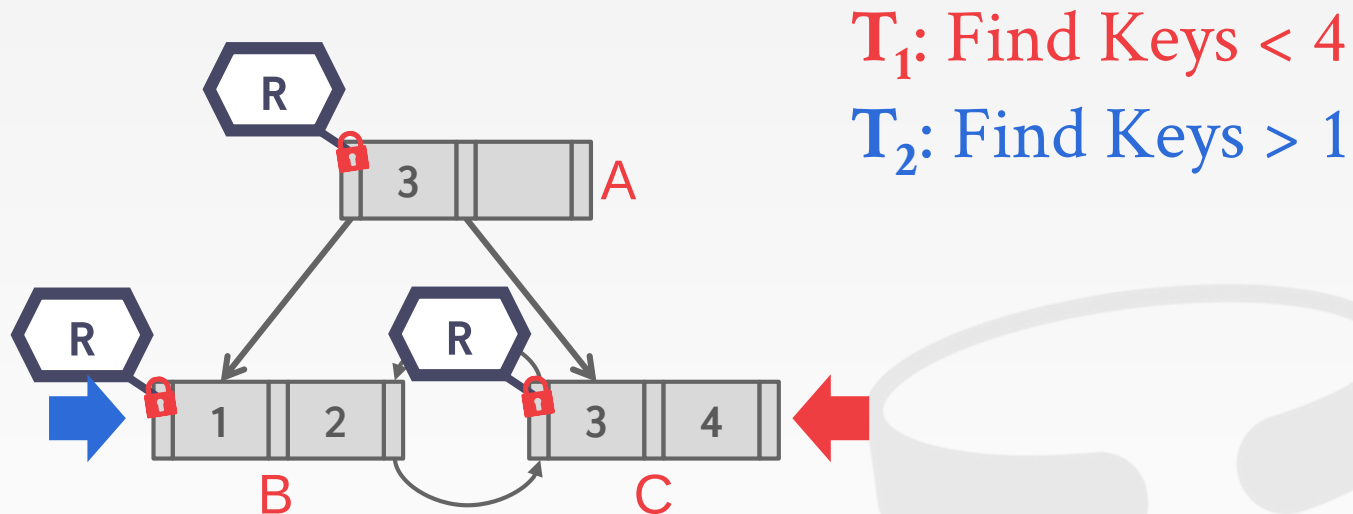
LEAF NODE SCAN EXAMPLE #2



T_1 : Find Keys < 4

T_2 : Find Keys > 1

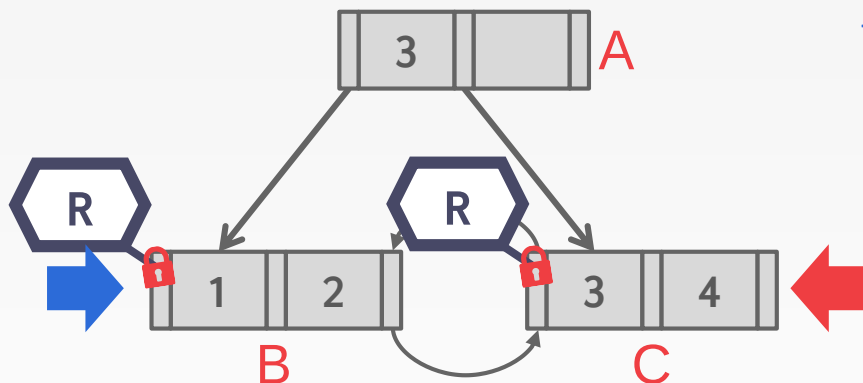
LEAF NODE SCAN EXAMPLE #2



LEAF NODE SCAN EXAMPLE #2

T_1 : Find Keys < 4

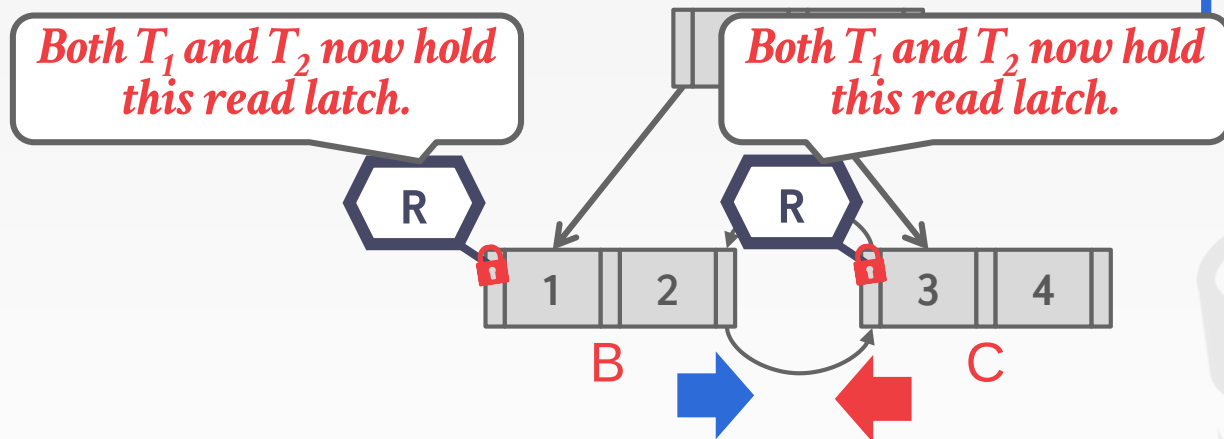
T_2 : Find Keys > 1



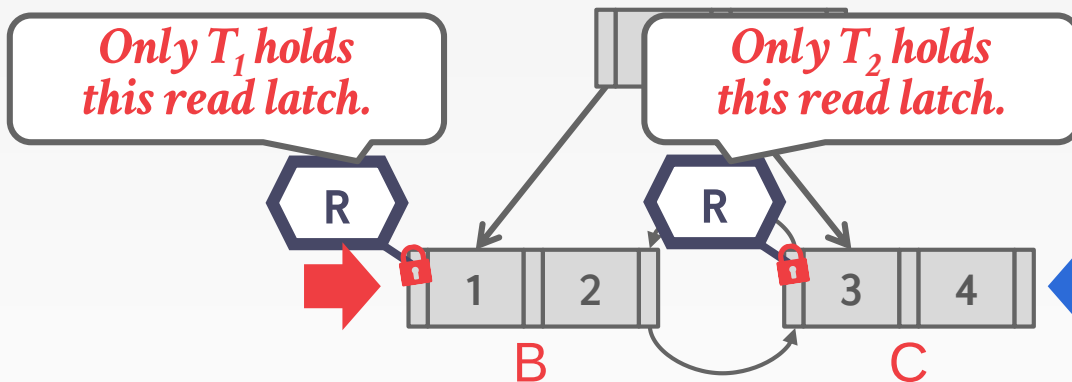
LEAF NODE SCAN EXAMPLE #2

T_1 : Find Keys < 4

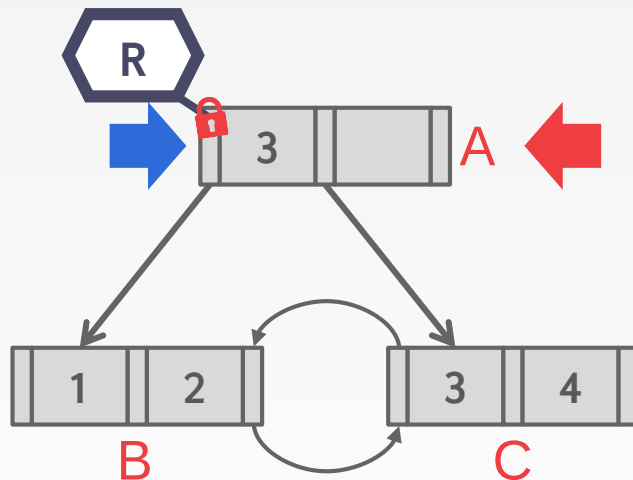
T_2 : Find Keys > 1



LEAF NODE SCAN EXAMPLE #2



LEAF NODE SCAN EXAMPLE #3



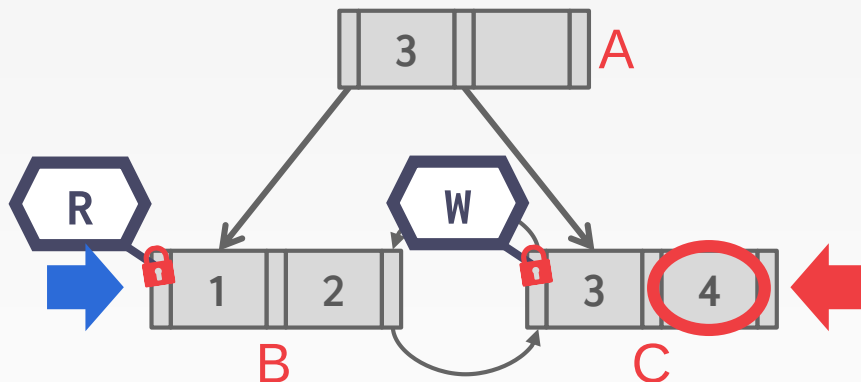
T_1 : Delete 4

T_2 : Find Keys > 1

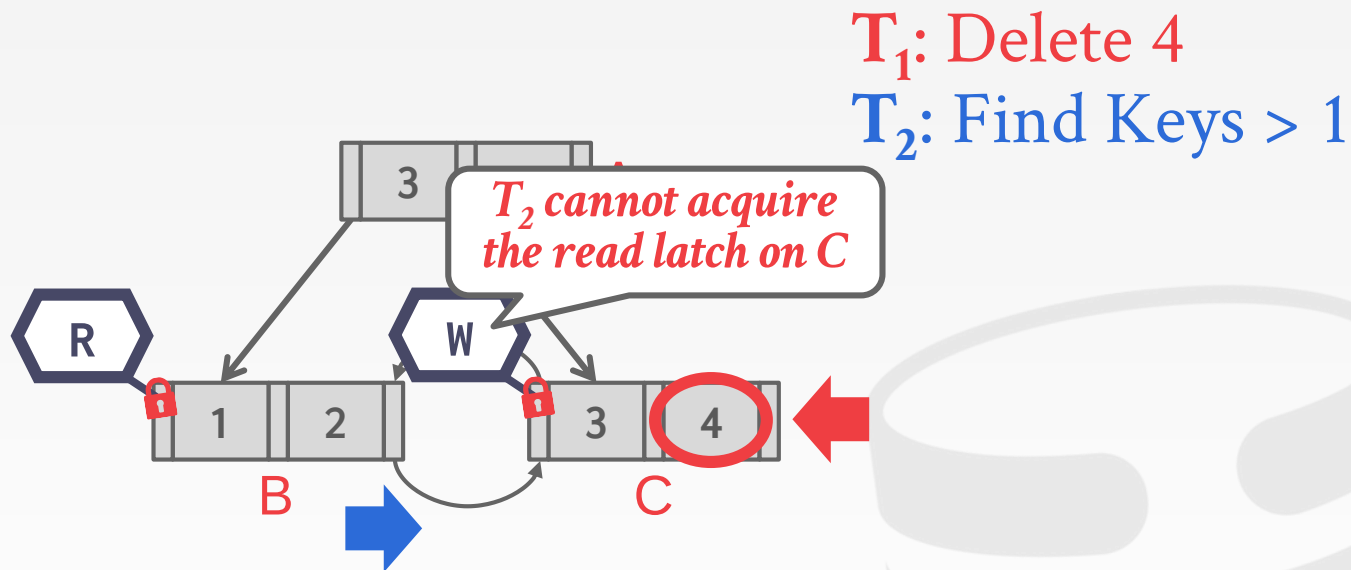
LEAF NODE SCAN EXAMPLE #3

T_1 : Delete 4

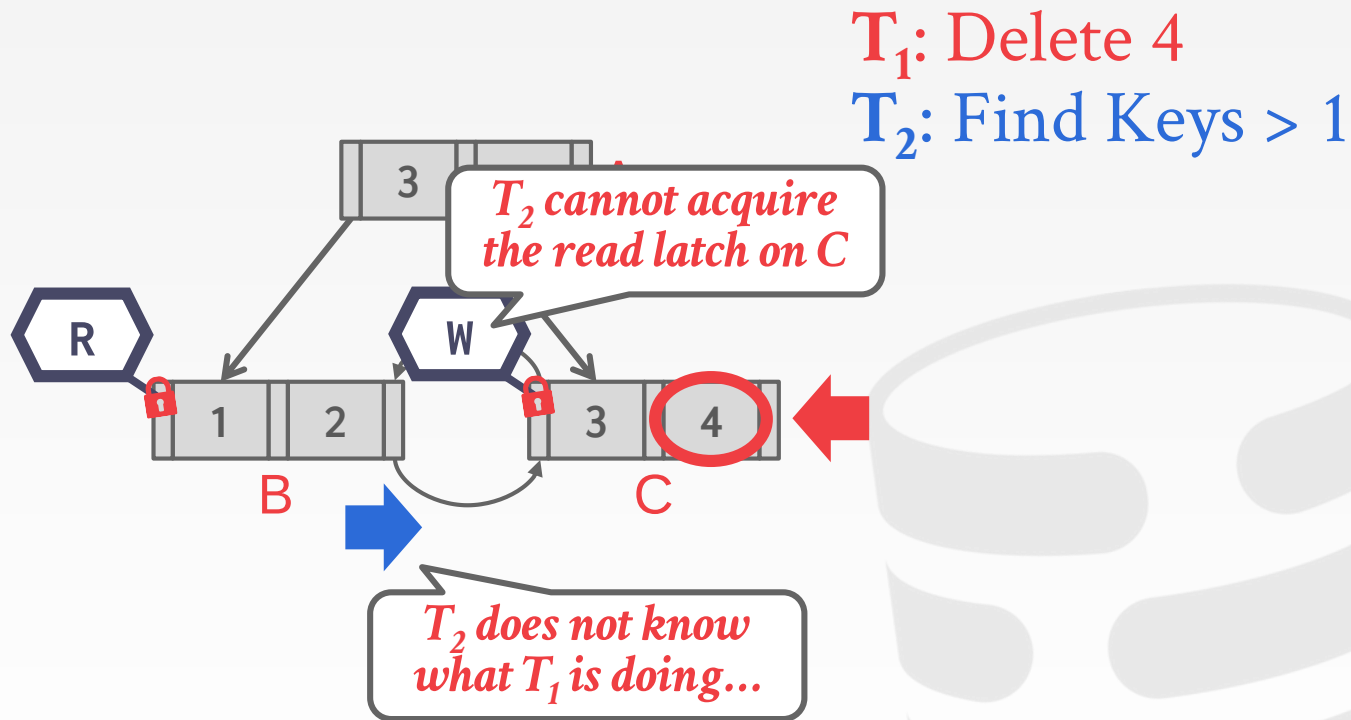
T_2 : Find Keys > 1



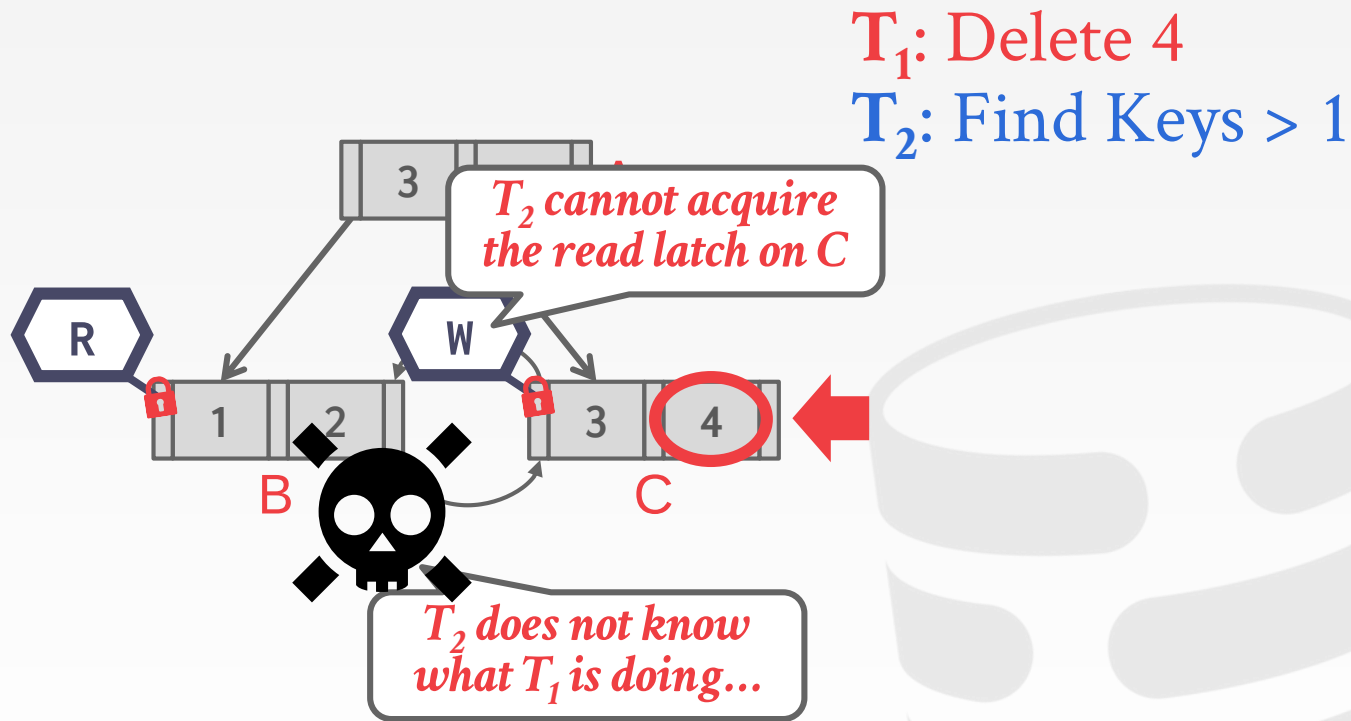
LEAF NODE SCAN EXAMPLE #3



LEAF NODE SCAN EXAMPLE #3



LEAF NODE SCAN EXAMPLE #3



LEAF NODE SCANS

Latches do not support deadlock detection or avoidance. The only way we can deal with this problem is through coding discipline.

The leaf node sibling latch acquisition protocol must support a "no-wait" mode.

The DBMS's data structures must cope with failed latch acquisitions.

DELAYED PARENT UPDATES

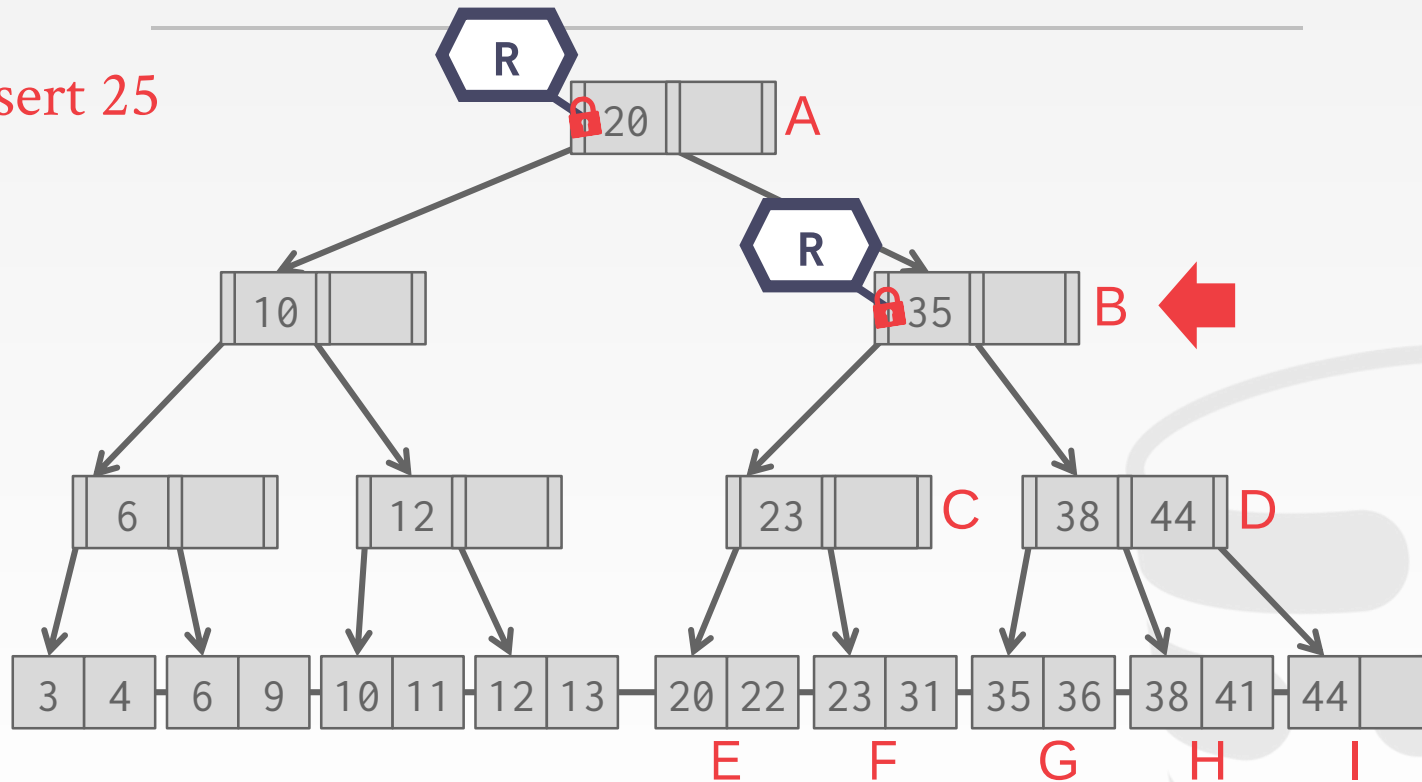
Every time a leaf node overflows, we must update at least three nodes.

- The leaf node being split.
- The new leaf node being created.
- The parent node.

B^{link}-Tree Optimization: When a leaf node overflows, delay updating its parent node.

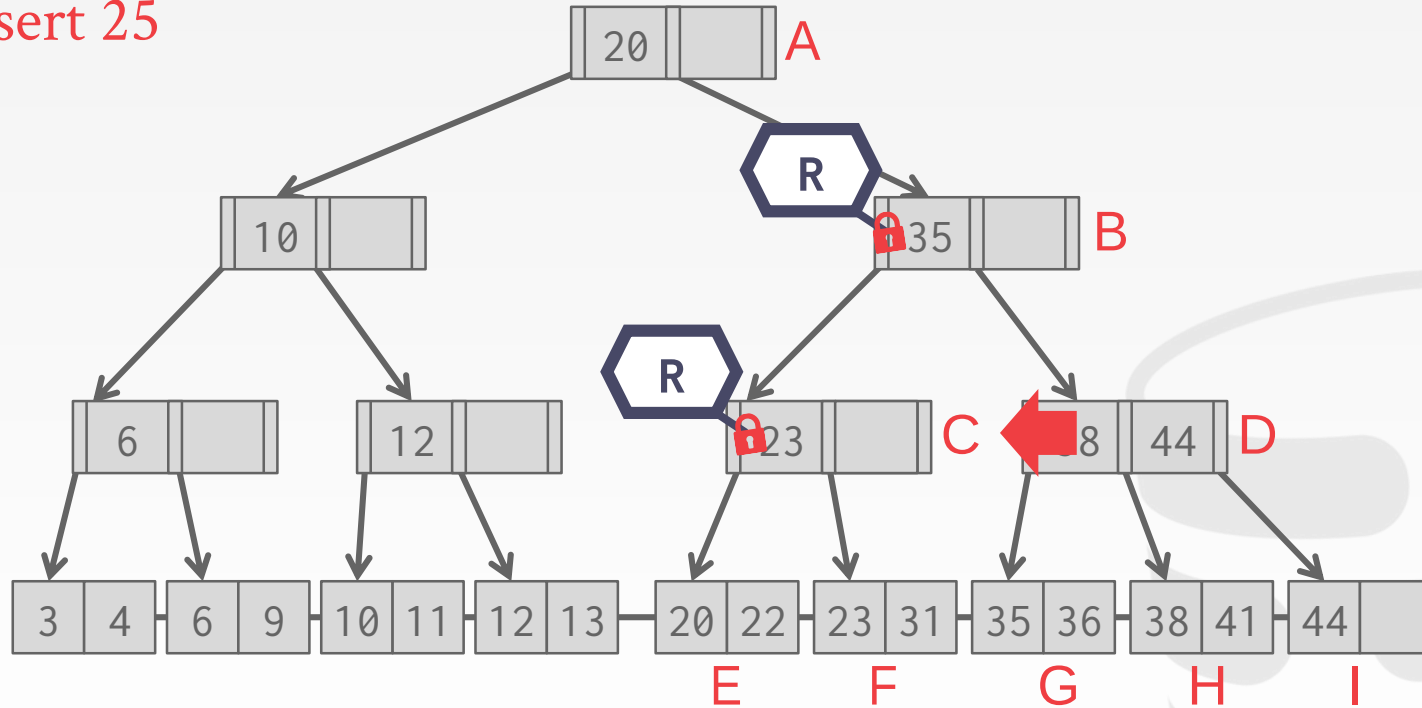
EXAMPLE #4 – INSERT 25

T_1 : Insert 25



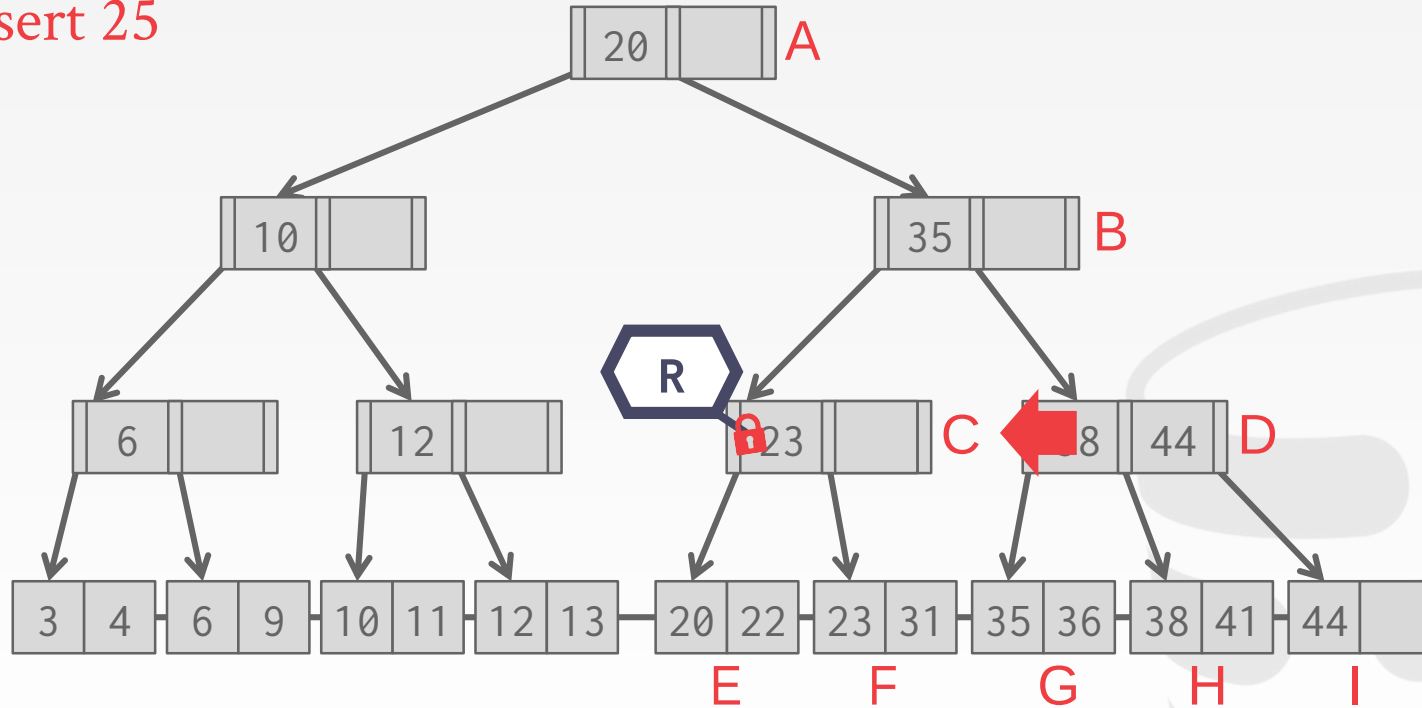
EXAMPLE #4 – INSERT 25

T_1 : Insert 25



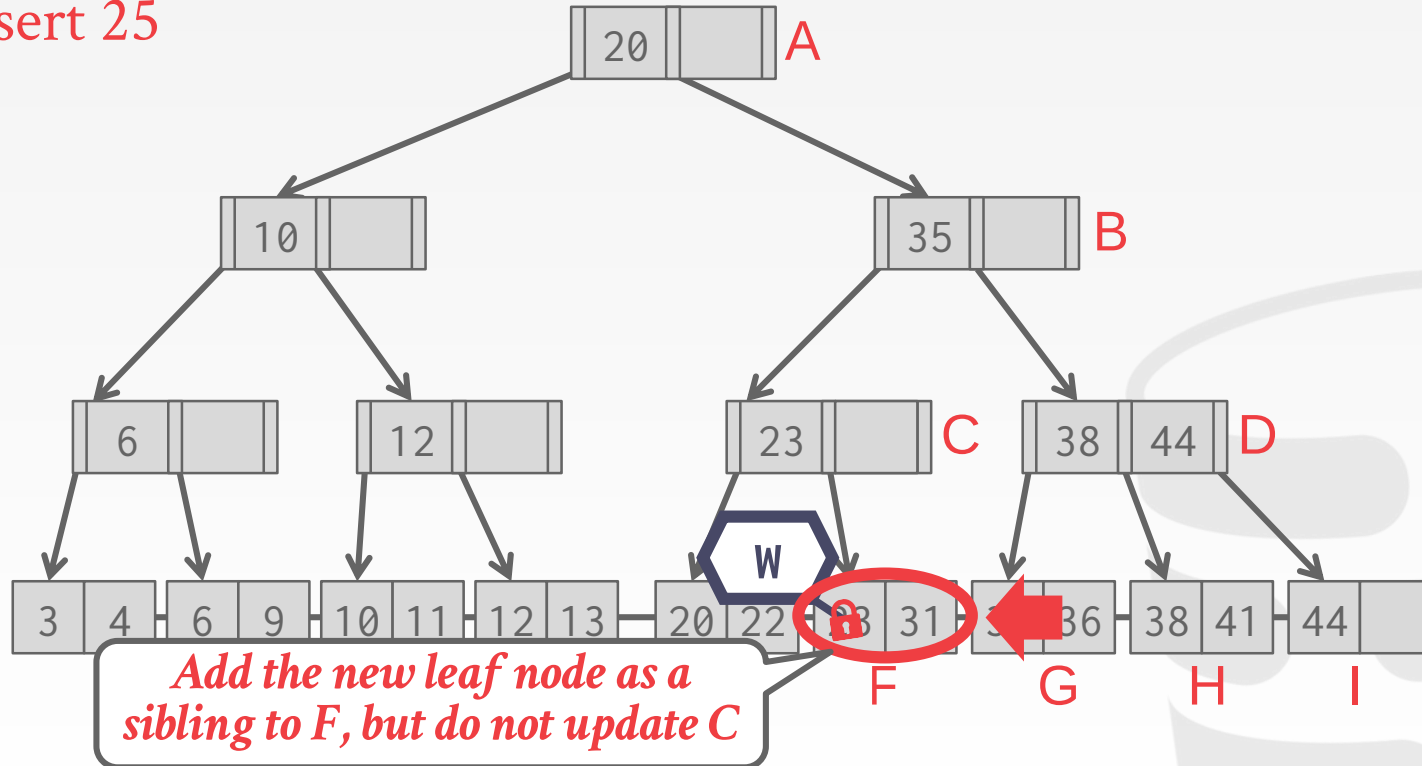
EXAMPLE #4 – INSERT 25

T_1 : Insert 25



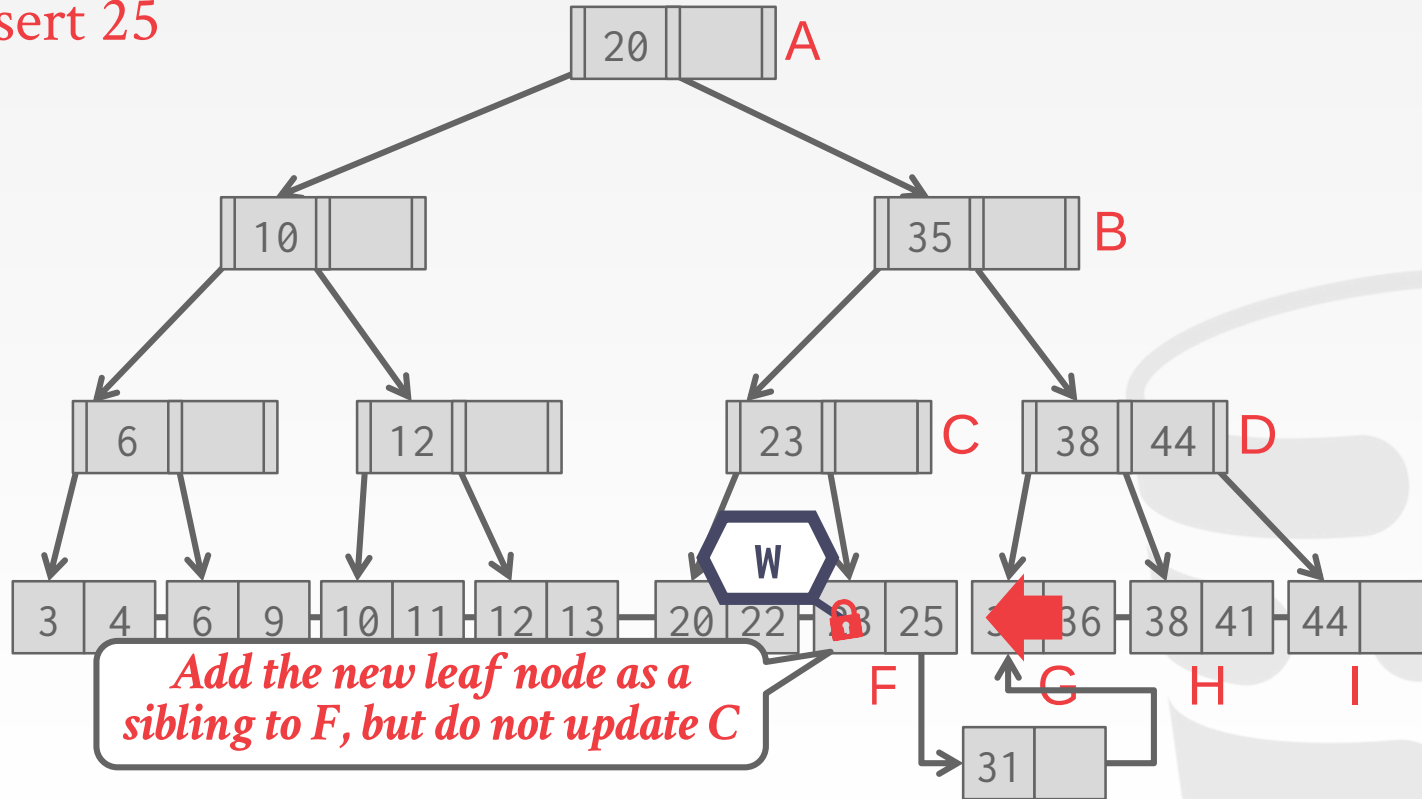
EXAMPLE #4 – INSERT 25

T_1 : Insert 25



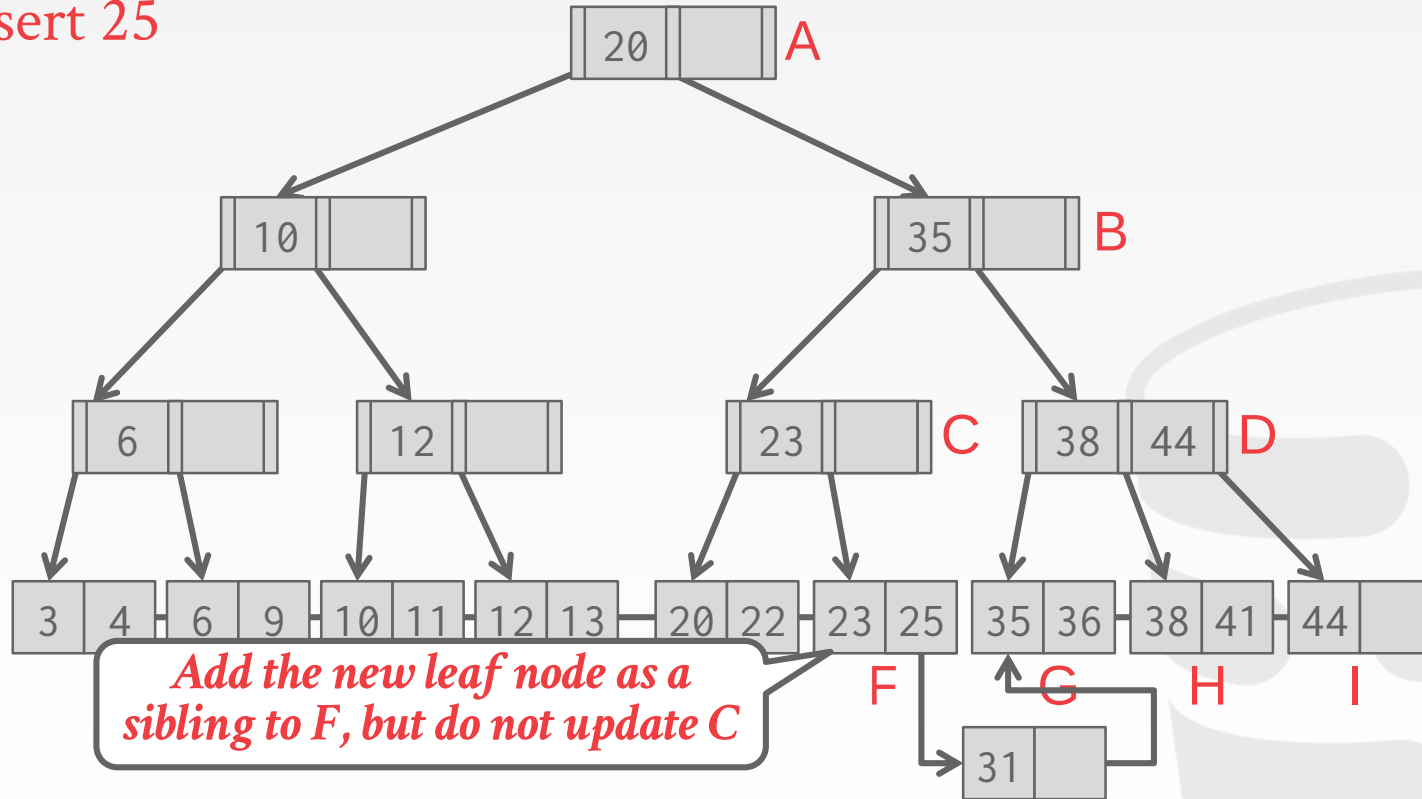
EXAMPLE #4 – INSERT 25

T_1 : Insert 25



EXAMPLE #4 – INSERT 25

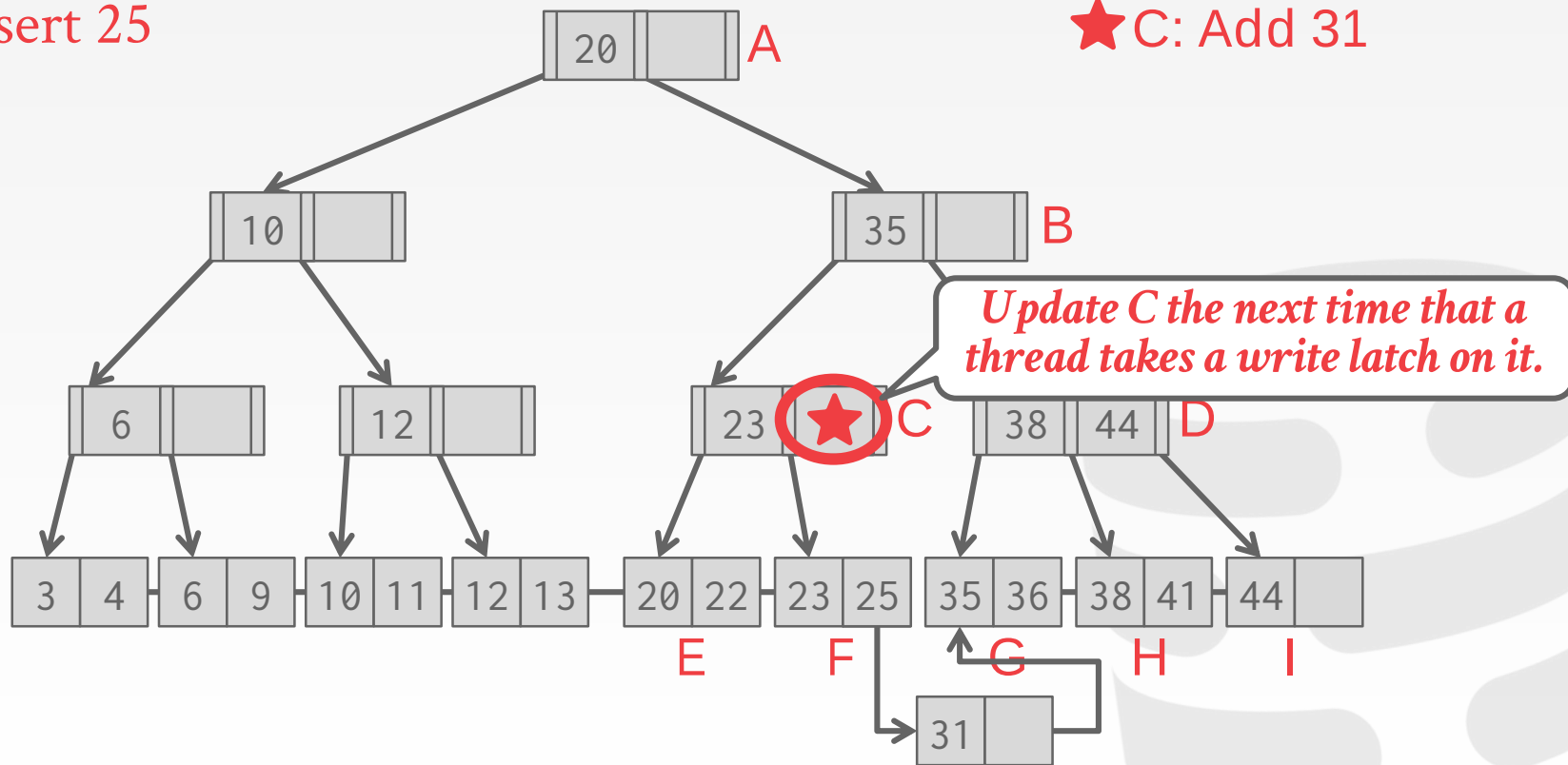
T_1 : Insert 25



EXAMPLE #4 – INSERT 25

T_1 : Insert 25

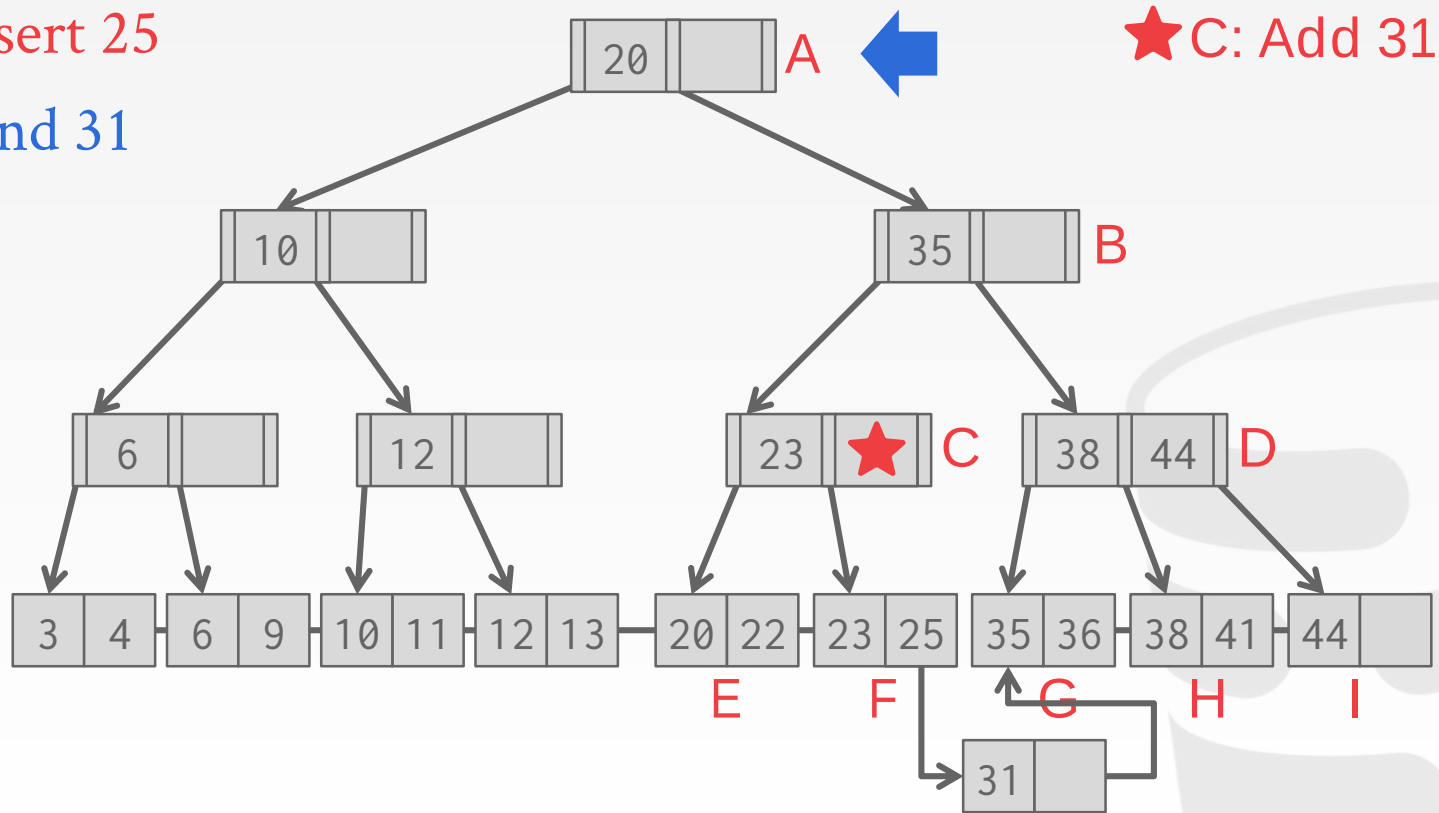
★ C: Add 31



EXAMPLE #4 – INSERT 25

T₁: Insert 25

T₂: Find 31

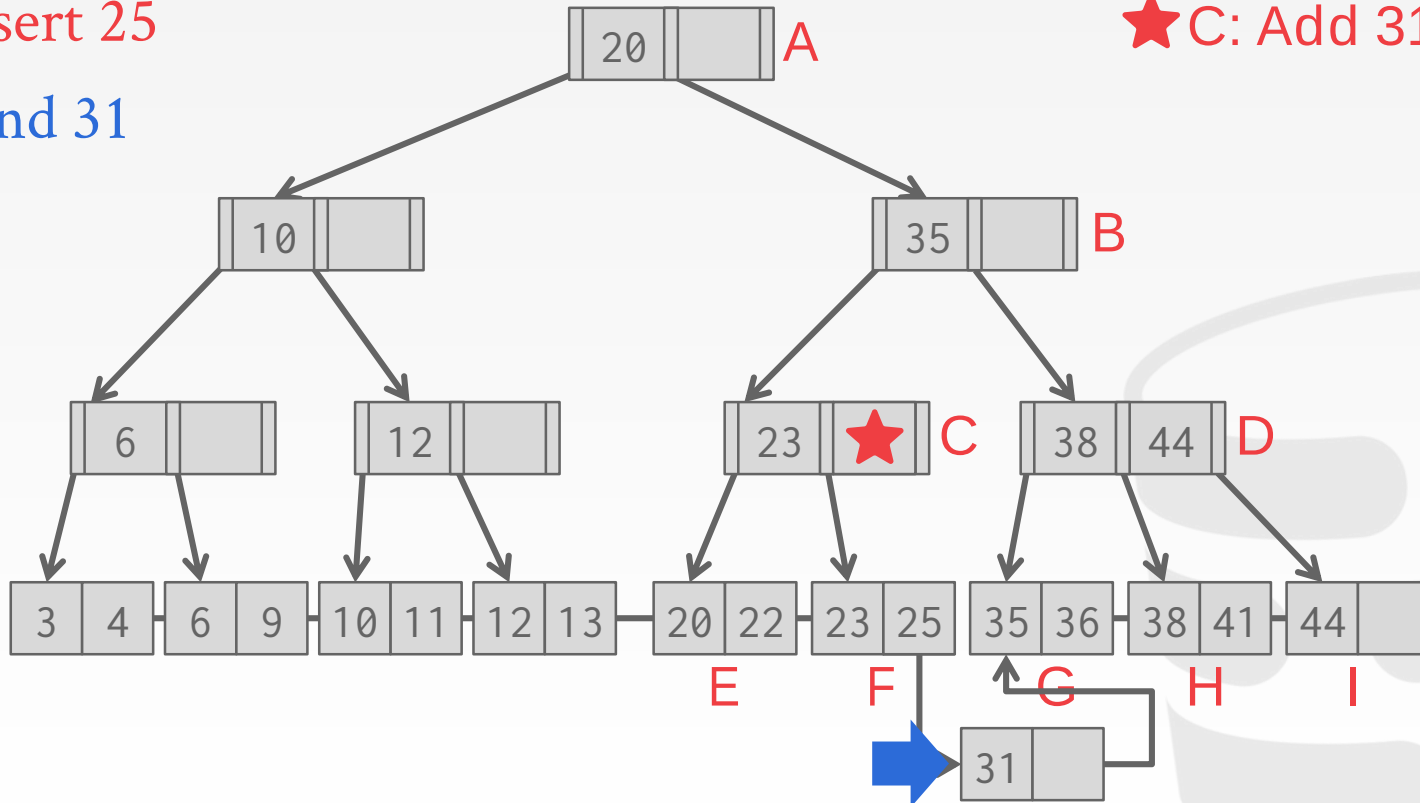


EXAMPLE #4 – INSERT 25

T_1 : Insert 25

T_2 : Find 31

★ C: Add 31

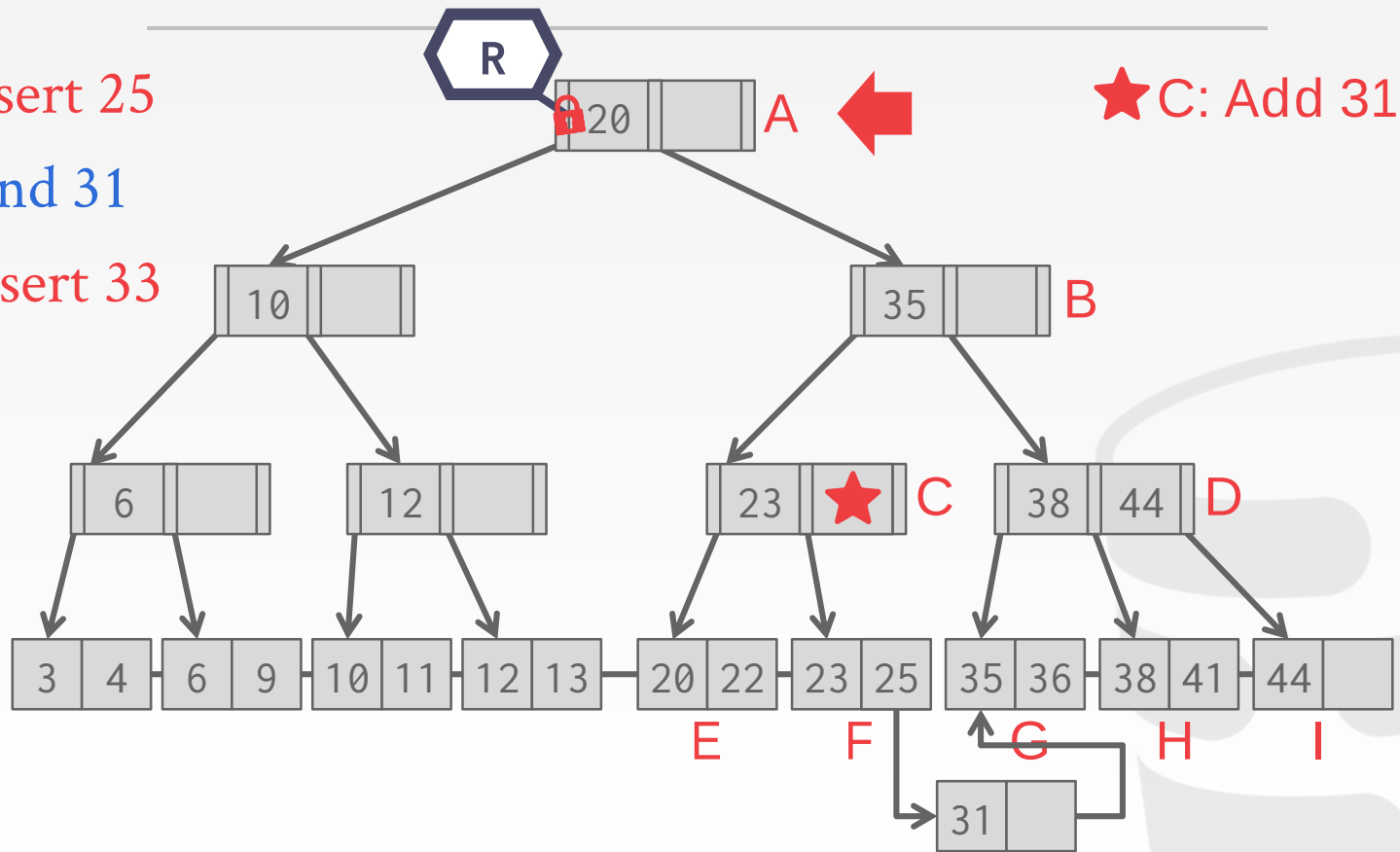


EXAMPLE #4 – INSERT 25

T_1 : Insert 25

T_2 : Find 31

T_3 : Insert 33



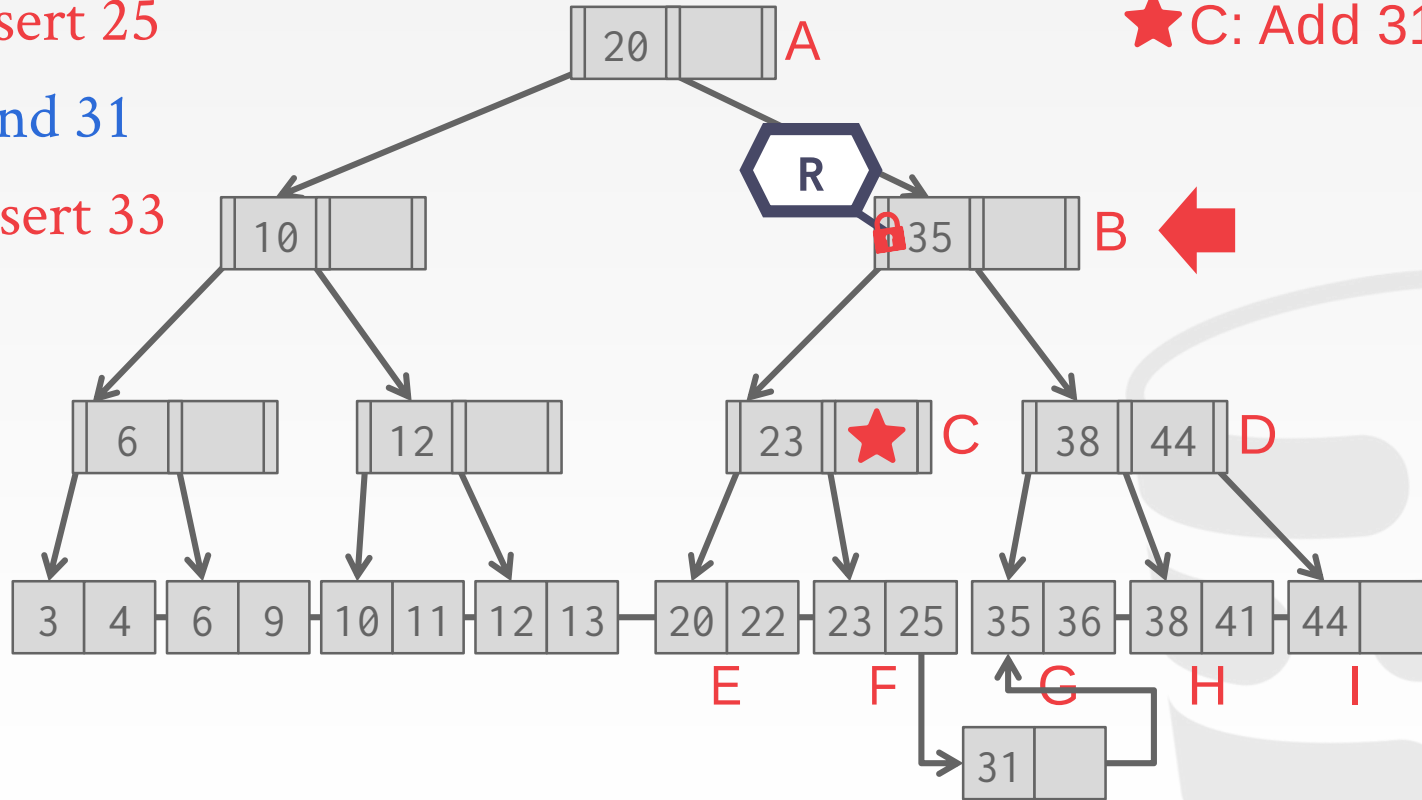
EXAMPLE #4 – INSERT 25

T_1 : Insert 25

T_2 : Find 31

T_3 : Insert 33

★ C: Add 31



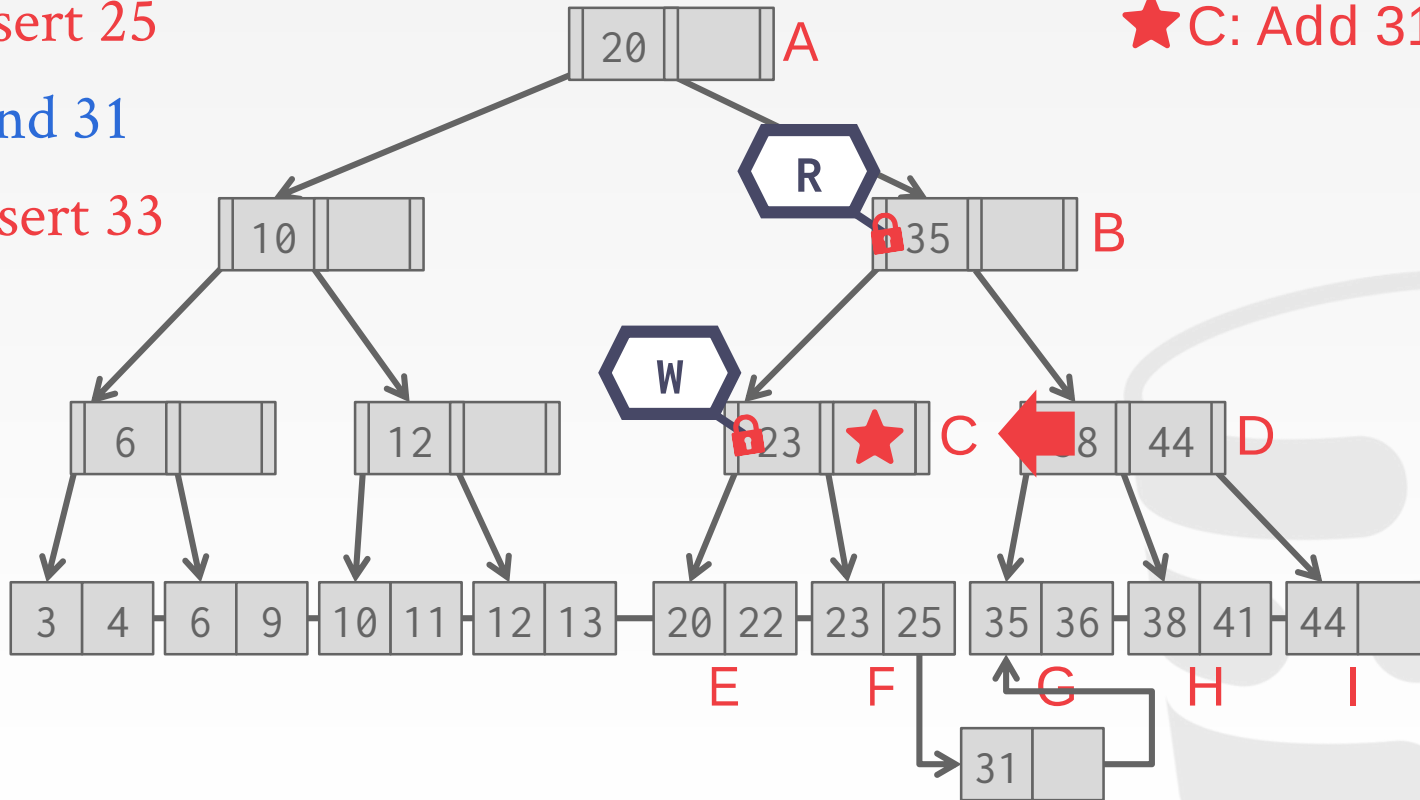
EXAMPLE #4 – INSERT 25

T_1 : Insert 25

T_2 : Find 31

T_3 : Insert 33

★ C: Add 31



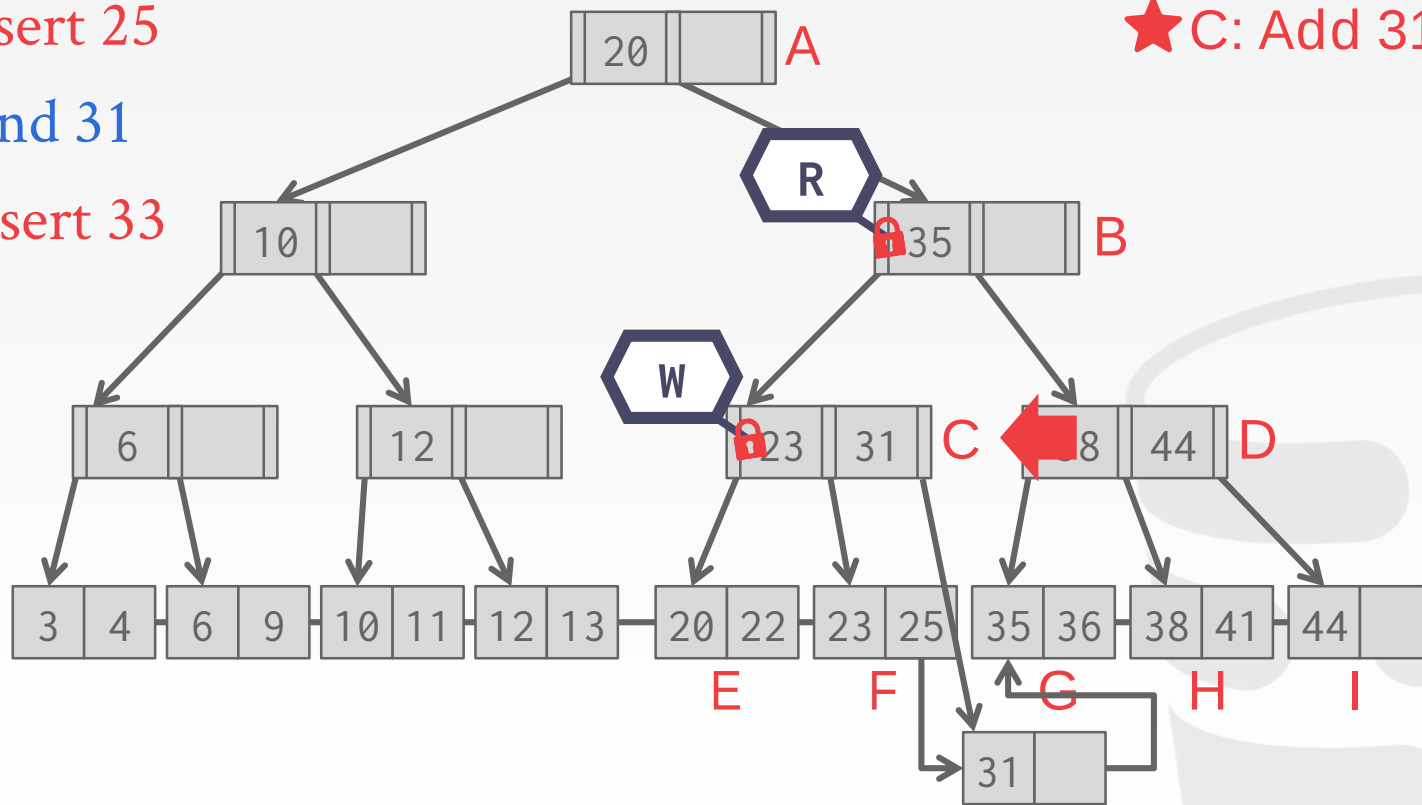
EXAMPLE #4 – INSERT 25

T_1 : Insert 25

T_2 : Find 31

T_3 : Insert 33

★ C: Add 31



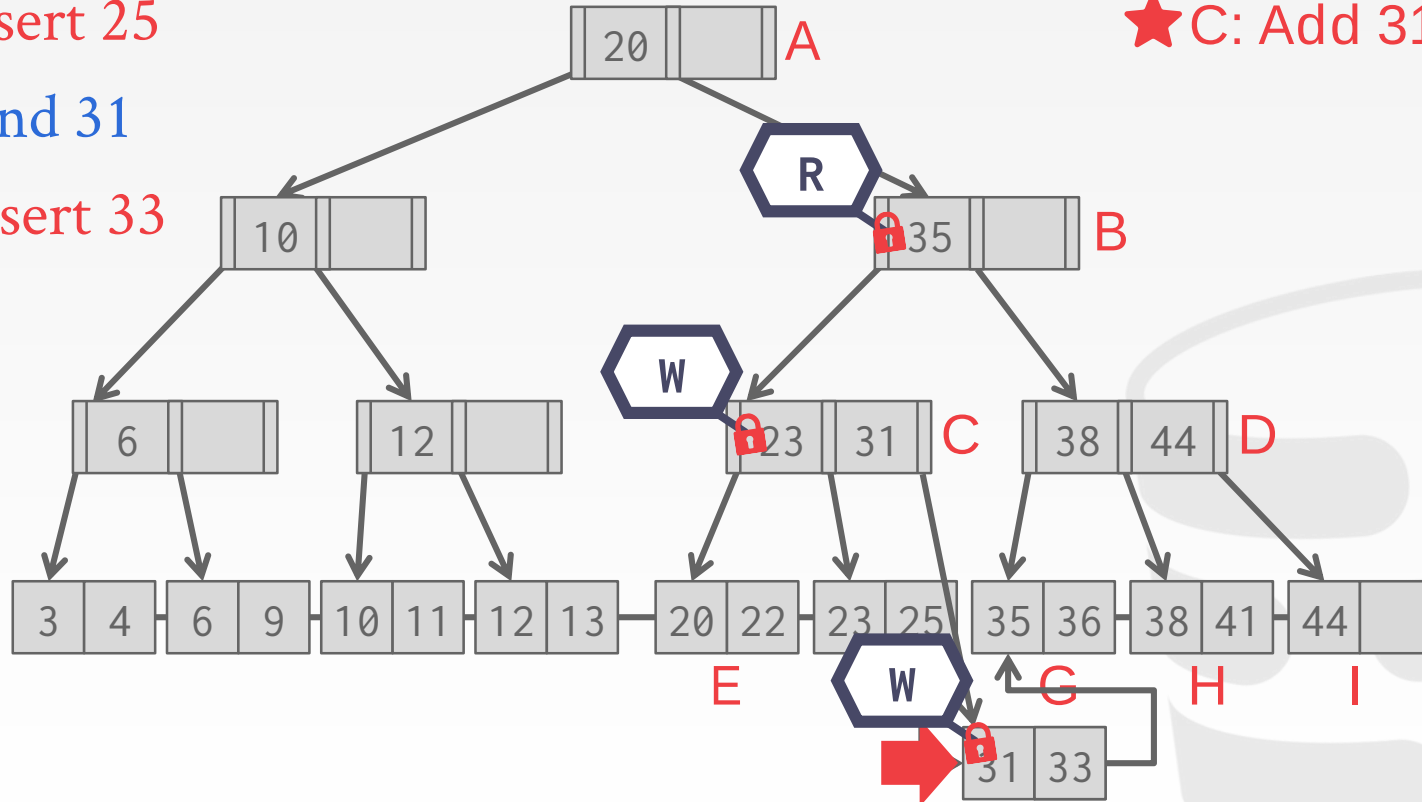
EXAMPLE #4 – INSERT 25

T_1 : Insert 25

T_2 : Find 31

T_3 : Insert 33

★ C: Add 31



CONCLUSION

Making a data structure thread-safe is notoriously difficult in practice.

We focused on B+Trees but the same high-level techniques are applicable to other data structures.

NEXT CLASS

We are finally going to discuss how to execute some queries...



CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

数据库

聚合

内存管理

树索引

排序

查询规划

索引构建

分组

并发控制

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**

称为 **AI 内容创作者**？回复 [添砖加瓦]